

TECHNISCHE UNIVERSITÄT DRESDEN

FAKULTÄT INFORMATIK
INSTITUT FÜR SYSTEMARCHITEKTUR
PROFESSUR FÜR DATENBANKEN
PROF. DR.-ING. WOLFGANG LEHNER
PROF. APL. DR.-ING. DIRK HABICH

Diplomarbeit

zur Erlangung des akademischen Grades
Diplom-Informatiker

Aktive cache-bewusste Hardware-Adaption: Eine Cache-Engine für Trie-basierte Indexstrukturen

Benjamin-Elias Probst
(Geboren am 11. April 1996 in Potsdam)

Hochschullehrer: apl. Prof. Dr.-Ing. Dirk Habich
Betreuer: apl. Prof. Dr.-Ing. Dirk Habich, Dr.-Ing. Alexander Krause

Dresden, 18. Juni 2026

Aufgabenstellung

Aktive cache-bewusste Hardware-Adaption:

Eine Cache-Engine für Trie-basierte Indexstrukturen

Kontext und Motivation

Suchalgorithmen und Indexstrukturen sind ein Kernbaustein moderner In-Memory-Datenbanksysteme. Die Professur für Datenbanken an der TU Dresden adressiert die hardware-nahe Optimierung solcher Systeme. Seit der Einführung des Adaptive Radix Tree wurde eine Vielzahl cache-bewusster Techniken und verwandter Algorithmenfamilien vorgeschlagen — von Knoten-Layout über Prefetching und Succinct-Repräsentationen bis zur Allokation —, doch nutzt fast jede Veröffentlichung eigene Datenstrukturen, verschiedene Lastprofile und inkonsistente Messmethodik. Ergebnisse sind dadurch über Veröffentlichungen hinweg schwer vergleichbar.

Problemstellung

Suchverfahren — von ausgewogenen Suchbäumen (B-/B⁺-Bäume) über Tries und Radix-Bäume bis zu spezialisierten, eigenentwickelten Konstrukten — spannen einen großen *Entwurfsraum* (*design space*) auf: Bereits eine zwei- bzw. dreikomponentige Datenstruktur erlaubt astronomisch viele Entwürfe, und ein konkreter Algorithmus ist ein einzelner Punkt in diesem Raum, beschrieben durch viele einzelne Entwurfsentscheidungen — wie Knoten repräsentiert, durchlaufen, im Speicher angeordnet und alloziert werden [IZA⁺18, IZH⁺18]. Cache-line-bewusstes Verhalten entsteht dabei nicht an einer einzelnen dieser Stellen, sondern an *vielen verschiedenen strukturellen Stellen* eines Algorithmus und aus deren Zusammenspiel. Der Stand der Technik umfasst zahlreiche ausgereifte Verfahren, doch optimiert jedes nur einzelne dieser Stellen und belegt seinen Nutzen in einer eigenen Messumgebung, was den Fortschritt über die Verfahren hinweg schwer vergleichbar macht. Es fehlt ein gemeinsames, generisches Konzept und **Prinzip**, das cache-line-bewusstes Verhalten an den verschiedenen Stellen eines Suchalgorithmus — austauschbar, systematisch und fair — messen und auswerten kann.

Zielsetzung

Ziel der Arbeit ist es, ein solches Prinzip (ein Rahmenwerk) zu finden, zu entwerfen und prototypisch als Experimentier-Framework umzusetzen, das (i) cache-bewusste Suchalgorithmen in austauschbare Entwurfsbestandteile zerlegt, (ii) Algorithmen des Stands der Technik als explizite Konfigurationen (Punkte im Entwurfsraum) rekonstruiert, (iii) cache-line-bewusstes Verhalten sowohl an den einzelnen direkt und indirekt betroffenen Stellen als auch für den gesamten Algorithmus auf gemeinsamer CPU-only-Basis misst und auswertet und (iv) auf Basis der feingliedrigen Messergebnisse den besten Gesamtalgorithmus als eigenständige Binary hinter einem einheitlichen Suchalgorithmus-Interface ausliefert. Als Prüfling für die testweise Eingliederung einer neuen Technologie in den Stand der Technik trägt die Arbeit PRT-ART bei (einen Probst-Redirect-Tree-/ART-Hybriden), der als konzeptionell neuartiger Suchalgorithmus untersucht wird.

Teilaufgaben

1. Systematische Erfassung der allgemeinen Entwurfsbestandteile von Suchalgorithmen — insbesondere der cache-bewussten — und Identifikation der Stellen, an denen cache-line-bewusste und suchalgorithmus-spezifische Entscheidungen wirksam werden.
2. Rekonstruktion von Entwürfen des Stands der Technik — möglichst aus dem Originalcode der jeweiligen Veröffentlichung — (ART, HOT, Masstree, CoCo-trie, START, B²-Tree, Wormhole, SuRF sowie weitere Verfahren) als explizite, vergleichbare Konfigurationen.
3. Entwurf und Implementierung von PRT-ART als Prüfling mit mehreren eigenen Entwurfsbestandteilen, der gegen den erfassten Stand der Technik antritt.
4. Aufbau einer reproduzierbaren Mess-Pipeline (Binary → CSV → LaTeX → Diagramm) mit profilbewusstem YCSB-Workload-Routing über *alle* verfügbaren Workloads und Hardware-Counter-

Erfassung.

5. Systematische, möglichst vollständige Messung über die Konfigurationen in drei verpflichtenden Messreihen — (A) Prüfling gegen den Stand der Technik, (B) systematische Variation der Entwurfsentscheidungen, (C) Merge/Regression alter gegen neue Varianten —, wobei jede Reihe in drei Granularitäten erhoben wird: *Micro-Benchmarking* (jeder einzelne Entwurfsbestandteil innerhalb der Suchalgorithmus-Hülle wird über das gemeinsame Interface seiner Kategorie isoliert gegen Lastprofile gemessen), *Makro-Benchmarking* (die einzelnen Interface-Operationen des Suchalgorithmus gegen Beispiel-Lasten, z. B. Einfügen, Punktsuche, Löschen) und *Gesamt-Benchmarking* (etablierte YCSB-Lastprofile über jede Rekombination der Entwurfsbestandteile).
6. Auswertung, an welchen Stellen und in welchen Kombinationen cache-line-bewusste Entscheidungen für welche datentyp-spezifischen Lastmuster vorteilhaft oder nachteilig sind — ermittelt aus der Rangbildung über die drei Benchmarking-Granularitäten und aufbereitet als auswertbare Diagramme und Tabellen.

Methodik und Evaluation

Die Evaluation umfasst neben dem CPU-only-Kern auch SIMD- und nebenläufige Multi-Threading-Pfade und vergleicht die Konfigurationen gegen etablierte Baselines, darunter eine nicht-baumartige Hash-Tabelle (`flat_hash_map`) als Gegenprobe innerhalb derselben Gattung. Als Datengrundlage dienen reale String-Datensätze unterschiedlicher Charakteristik — etwa `url`, `dna`, `protein`, `xml`, `tpcds-id` und `trec-terms` — unter variierten Datenvolumina, Keylängen, Präfixüberlappungen, lokaler Dichte, Skew, Treffer-/Fehlerquoten und Value-Größen. Gemessen werden Exact-/Prefix-Lookup, Prefix-Enumeration, Einfügen/Aktualisieren/Löschen, p50/p95/p99-Latenz, Durchsatz, Bytes pro Key sowie Hardwarezähler (Cycles, Instructions, L1/L2/LLC- und dTLB-Misses, Branch-Mispredictions). Zunächst wird das Single-Thread-Verhalten untersucht, danach folgen leseparallele Multi-Thread-Szenarien.

Faire Vergleichbarkeit wird durch feste Regeln gesichert: ein gemeinsamer Minimalmodus (Common-Denominator) wird vom PRT-ART-Native-Modus getrennt, alle Verfahren erhalten identische Schlüsselräume und Seeds, fehlende Fähigkeiten werden als N/A ausgewiesen statt verdeckt emuliert, und Compiler, Flags, ISA-Pfad, Allokator sowie Commit-Hash jeder Baseline werden protokolliert. Für die Reproduzierbarkeit wird jede Konfiguration in mehreren getrennten Läufen vermessen, deren Streuung Teil der Aussage bleibt (Perzentile über HDR-Histogramme statt Mittelung); jeder Datensatz trägt eine Akte aus Quelle, Prüfsumme und Seed-Regel.

Umfangsabgrenzung

Der Kernbeitrag ist CPU-fokussiert mit SIMD-Erweiterungen und Multi-Threading, beschränkt sich dabei aber auf leseparallele Verfahren mit serialisiertem Schreiben (multi-read, single-write). Die GPU liegt vorerst außerhalb des Umfangs (zukünftig erweiterbar); volle Schreibparallelität und eine Engine-/Optimierer-Integration sind optionale Erweiterungen.

Selbstständigkeitserklärung

Hiermit erkläre ich, Benjamin-Elias Probst (Mat.-Nr. 4510512, geboren am 11. April 1996 in Potsdam), dass ich die von mir am heutigen Tag dem Prüfungsausschuss der Fakultät Informatik eingereichte Diplomarbeit zum Thema:

*Aktive cache-bewusste Hardware-Adaption:
Eine Cache-Engine für Trie-basierte Indexstrukturen*

vollkommen selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt sowie Zitate kenntlich gemacht habe.

Dresden, den 18. Juni 2026

Benjamin-Elias Probst
Mat.-Nr. 4510512

Kurzfassung

Suchalgorithmen und Indexstrukturen sind ein Kernbaustein moderner In-Memory-Datenbanken. Seit dem Adaptive Radix Tree wurde eine Vielzahl cache-bewusster Optimierungen vorgeschlagen—von Knoten-Layout über Prefetching und Succinct-Repräsentationen bis zur Allokation—, doch wurden sie nie in einem einzigen, fairen Messrahmen kombiniert und verglichen: Jede Veröffentlichung nutzt eigene Datenstrukturen, Lastprofile und Messmethodik, wodurch Ergebnisse über Veröffentlichungen hinweg schwer vergleichbar sind. Diese Arbeit stellt eine dreischichtige Cache-Engine-Architektur (CacheEngine → ExecutionEngine → SearchEngine) vor, die cache-bewusste Suchalgorithmen in austauschbare Entwurfsdimensionen eines großen Entwurfsraums (*design space*) zerlegt [IZA⁺18, IZH⁺18]. Strukturell verschiedene Verfahren—von baumartigen Indizes (Tries, B⁺-Bäume) bis zu nicht-baumartigen Hash-Tabellen—werden als Punkte desselben Entwurfsraums hinter einem einheitlichen Interface ausgedrückt; nicht-baumartige Verfahren belegen dabei ein eingeschränktes Subset der Dimensionen. Algorithmen des Stands der Technik werden als explizite Konfigurationen rekonstruiert und auf drei Granularitäten gemessen: einzelne Entwurfsentscheidungen, einzelne Interface-Operationen und ganze Algorithmen unter YCSB-Lastprofilen. Als Prüfling trägt zu dieser Arbeit der PRT-ART bei, ein Probst-Redirect-Tree-/ART-Hybrid mit acht eigenen Bausteinschichten; ein Mess-Treiber mit profilbewusstem Workload-Routing bewertet ihn unter drei verpflichtenden Messreihen gegen dreißig rekonstruierte Stand-der-Technik-Verfahren (acht etablierte Referenzverfahren und weitere) und liefert die beste Kombination als eigenständige Binary aus.

Inhaltsverzeichnis

1	Einleitung	5
1.1	Motivation	5
1.2	Problemstellung	5
1.3	Forschungsfragen	6
1.4	Zielsetzung und Beiträge	7
1.5	Aufbau der Arbeit	8
2	Grundlagen	9
2.1	Cache-Hierarchie und Cache-Bewusstheit	9
2.1.1	Speicher- und Cache-Ebenen	9
2.1.2	Cache-Line-Größen: statisch und dynamisch	9
2.1.3	Adressübersetzung: TLB und dTLB	10
2.1.4	Cache-Kohärenz, Schreib- und Update-Verhalten je ISA	10
2.1.5	Cache-bewusst und cache-oblivious	10
2.2	Klassen von Suchstrukturen	11
2.2.1	Etablierte Untergliederung der Suchalgorithmen	11
2.2.1.1	Vergleichsbasierte Suchbäume	11
2.2.1.2	Digitale und präfixbasierte Strukturen	12
2.2.1.3	Hashing	12
2.2.1.4	Räumliche Strukturen	12
2.2.1.5	Flache Verfahren und theoretische Exoten	12
2.2.2	Untergliederung der Container	12
2.2.3	Begriffliche Einordnung im vorliegenden Framework	13
2.3	Einheitliche Vergleichsinterfaces	13
2.3.1	Das <code>std::map</code> -Interface	13
2.3.2	Das <code>std::vector</code> -Interface	13
2.3.3	Ebenen der Dynamik	14
2.4	Lasten und Workloads	14
2.4.1	Begriff: Last, Workload und Ziel der Variation	14
2.4.2	Workload-Frameworks	14
2.5	Wissenschaftliches Messen	15
2.5.1	Gütekriterien wissenschaftlichen Messens	15
2.5.2	Verwendete Mess-Muster und -Konzepte	15
2.6	C++23-Metaprogrammierung und Design-Pattern	16
2.6.1	Klassische Design-Pattern	16
2.6.2	Metaprogrammier-Mittel	16

2.6.3	Neue und unbenannte Metaprogrammier-Pattern	16
3	Stand der Technik	17
3.1	Überblick und Sezierungs-Prinzip	17
3.1.1	Der Korpus nach Herkunfts-Clustern	17
3.2	Cache-Konzepte als Instanzen	18
3.2.1	Speicher- und Cache-Ebenen	18
3.2.2	Cache-Line-bewusstes Layout	18
3.2.3	Adressübersetzung: TLB und dTLB	18
3.2.4	Schreib-Protokolle, Update-Heuristiken und Inklusions-Politik	19
3.2.5	Cache-bewusste und cache-oblivious Entwürfe	19
3.3	Achsen-Seizierung der Suchverfahren	19
3.3.1	Such- und Navigations-Organe	20
3.3.2	Speicher-, Allokations- und Wert-Organe	21
3.3.3	Hardware-nahe Organe	21
3.3.4	Nebenläufigkeit, I/O, Migration und Pufferung	21
3.3.5	Index-Organisation und Filter	22
3.4	Workload-Frameworks als Instanzen	23
3.4.1	Benchmark-Frameworks	23
3.4.2	Lastprofile und Workload-Affinität der Verfahren	23
3.4.3	Bias und Vergleichbarkeit	24
3.5	Mess-Technologien als Instanzen	26
3.5.1	Erhebungs-Technologien	26
3.5.2	Statistische Methodik	26
3.5.3	TU-Dresden-Hardware-Messmethodik	26
3.5.4	Mess-Kategorien und Brücke zur eigenen Apparatur	26
3.6	Architektur-Design und Software-Engineering von Suchstrukturen	27
3.6.1	Datenstrukturen als Entwurfsraum	27
3.6.2	Selbst-designende und gelernte Indizes	27
3.6.3	Design-Pattern-getriebene Architektur als Beitrag	27
3.7	Forschungslücke	28
4	Konzept und Architektur	29
4.1	Das Achsen-Bibliothek-Framework	29
4.2	Vier-Subsystem-Trennung (M-Modell)	30
4.3	Drei-Schichten-Hierarchie	30
4.4	ABI-stabiles C++23-Modul-Interface	31
4.5	Die Bausteine-Achsen	31
4.6	PRT-ART als Prüfling	31
4.7	CacheEngineBuilder und Drei-Stufen-Prüfung	31
5	Implementierung	33
5.1	Drei-Repository-Architektur	33

5.2	Adapter für Stand-der-Technik-Algorithmen und Allokatoren	33
5.3	Permutations-Codegenerierung und Flag-System	34
5.4	Concept-/CRTP-Realisierung der Achsen	34
5.5	Nebenläufigkeit und Speicherfreigabe	34
5.6	Telemetrie gegen Cache-Kohärenz-Effekte	34
5.7	Mess-Pipeline	34
6	Evaluationsmethodik	37
6.1	Drei Messreihen	37
6.2	ExperimentDriver-Phasen	37
6.3	Hypothesen	38
6.4	Workloads und Datensätze	38
6.5	Versuchsplattformen	38
6.6	Fairness und Reproduzierbarkeit	39
6.7	Permutations-Explosion und Reduktion	39
7	Ergebnisse und Auswertung	41
7.1	Auswertungs-Pipeline	41
7.2	Messreihe A: PRT-ART vs. Stand der Technik	41
7.3	Messreihe B: Cache-Engine-Permutationen	41
7.4	Messreihe C: Full Join	41
7.5	Achsen-Sensitivitätsanalyse	41
7.6	Diskussion	41
8	Fazit und Ausblick	43
8.1	Beantwortung der Forschungsfragen	43
8.2	Limitierungen	44
8.3	Ausblick	44
	Literaturverzeichnis	45
	Abbildungsverzeichnis	53
	Tabellenverzeichnis	55
A	Vollständige Messreihen	57
A.1	Kartesische Smoke-Messreihe (43 Permutationen, echte DLL-Messung)	57
A.2	Bias-Bruch-Matrix	57
A.3	Latenz-Surfaces je Schnittstellen-Funktion	57
A.4	Achsen-Austauschbarkeit	67
A.5	Limitierungen	75
B	Code-Struktur	77
C	Glossar	79

D	Bausteine-Matrix	81
D.1	T0 Suchalgorithmen-Achse T0 (Such-Strukturen und Such-Methoden)	81
D.2	T1 Cache-Speicher-Traversierung (Fanout-Lookup)	83
D.3	T2 Slot-zu-Offset-Mapping (Traversal-Sublayer)	84
D.4	T3 Pfadkompression	85
D.5	T4 Knotentypen (Node-Type-Achse)	86
D.6	T5 Speicher-Layout (Memory Layout)	87
D.7	T6 Allokator-Achse (T6)	89
D.8	T7 Prefetch-Strategie	91
D.9	T8 Nebenläufigkeits-Synchronisations-Achse (Concurrency Axis T8)	92
D.10	T9 Serialisierung (Kodierung und Kompression)	93
D.11	T10 Telemetrie und Messinstrumentierung (axis_11)	94
D.12	T11 Wert-Handle (Value-Handle)	95
D.13	T12 Instruktionssatz-Architektur (ISA)	96
D.14	T13 Index-Organisation	97
D.15	T14 I/O-Dispatch-Achse	98
D.16	T15 Migrations-Strategie-Achse	99
D.17	T16 Filter-Achse	101
D.18	T17 Pufferung Q1 (Operations-Puffer)	102
D.19	T18 Pufferung Q2 (Flush-Policy)	104
D.20	Build-PG Seitentyp-Achse (PAGE-TYPE)	105
D.21	Build-SIMD: SIMD-Erweiterungs-Achse (Hardware-Beschleuniger)	106
D.22	Build-HW — Plattform-Hardware-Strategie	107
D.23	A-Korpus Allokator-Korpus (A01-A23)	108
E	Architekturentscheidungen	113
F	Vergleichsinterfaces: <code>std::map</code> und <code>std::vector</code>	115
F.1	<code>std::map</code> (geordnet-assoziativ)	115
F.2	<code>std::vector</code> (Sequenz)	118
F.3	Zusammengesetzte Funktionen und Konformitäts-Teilmenge	120

1 Einleitung

1.1 Motivation

Suchalgorithmen und Indexstrukturen sind das Rückgrat moderner Hauptspeicher-Datenbanken; trie-basierte Indizes und cache-line optimierte Verarbeitung stehen im Zentrum dieser Arbeit. Seit dem Adaptive Radix Tree (ART) von Leis et al. [LKN13] im Jahr 2013 hat sich die Forschung in mehrere Richtungen ausdifferenziert: Trie-Höhen-Optimierung (HOT [BZP⁺18]), hybride B⁺-Strukturen (Masstree [MKM12], B²-Baum [SSL⁺21]), succinct-Repräsentationen (CoCo-Trie [BFTV24], SuRF [ZLL⁺18]) und selbstoptimierende Ansätze (START [FJKN20]).

Parallel dazu hat sich ein eigener Forschungskorpus zu Cache-Bewusstheit, Prefetching und Allokationsstrategien entwickelt. Heuristiken wie Cache-Sensitive Search Trees, cache-oblivious Layouts und hierarchisches Prefetching adressieren jeweils einen separat betrachteten Aspekt der Suchalgorithmus-Architektur. Aufgrund der fundamentalen Vorkommnisse auf cache-line optimiertem Verhalten basierender Code, ist eine Segmentierung der Suchalgorithmus-Hüllen-Bestandteile in Achsen sinnvoll. Dies entspricht der rückwärts Abstraktion von bekannten Suchalgorithmus-Bestandteilen in wiederverwendbare Module über die wichtigsten Paper des Standes der Technik hinweg.

Diese Arbeit schließt diese Lücke mit einem Rahmenwerk, dessen Bausteine als austauschbare Entwurfsdimensionen eines großen Entwurfsraums (*design space*) aufgefasst werden [IZA⁺18, IZH⁺18]. Strukturell verschiedene Suchverfahren — von baumartigen Indizes bis zu nicht-baumartigen Hash-Tabellen, sowie Spezialeigenschaften wie cache-lines — werden so als Punkte desselben Entwurfsraums vergleichbar; nicht-baumartige Verfahren belegen dabei ein eingeschränktes Subset der Dimensionen über demselben Suchalgorithmus-Interface. Als Prüfling für die Stand-der-Technik-Zugehörigkeit dient PRT-ART (ein Probst-Redirect-Tree-/ART-Hybrid) als eine abstrakte unvollständige Ansammlung neuartiger Suchalgorithmus-Bestandteile.

1.2 Problemstellung

Bestehende cache-bewusste Index-Entwürfe optimieren ein oder wenige Aspekte isoliert und berichten Ergebnisse, die sich über Veröffentlichungen hinweg schwer vergleichen lassen, da jede Arbeit eigene Datenstrukturen, Workloads und Mess-Methodiken verwendet. Es fehlt ein gemeinsamer Rahmen, der (i) solche Entwürfe in austauschbare Achsen zerlegt, (ii) publizierte Algorithmen als explizite Konfigurationen rekonstruiert und (iii) sowohl einzelne Achsen als auch ganze Algorithmen auf einer einheitlichen, fairen Basis vermessen kann.

1.3 Forschungsfragen

Aus der Aufgabenstellung und den Betreuungs-Terminen ergeben sich eine mit dem Betreuer eingefrorene Hauptfrage (FF0) und vier messbare Teilfragen (FF1–FF4); ergänzende Aspekte sind als *Teilfragen* zugeordnet.

FF0 (Hauptfrage) Wie stark verbessern *aktive, messgetriebene* cache-bewusste Entscheidungen — Seitentyp, Speicher-Layout, Value-Ablage und Prefetching — die Leistung trie-basierter Suchstrukturen gegenüber *passiven, statisch ausgelegten* Varianten auf unterschiedlichen CPU-Plattformen (Hybrid-CPU, Sapphire Rapids)?

FF1 (Komposition) Wie lässt sich ein Suchalgorithmus-Entwurf kanonisch in orthogonale Achsen zerlegen und in einer Permutationsmatrix systematisieren, sodass die Stand-der-Technik-Verfahren möglichst aus ihrem Originalcode bias-frei als Profil-Konfigurationen rekonstruierbar und beliebig permutierbar werden? *Teilfrage:* Welche Achsen-Kopplungen (Knotentyp $\times$ SIMD $\times$ Pfadkompression) begrenzen die Isolation einzelner Achsen, und wie verhält sich dieselbe achsen-konstante Struktur im Cache-Update- und Cache-Line-Füllverhalten, wenn allein die ISA-/SIMD-Achse (SSE2/AVX2/AVX-512, NEON/SVE2, RVV) variiert wird?

FF2 (Mess-Methodik) Wie muss eine reproduzierbare Mess-Pipeline mit Hardware-Zähler-Erhebung gestaltet sein, damit jede Achsen-Permutation fair (*ceteris paribus*, gleiche Compiler- und Flag-Basis) in drei Messreihen (A–C) über drei Granularitäten (Micro-, Makro- und Gesamt-Benchmarking) gemessen wird, und unter welchen plattform-kalibrierten Kriterien gilt eine Plattform als erfolgreich verifiziert? *Teilfrage:* Wie lassen sich widersprüchliche Literaturbefunde zur optimalen Knoten-/Cache-Line-Größe (eine Cache-Line bei CSS/CSB⁺ gegenüber bis zu sechzehn bei Hankins/Patel) durch achsen-isolierte, *ceteris-paribus* permutierte Messung bias-frei nachmessen, statt sie aus inkompatiblen Original-Setups zu übernehmen?

FF3 (Prüfling-Vergleich) Gewinnt der Prüfling PRT-ART gegenüber den acht Rang-1-SOTA-Entwürfen (ART, HOT, Masstree, CoCo-Trie, START, B²-Baum, Wormhole, SuRF)¹ und der Rang-2/3-SOTA *nicht* durch asymptotische Neuheit, sondern durch bessere Mikroarchitektur-Eigenschaften (höhere Cache-Line-Auslastung, weniger LLC- und dTLB-Misses, geringere Branch-Kosten, kleinere heiße Suchpfade) bei Exact-Lookup, Prefix-Lookup und Prefix-Enumeration — gemessen unter den drei Pflicht-Messreihen A (Prüfling vs. Stand der Technik), B (systematische Variation der Entwurfsentscheidungen) und C (Merge/Regression alt gegen neu)? *Teilfrage:* Welche lokale Seitendarstellung (8-/16-Bit-Punktseite gegenüber -Kompaktseite) ist je Verzweigungspunkt am günstigsten, und nach welchen plattform-kalibrierten Umschaltregeln — einschließlich der Reihenfolge $\text{Array}_{256} \rightarrow \text{Array}_{65535} \rightarrow \text{Vector}\langle u8, u8 \rangle \rightarrow \text{Vector}\langle u16, u16 \rangle$ je Füllstand und Suchtyp — soll adaptiv gewechselt werden?

FF4 (Compile-Time vs. Laufzeit) Welche Achsen-Eigenschaften müssen zur Übersetzungszeit fixiert werden, um plattform-optimierte, provenienz-nachweisbare Binaries je Permutation zu erzeugen,

¹Die *Ränge* ordnen die Stand-der-Technik-Veröffentlichungen nach Zentralität für diese Arbeit: Rang-1 umfasst die zehn wichtigsten Kern-Arbeiten — darunter die acht eigenständigen Suchentwürfe sowie ARTSync (P08) und LOUDS (P09), die als Organe eingehen —, Rang-2 die cache-bewussten Layout- und B⁺-Vorarbeiten und Rang-3 die Prefetching-, Synchronisations- und TU-Dresden-Hardware-Arbeiten.

und welche können bzw. müssen zur Laufzeit gewählt werden (bit-codierter Permutations-Identifizier, adaptive Layout-Selection), ohne die wissenschaftliche Vergleichbarkeit (gleiche Compiler- und Flag-Basis) zu verletzen?

1.4 Zielsetzung und Beiträge

Die Zielsetzung folgt der Aufgabenstellung in vier Schritten: cache-bewusste Suchalgorithmen in austauschbare Entwurfsbestandteile zerlegen, Stand-der-Technik-Entwürfe als explizite Konfigurationen rekonstruieren, cache-line-bewusstes Verhalten sowohl je Bestandteil als auch für den ganzen Algorithmus auf gemeinsamer CPU-Basis messen und den besten Gesamtalgorithmus als eigenständige Binary ausliefern. Der Code der Diplomarbeit ist dabei der *formale Bediener* der Cache-Engine: Er lädt den Prüfling PRT-ART, kompiliert ihn per Metaprogrammierung gegen die Kernbestandteile der Cache-Engine, liefert ABI-stabile *Lebewesen* (Suchalgorithmus-Instanzen) aus und treibt sie über das Prüf-Dock zur Messung. Architektonisch trennt das Modell drei Ebenen — ein Gattungs-Interface (SearchAlgorithm, Container, Graph), darunter Lebewesen-Unterklassen mit festem Achsensatz, darunter die neunzehn Achsen als Organe — und vier Subsysteme (Mess-Treiber, CacheEngineBuilder, CacheEngine, Prüfling); in früheren Entwürfen wurde diese Schichtung als Kette `ExecutionEngine → SearchEngine → SearchEngineType` gelesen. Das Konzept der Achsen-Architektur selbst ist aus einer Vorarbeit des Autors übernommen (Comdare / BEP Venture UG, ursprünglich zur Optimierung von Deduplikations-Verfahren in der UltiHash-Datenbank); der Beitrag dieser Arbeit ist seine Übertragung auf cache-bewusste Suchstrukturen und seine mess-orientierte Realisierung. Die zentralen Beiträge sind:

- Eine dreischichtige Architektur (`CacheEngine → ExecutionEngine → SearchEngine`) mit ABI-stabilen C++23-Modul-Schnittstellen und variadisch templatierten Hybrid-API-Spezialisierungen (`1 Parameter ⇒ std::vector-API`; `2 Parameter ⇒ std::map-API`; `N > 2 ⇒ std::map<K, std::tuple<V1...VN>>`).
- Eine neunzehn-Achsen-Permutationsmatrix mit `std::variant`-Bausteinen und Compile-Time-Fallback zwischen Prüfling-Stack und Stand-der-Technik-Stack.
- Dreißig SOTA-Algorithmus-Profile als persistierte XML-Konfigurationen (8 Rang-1 + 22 Rang-2/3), die der `ExperimentDriver` automatisch in vorkompilierte C++23-Module überführt.
- PRT-ART als Prüfling mit acht eigenen Bausteinschichten (4+2-Pool-Allokator, optimistisches Lock-Coupling mit reservierten Blöcken, MultiLevel-Layout, pfadorientiertes Prefetch, Density-Tracker, Hypothesen-Metriken H1/H2/H3, Inline/External/ChainRef-ValueHandles, Signaling-Bit-Serialisierung).
- Ein Mess-Treiber mit Defined-/Full-Modus-Unterscheidung, profilbewusstem Workload-Routing und drei Granularitäten — Micro-, Makro- und Gesamt-Benchmarking (einzelne Bestandteile, Interface-Operationen, ganze Algorithmen).
- Die Auslieferung des besten Gesamtalgorithmus — ausgewählt aus den feingliedrigen Messergebnissen — als eigenständige, ABI-stabile Binary hinter dem einheitlichen Suchalgorithmus-Interface.

- Eine generische Plattform-Probe, die Cache-Eigenschaften (Line-Größe, Hierarchie-Tiefe, Topologie) je Zielplattform ermittelt und als feste Größen in eine systemoptimierte Binary einfließen lässt; die beiden Produktionsmaschinen werden dafür unter zwei Betriebssystemen (immutablees Talos und root-Linux mit vollem Hardware-Zähler-Zugriff) vermessen.
- Eine bias-freie Vergleichs-Matrix, die *alle* Lastprofile gegen *alle* rekonstruierten Lebewesen fährt und so dem methodischen Heimspiel-Bias der Einzelveröffentlichungen begegnet.

1.5 Aufbau der Arbeit

Kapitel 2 führt die nötigen Grundlagen als Definitions-Klassen ein. Kapitel 3 spiegelt diese im Stand der Technik als Instanzen: einen Überblick mit Korpus-Clustern, die Cache-Konzepte, die achsenweise Sezierung der Verfahren, die Workload-Frameworks, die Mess-Technologien und das Architektur-Design — abschließend die Forschungslücke. Kapitel 4 stellt das Achsen-Bibliotheks-Framework, die geschichtete Architektur, die ABI-Schnittstelle und die neunzehn-Achsen-Permutationsmatrix vor, einschließlich der thematischen Achsen-Repositoryn und ihrer Sub-Achsen. Kapitel 5 dokumentiert die Implementierung über die drei Repositoryn. Kapitel 6 beschreibt die Mess-Methodik mit den drei Pflicht-Messreihen (A–C), den drei Granularitäten (Micro-/Makro-/Gesamt-Benchmarking), den realen Datensätzen und den Fairness-Regeln. Kapitel 7 präsentiert und diskutiert die Ergebnisse einschließlich der Hypothesen H1–H3. Kapitel 8 schließt mit Zusammenfassung und Ausblick. Der eigene Volltext ist auf einen Umfang von rund 60 bis 80 Seiten ausgelegt.

2 Grundlagen

Dieses Kapitel schafft die für den Rest der Arbeit nötigen Grundlagen: die Cache-Hierarchie und den Begriff der Cache-Bewusstheit, trie-basierte Indexstrukturen mit dem Adaptive Radix Tree als durchgängiger Referenz, das einheitliche `std::map`-Vergleichsinterface als Suchalgorithmus-Hülle mit `std`-Interfaces, das einheitliche `std::vector`-Vergleichsinterface als Container Hülle, die verschiedenen Messdimensionen und -methoden, die YCSB-Workload-Typen und die C++23-Metaprogrammierungs-Mittel, auf denen die Cache-Engine aufbaut.

2.1 Cache-Hierarchie und Cache-Bewusstheit

2.1.1 Speicher- und Cache-Ebenen

Moderne Prozessoren schalten mehrere Cache-Ebenen (L1, L2 und — nicht auf jeder Plattform — L3) zwischen die Register und den Hauptspeicher [HP19]. Die Übertragungseinheit zwischen den Ebenen ist die *Cache-Line*; ihre Größe ist jedoch keine universelle Konstante, sondern eine je Mikroarchitektur festgelegte Eigenschaft (Abschnitt 2.1.2). Auch der Transfer erfolgt nicht streng „blockweise am Stück“: Verfahren wie *critical-word-first* liefern zuerst das angefragte Wort; DDR5 unterteilt den Speicherkanal in zwei unabhängige 32-Bit-Sub-Kanäle; und nicht-temporale bzw. Write-Combining-Schreibzugriffe umgehen die reguläre Line-Granularität [Int24b]. Ein Zugriff, der sein Datum in einer Cache-Ebene findet, ist günstig; ein Fehlzugriff (Miss) wird an die nächste, langsamere Ebene weitergereicht, und ein Miss im Last-Level-Cache erreicht den Hauptspeicher zu Kosten von einigen hundert Zyklen [HP19, Dre07]. Auf Mehr-Sockel- und Chiplet-Systemen macht der nicht-uniforme Speicherzugriff (NUMA) die Latenz einer Referenz zusätzlich davon abhängig, welcher Knoten die Seite besitzt.

2.1.2 Cache-Line-Größen: statisch und dynamisch

Die Cache-Line-Größe ist je Mikroarchitektur fest verdrahtet, unterscheidet sich aber zwischen Architekturen. Auf x86-64 (AMD wie Intel) und auf gängigen ARM-Kernen (etwa Cortex-A76 oder Neoverse V2) beträgt sie 64 Byte; einzelne Designs verwenden auf SoC-Ebene breitere Linien — Apple-Silicon etwa 128 Byte auf L2 und System-Level-Cache [App24]. Programme können die tatsächliche Größe zur Laufzeit *auslesen* — über `CPUID` auf x86, das Register `CTR_EL0` auf ARM und die `riscv_hwprobe`-Schnittstelle auf RISC-V [Arm24, Int24b] —, sie aber nicht verändern. Ebenso variiert die Tiefe der Hierarchie: nicht jede Plattform besitzt einen L3-Cache (einfache RISC-V-Kerne wie der SiFive U74 enden bei einem geteilten L2), während andere einen großen oder sogar asymmetrisch verteilten L3 mitbringen (etwa der gestapelte 3D-V-Cache mancher AMD-Prozessoren). Für ein cache-line-bewusstes Layout ist die je Zielplattform ermittelte Line-Größe und Cache-Tiefe daher ein *Eingabeparameter der Übersetzung* und keine feste Annahme; Abschnitt 2.1.5 greift dies für die Abgrenzung der Arbeit wieder auf.

2.1.3 Adressübersetzung: TLB und dTLB

Neben der Daten-Cache-Hierarchie existiert eine zweite, für die Adressübersetzung: der *Translation Lookaside Buffer* (TLB) cacht Virtuell-zu-Physisch-Abbildungen, und ein TLB-Miss löst einen teuren Page-Walk aus [HP19]. Aus Lokalitätsgründen trennen Prozessoren den *Instruktions-TLB* (iTLB) vom *Daten-TLB* (dTLB) — da Instruktionszugriffe deutlich lokaler sind als Datenzugriffe —; verbreitet ist zudem eine mehrstufige Auslegung (L1-dTLB plus gemeinsamer L2-STLB) [Int24b, Dre07]. Für Suchstrukturen ist der dTLB die zweite, eigenständige Lokaltätsgrenze neben der Cache-Line: große oder weit gestreute Knoten erhöhen den dTLB-Druck unabhängig von der Cache-Auslastung, weshalb große Seiten (Huge-Pages, 2 MiB/1 GiB) den TLB-Reach vergrößern.

2.1.4 Cache-Kohärenz, Schreib- und Update-Verhalten je ISA

Schreibzugriffe behandeln Caches nach zwei Grundstrategien: *Write-Through* schreibt unmittelbar bis in die nächste Ebene durch, während *Write-Back* die Änderung zunächst nur in der Cache-Line vermerkt (*dirty*) und erst bei deren Verdrängung zurückschreibt — das spart Bus-Verkehr und senkt die Schreiblatenz [HP19]. Orthogonal dazu legt die *Allokations-Politik* fest, wie ein Schreib-Fehlzugriff behandelt wird: *Write-Allocate* lädt die Ziel-Line zuerst in den Cache (typisch bei Write-Back), *No-Write-Allocate* schreibt direkt zur nächsten Ebene durch (typisch bei Write-Through). ARMv8 macht diese Wahl über Seiten-Attribute — Write-Through, Write-Back oder Non-Cacheable sowie Read- bzw. Read-Write-Allocate — explizit [Arm24], während x86 als Normalfall Write-Back mit Write-Allocate verwendet [Int24b].

Welche Line bei einem Fehlzugriff *verdrängt* und damit ersetzt wird, entscheidet eine *heuristische Ersetzungsstrategie*. Neben exakten und approximierten Klassikern (LRU, Pseudo-LRU) sind scan- und thrash-resistente Verfahren verbreitet: die *Re-Reference Interval Prediction* (RRIP, dynamisch DRRIP) kommt mit zwei Bit je Line aus [JTJE10] und baut auf der adaptiven Insertions-Politik auf [QJP⁺07]. Wie viele Ebenen eine Line zugleich führen, regelt schließlich die *Inklusions-Politik*: ein *inklusive* Cache hält jede Line der höheren Ebenen redundant, ein *exklusiver* (Victim-Cache) schließt sie aus, ein *nicht-inklusive* lässt beides zu [HP19].

In Mehrkern-Systemen sichert ein *Kohärenzprotokoll* die Konsistenz gemeinsam genutzter Linien; das verbreitete, invalidierungsbasierte *MESI*-Protokoll führt jede Line in einem der Zustände *Modified*, *Exclusive*, *Shared* oder *Invalid*, herstellerseitig um Varianten wie MOESI (AMD) oder MESIF (Intel) erweitert [HP19]. Welche Schreib-, Allokations-, Ersetzungs-, Inklusions- und Kohärenz-Strategie konkret greift, ist je ISA und je Cache-Ebene Teil der Mikroarchitektur-Definition und zur Laufzeit nicht wählbar; ihre konkreten Ausprägungen führt der Stand der Technik an (Abschnitt 3.2), und ihre Wirkung — insbesondere das *Cache-Line-Ping-Pong* bei konkurrierenden Schreibzugriffen auf dieselbe Line — motiviert die Telemetrie- und Nebenläufigkeits-Achsen der Engine (Kapitel 4 und 5).

2.1.5 Cache-bewusst und cache-oblivious

Eine Datenstruktur ist *cache-bewusst* (*cache-aware*), wenn ihr Layout auf konkrete Cache-Parameter (Line-Größe, Ebenen-Kapazitäten) abgestimmt ist, und *cache-oblivious*, wenn sie über alle Ebenen hinweg gute Lokalität erreicht, ohne sie zu kennen [FLPR99]. Für baumartige Strukturen ist die cache-oblivious Anordnung theoretisch fundiert: Bender, Demaine und Farach-Colton minimieren die erwartete Zahl der Blocktransfers beim Layout eines Baums fester Topologie nahezu optimal [BDFC02] und pflegen

dynamische geordnete Mengen mit lokalitätserhaltender $O(k/B)$ -Traversierung [BCDFC02]. Beide Philosophien tauchen im Stand der Technik (Kapitel 3) wiederholt auf und motivieren die Layout-, Prefetch- und Hardware-Achsen der vorgeschlagenen Engine.

Die vorliegende Arbeit verfolgt einen dritten, dazwischenliegenden Weg. *Erstens* werden die tatsächlichen Cache-Eigenschaften (Line-Größe, Hierarchie-Tiefe, Topologie) je Zielplattform *ermittelt* und fließen als feste Größen in eine *systemoptimierte Binary* ein — die Line-Größe selbst ist Übersetzungseingabe, da sie auf keiner Plattform zur Laufzeit veränderbar ist (Abschnitt 2.1.2). *Zweitens* werden die *konfigurierbaren* cache-nahen Einstellungen — Prefetch-Distanz, Stapel- und Inline-Schwellen, Pool-Budgets sowie, wo Architektur und Betriebssystem es erlauben, der Hardware-Prefetcher — als Achsen-Parameter geführt, die zur Laufzeit gegen eine bereits geladene Binary permutiert werden (Abschnitt 2.3.3 und Kapitel 4). *Drittens* grenzt sich dieser Spielraum hardware- und betriebssystemabhängig ab: von voll laufzeit-konfigurierbar (mutables Linux mit MSR-Zugriff) über boot- bzw. übersetzungsstatisch (immutable Betriebssysteme) bis fixiert (read-only-Cache-Register auf ARM und RISC-V). So wird cache-oblivious-Verhalten um *gemessenes* Lokaliätswissen ergänzt, das eine heuristische, an den Messwerten orientierte Auswahl der Einstellungen je Lastmuster ermöglicht; das *Warum* dieser Konstruktion entwickelt Kapitel 4.

2.2 Klassen von Suchstrukturen

2.2.1 Etablierte Untergliederung der Suchalgorithmen

Suchstrukturen lassen sich danach einordnen, mit welchem Mechanismus sie einen Schlüssel auf einen Datensatz abbilden [Knu98]. *Vergleichsbasierte* Strukturen lokalisieren den Schlüssel über Ordnungsvergleiche und setzen eine totale Ordnung voraus; hierzu zählen die balancierten Suchbäume von AVL- und Rot-Schwarz-Bäumen bis zu den blockorientierten B-/B⁺-Bäumen. *Digitale* Strukturen verzweigen nach den Bytes des Schlüssels statt durch Vergleich (Tries, Radix-Bäume, ART). *Hashing* bildet Schlüssel direkt über eine Hashfunktion ab und kennt keine inhärente Ordnung. *Räumliche* Strukturen schließlich partitionieren einen mehrdimensionalen Raum (k-d-Baum, Quadtree, R-Baum). Diese Arbeit konzentriert sich auf die *baumartigen* Strukturen der vergleichs- und der digitalbasierten Klasse — den gemeinsamen Nenner moderner Hauptspeicher-Indizes —, während Hashing nur als Vergleichsgröße auftritt und räumliche Strukturen außerhalb des Umfangs liegen. Das in Kapitel 4 entwickelte Prinzip ist jedoch nicht an Bäume gebunden: über eine höherwertige Abstraktion lassen sich weitere Strukturklassen anschließen, ohne den Mess- und Permutationsapparat neu zu bauen (Abschnitt 8.3).

Innerhalb dieser Mechanismus-Klassen hat die Forschung konkrete Struktur-Klassen hervorgebracht; sie werden hier auf Klassen-Ebene benannt und belegt, während ihre interne Sezierung in Achsen — inklusive konkreter interner Ausprägungen wie ARTs Knotentypen — erst im Stand der Technik (Kapitel 3) als *Instanzen* erfolgt.

2.2.1.1 Vergleichsbasierte Suchbäume

Vergleichsbasierte Bäume lokalisieren Schlüssel über Ordnungsvergleiche. Aus dem unbalancierten binären Suchbaum gehen die selbstbalancierenden *AVL-Bäume* [AVL62] und *Rot-Schwarz-Bäume* [Bay72, GS78] hervor; für block- und externspeicherorientierte Indizes etablierten sich *B-* und *B⁺-Bäume* [BM72,

Com79]. Für den Hauptspeicher unmittelbar einschlägig ist der *T-Baum* [LC86].

2.2.1.2 Digitale und präfixbasierte Strukturen

Digitale Strukturen verzweigen nach den Bytes des Schlüssels statt durch Vergleich, sodass die Suchpfadlänge von der Schlüssellänge abhängt und nicht von der Schlüsselanzahl. Der *Trie* [Bri59, Fre60] ist die Grundform; *Radix*- und *PATRICIA*-Bäume [Mor68] kollabieren dünn belegte Ein-Kind-Ketten durch Pfadkompression. Der *Adaptive Radix Tree* (ART) [LKN13] passt die Knotenrepräsentation an den tatsächlichen Verzweigungsgrad an und dient dieser Arbeit durchgängig als Referenzentwurf; *Verzweigungsgrad* (Fanout) und *Knotenrepräsentation* bestimmen dabei Platzbedarf wie Cache-Verhalten.

2.2.1.3 Hashing

Hashing bildet Schlüssel direkt über eine Hashfunktion ab und kennt keine inhärente Ordnung, weshalb Bereichsabfragen teuer sind [Knu98]. Neben offener und verketteter Adressierung erreicht *Cuckoo-Hashing* [PR04] konstante Worst-Case-Lookups, und moderne, SIMD-beschleunigte Tabellen wie *SwisTable* [Goo18] erzielen hohe praktische Dichte. Hashing tritt in dieser Arbeit nur als nicht-baumartige Vergleichsgröße auf.

2.2.1.4 Räumliche Strukturen

Räumliche Strukturen partitionieren einen mehrdimensionalen Schlüsselraum — etwa der *k-d-Baum* [Ben75], der *Quadtree* [FB74] und der *R-Baum* [Gut84]. Sie liegen mit ihren mehrdimensionalen Schlüsseln außerhalb des Umfangs dieser Arbeit und werden nur der Vollständigkeit halber genannt.

2.2.1.5 Flache Verfahren und theoretische Exoten

Als Grenzfall steht das *flache* Verfahren: ein sortiertes Array mit Binär- oder Interpolationssuche [Knu98] ist streng genommen das einzelne Blatt eines Baumes. Auf String-Schlüsseln verallgemeinert das *Suffix-Array* dieses flache, sortierte Layout zu einem platzsparenden Volltext-Index, dessen leichtgewichtige, speicher-budgetierte Konstruktion ein Musterbeispiel des Algorithm-Engineerings ist [MF02]. Theoretische „Exoten“ verschieben die Komplexitätsschranken — der *van-Emde-Boas-Baum* [EB75] und der *y-fast Trie* [Wil83] erreichen $O(\log \log u)$ über einem Schlüssel-Universum der Größe u , der *Fusion Tree* [FW93] unterschreitet die informationstheoretische Vergleichsschranke. Sie sind theoretisch bedeutsam, für den cache-line-bewussten Hauptspeicher-Index dieser Arbeit aber nur Randbetrachtung.

2.2.2 Untergliederung der Container

Quer zur Suchalgorithmus-Sicht steht die etablierte Untergliederung der *Container* der C++-Standardbibliothek [ISO24]: *Sequenz-Container* (etwa `std::vector`) ordnen Elemente nach Einfügeposition, *geordnet-assoziative* Container (`std::map`, `std::set`) nach Schlüsselordnung, und *ungeordnete* Varianten (`std::unordered_map`) bilden über eine Hashfunktion ab. Diese Container stellen genormte Schnittstellen-Verträge bereit, gegen die sich strukturell verschiedene Verfahren einheitlich vermessen lassen (Abschnitt 2.3): die geordnete Map ist der gemeinsame Nenner für vergleichs-

wie digitalbasierte Suchstrukturen, ein `std::vector` liefert die Sequenz-Sicht und eine Hash-Tabelle (etwa `absl::flat_hash_map/SwissTable`) die ungeordnete Gegenprobe.

2.2.3 Begriffliche Einordnung im vorliegenden Framework

Die vorliegende Arbeit ordnet diese fremden Verfahren in ein eigenes Begriffsgerüst ein, das Kapitel 4 ausführlich entwickelt und hier nur einführt. Auf oberster Ebene steht die *Gattung* als ein nach außen sichtbares Interface — `SearchAlgorithm`, `Container` und `Graph`; darunter liegen *Lebewesen-Unterklassen* mit festem Achsensatz (die `SearchAlgorithm`-Unterklasse mit ihren neunzehn Achsen sowie `Set`, `Sequence`, `Adapter` und `View` unter dem `Container`-Interface). Eine *Achse* ist eine orthogonale Teilentscheidung — ein „Organ“ eines Gesamt-Algorithmus; ein konkretes Verfahren ist damit ein Punkt im kartesischen Produkt der Achsen, und die klassischen Familien aus Abschnitt 2.2.1 erweisen sich als bloße Lese-Gruppierungen orthogonaler Achsen-Belegungen [IZA⁺18, IZH⁺18]. Als eigenen Prüfling steuert diese Arbeit den PRT-ART bei (einen Probst-Redirect-Tree-/ART-Hybrid), dessen Achsen-Beiträge ebenfalls erst in den Kapiteln 4 und 5 seziert werden. So bleiben die fremde Klassifikation (Abschnitte 2.2.1 und 2.2.2) und die in dieser Arbeit verwendete, achsenorientierte Sezierungs-Sicht sauber getrennt.

2.3 Einheitliche Vergleichsinterfaces

2.3.1 Das `std::map`-Interface

Um strukturell verschiedene Suchalgorithmen fair zu vergleichen, werden sie alle auf eine einzige, gut verstandene Schnittstelle abgebildet: den C++-Vertrag der geordneten Map `std::map<Key, Value>` (`find/insert/erase/Bereichs-Iteration`). Die in Kapitel 4 eingeführten Achsen beschreiben das *innere* Verhalten hinter dieser Schnittstelle, während die Schnittstelle selbst fest bleibt — der Vergleich zweier konformer `std::map`-Implementierungen ist somit der zentrale Mess-Akt. Eine variadische Spezialisierung hält die Schnittstelle ergonomisch: ein Template-Parameter stellt eine `std::vector`-artige API bereit, zwei Parameter die `std::map`-API und $N > 2$ eine `std::map<K, std::tuple<...>>`. Der Kern-Vertrag der geordneten Map umfasst die Punktanfrage (`find`, `at`, `operator[]`), das Einfügen und Aktualisieren (`insert`, `insert_or_assign`, `emplace`), das Löschen (`erase`) sowie die ordnungsbasierte Navigation (`lower_bound`, `upper_bound`, `Bereichs-Iteration`) [ISO24]; die vollständige Auflistung *aller* Standard-Funktionen mit erwarteter Semantik und zugehörigen Konformitätstests führt Anhang F.

2.3.2 Das `std::vector`-Interface

Parallel zur Map-Hülle dient die Sequenz-Hülle `std::vector` als Container-Vergleichsfall [ISO24]. Ihr Ein-Parameter-Vertrag bietet indizierten Zugriff (`operator[]`, `at`), Größenänderung (`push_back`, `emplace_back`, `insert`, `erase`, `resize`) und zusammenhängenden Speicher (`data`); mit der Map geteilt sind `Iteration` sowie `size/empty/clear`, während sich die Adressierung unterscheidet (Position statt Schlüsselordnung). So lässt sich prüfen, ob ein Achsen-Komposit auch die Container-Semantik trägt und nicht nur die Suchsemantik; die vollständige Funktions-Tabelle beider Hüllen steht in Anhang F.

2.3.3 Ebenen der Dynamik

Hinter der festen Außen-Schnittstelle unterscheidet das System mehrere *Ebenen der Dynamik*, die festlegen, *wann* eine Achsen-Entscheidung fällt:

- **Übersetzungszeit vs. Laufzeit** als oberste Trennung;
- der **Lebewesen-Typ**, der nur eine bestimmte Teilmenge der vorhandenen Achsen belegt;
- die **Achsenalgorithmen**, die zur Übersetzungszeit in eine Lebewesen-Binary einkompiliert werden (statische Rekombination);
- die **dynamischen Sub-Achsen**, deren Laufzeit-Parameter (Prefetch-Distanz, Stapelgröße, Inline-Schwelle, Pool-Budget) ohne Neuübersetzung gegen eine bereits geladene Binary variiert werden;
- die **statischen Sub-Achsen** als übersetzungszeitliche Auswahl unter statischen Teil-Algorithmen unterschiedlicher Komplexität.

Diese Trennung ist im Permutations-Baum der Cache-Engine verankert (Kapitel 4): statische Achsen-Ebenen erzeugen je Pfad eine eigene Binary, dynamische Sub-Achsen sind virtuelle Laufzeit-Schleifen über derselben Binary. Erst dadurch wird je Interface-Funktion erklärbar, wie sie intern verdrahtet ist — ein Aspekt, den ein eigenes Entwickler-Dokument der Cache-Engine vertieft.

2.4 Lasten und Workloads

2.4.1 Begriff: Last, Workload und Ziel der Variation

Eine *Last* (Operation) ist ein einzelner Aufruf der Such-Schnittstelle — eine Punktanfrage, ein Einfügen, ein Löschen oder ein Bereichs-Scan. Ein *Workload* ist ein strukturiertes Profil solcher Operationen: ein Operations-Mix (etwa 50 % Lesen / 50 % Aktualisieren) über einer Schlüssel-Zugriffsverteilung (gleichverteilt, Zipf-verteilt, jüngste Werte) auf einem Datensatz definierter Größe und Schlüssel-Charakteristik. Realistische Workloads sind dabei *Playbooks* aus vielen aufeinanderfolgenden Interface-Aufrufen — nicht isolierte, unrealistische Einzelzugriffe.

Das *Ziel der Variation* ist der Bruch eines methodischen Bias: jede Veröffentlichung wählt typischerweise das Lastprofil, unter dem ihr eigener Algorithmus am besten abschneidet (ein read-only-Trie meidet Aktualisierungen, ein komprimierter Filter sucht negative Anfragen, ein cache-bewusstes Layout bevorzugt Zipf-Hotspots). Erst wenn *alle* Lastprofile gegen *alle* Lebewesen-Binaries (und zusätzlich die Container-Hüllen) laufen, lässt sich absolute, bias-freie Vergleichbarkeit gewinnen. Die konkrete „alle gegen alle-Matrix“ gehört in die Evaluationsmethodik (Kapitel 6); hier wird nur der konzeptionelle Rahmen gelegt.

2.4.2 Workload-Frameworks

Die in der Literatur verwendeten Workloads entstammen *mehreren* Benchmark-Frameworks, die diese Arbeit gemeinsam berücksichtigt; das prominenteste ist der Yahoo! Cloud Serving Benchmark (YCSB) [CST⁺10]. Er definiert eine Familie von Schlüssel-Wert-Workloads, die hier als gemeinsames Lastprofil dienen: YCSB-A (50/50 Lesen/Aktualisieren, Zipf-verteilt), YCSB-B (95/5 Lesen/Aktualisieren), YCSB-C (nur Lesen), YCSB-D (jüngste Werte), YCSB-E (kurze Bereichs-Scans) und YCSB-F

(Read-Modify-Write). Jedes Algorithmus-Profil deklariert einen erwarteten Workload (Kapitel 3), den der Mess-Treiber für das profilbewusste Routing nutzt (Kapitel 6).

Daneben stützen sich die analysierten Veröffentlichungen auf weitere Frameworks: den Index-Benchmark *SOSD* [KMR⁺19], die Transaktions-Benchmarks der *TPC*-Familie (TPC-C, TPC-DS) [Tra24], die Integration in reale Systeme (etwa SuRF als Bloom-Filter-Ersatz in RocksDB), reale String-Korpora (URLs, DNA, Wörterbuch-Terme) sowie — bei den hardware-nahen Arbeiten — *SPEC CPU* [Sta17] und *CloudSuite* [FAK⁺12]; der Allokator-Korpus wird mit allokator-spezifischen Suites vermessen (*mimalloc-bench* [Lc21], *Larson*, *threadtest*, *shbench*). Da diese Frameworks unterschiedliche, untereinander nicht vergleichbare Lasten erzeugen, abstrahiert die Arbeit sie zu einem gemeinsamen Satz von Lastprofilen und fährt diese als *dynamische Workload-Achse* über alle Lebewesen-Binaries (Kapitel 6); die Anbindung der nicht-YCSB-Frameworks erfolgt über einen Datensatz-Lader.

2.5 Wissenschaftliches Messen

2.5.1 Gütekriterien wissenschaftlichen Messens

Wissenschaftliches Messen unterliegt etablierten Gütekriterien [HB15]. *Reproduzierbarkeit* verlangt feste Zufalls-Seeds, eine deterministische Ausführungsreihenfolge und vollständig dokumentierte Mess-Konfigurationen, sodass ein Lauf bit-genau wiederholbar ist. *Validität* gliedert sich in interne (kausal sauber isolierte Effekte), Konstrukt- (die Metrik misst tatsächlich das gemeinte Konzept) und externe Validität (Übertragbarkeit auf andere Plattformen und Lasten). *Fairness* fordert identische Eingaben — gleiche Schlüssel, Workloads und Warmup — für alle Vergleichskandidaten; fehlende Fähigkeiten einer Baseline werden als *N/A* ausgewiesen statt über Hilfskonstrukte verdeckt emuliert. Schließlich ist die *Streuung* Teil der Aussage: statt arithmetischer Mittel werden Perzentile über HDR-Histogramme berichtet und die statistische Signifikanz von Unterschieden geprüft [HB15].

2.5.2 Verwendete Mess-Muster und -Konzepte

Auf diesen Kriterien bauen mehrere Mess-Muster auf, deren *Warum* erst Kapitel 4 entwickelt. Eine *Zwei-Phasen-Operationsschleife* trennt einen verworfenen Warmlauf von der eigentlichen Messung und entkoppelt so die Latenz vom akkumulierten Zustand. Die Erhebung erfolgt *hybrid* auf zwei Wegen: isolierte Achsen-Algorithmen werden direkt gegeneinander vermessen, während ein zusammengesetztes Lebewesen zentral über eine ABI-stabile Beobachter-Schnittstelle (Per-Achsen-Observer) erhoben wird. Ein *Konformitäts-Gatter* prüft vor jeder Messung, dass eine Implementierung den `std::map`-Vertrag erfüllt, damit nur Vergleichbares verglichen wird; jede Konfiguration wird in mehreren getrennten Läufen mit ausgewiesener Streuung erhoben.

Diese Konzepte bilden die *Definitions-Klassen* des Messens; der Stand der Technik (Kapitel 3) führt die *Instanzen* an, die sie technisch umsetzen — etwa Hardware-Performance-Zähler (PMC), die OTF2-/VAMPIR-Profiling-Werkzeuge oder die TU-Dresden-Hardware-Messmethodik. So greifen Grundlagen und Stand der Technik unmittelbar ineinander.

2.6 C++23-Metaprogrammierung und Design-Pattern

2.6.1 Klassische Design-Pattern

Das System verwendet durchgängig benannte Entwurfsmuster (GoF) [GHJV94]; eingeführt werden hier nur die im Code tatsächlich verwendeten. *Strategy* kapselt jede der neunzehn Achsen als austauschbaren Algorithmus; *Abstract Factory* erzeugt die statischen und dynamischen Knoten des Permutations-Baums, *Composite* bildet diesen Baum und ein *Iterator* durchläuft ihn lazy. *Adapter* bindet Fremd-Code hinter der stabilen Engine-Schnittstelle an (Habich-Direktive). *Observer* (Pull-Modell) und *Decorator* (nicht-intrusive Mess-Hüllen um die Achsen) erheben die Messwerte, ein *Memento* (Copy-on-Write) sichert und verwirft den Zustand zwischen Warmlauf und Messung. Statischer Polymorphismus entsteht durch das *Curiously Recurring Template Pattern* (CRTP) samt *Template-Method-Concept*-Prüfungen. Nicht verwendet werden hingegen Singleton, Command, Flyweight und Bridge.

2.6.2 Metaprogrammier-Mittel

Die Engine stützt sich auf Komposition zur Übersetzungszeit, um Laufzeit-Dispatch im Hot-Path zu vermeiden. *Concepts* schränken die Bausteine-Schnittstellen ein; das *Curiously Recurring Template Pattern* (CRTP) liefert statischen Polymorphismus; `if constexpr` und `std::conditional_t` wählen pro Achse zwischen einer Prüfling-Implementierung und einem Cache-Engine-Standard; und C++23-Module bilden eine ABI-stabile Grenze zwischen den Permutations-Binaries und dem Builder. Zusammen realisieren diese Mittel die *zero-cost abstraction*, auf der die Permutationsmatrix beruht. Hinzu kommen *Policy-based Design* (die Komposition als Tupel von neunzehn Achsen-Policies), *Tag-Dispatch* für die Prüfling-gegen-Standard-Auflösung sowie *Type-List-Computation* (Boost.MP11), die den Achsen-Produktraum beschreibt, ohne ihn vollständig zu materialisieren [Ale01].

2.6.3 Neue und unbenannte Metaprogrammier-Pattern

Über die etablierten Muster hinaus treten zwei Konstruktionen auf, für die eine Web-Recherche keine etablierte Benennung fand und die daher als Beitrag dieser Arbeit benannt werden. Erstens die *lazy-spärliche kartesische Materialisierung*: der Produktraum aller Achsen-Varianten wird zur Übersetzungszeit nur arithmetisch gezählt — über 10^{14} mögliche Achsen-Kombinationen, von denen die tatsächlich als eigenständige Binaries materialisierte, lauffähige Teilmenge entsprechend kleiner ist — und je Pfad mixed-radix *einzel*n on-demand materialisiert, statt den vollen Typ-Baum zu instanziiieren (der den Compiler sprengte). Zweitens das *typgetriebene Quelltext-Emittieren*: aus einer compile-zeitlichen Achsen-Komposition werden die voll-qualifizierten Typnamen extrahiert, zu C++-Quelltext gerendert, je Permutation als eigenständiges Modul übersetzt und über die ABI-Grenze geladen — so wird der heruntergeladene Prüfling (PRT-ART) per Metaprogrammierung in Permutations-/Matrix-Schichten gegen die Kernbestandteile der Cache-Engine kompiliert. Beide Muster verbinden bekannte Idiome (Concepts, CRTP, Type-Lists) zu einer architektur-spezifischen Neuheit; ihre vollständige Realisierung entwickeln die Kapitel 4 und 5.

3 Stand der Technik

Wo Kapitel 2 die konzeptionellen *Definitions-Klassen* bereitstellt, führt dieser Stand der Technik ihre wissenschaftlich nachweisbaren *Instanzen* an: zu jeder Grundlagen-Klasse die konkreten Veröffentlichungen, Algorithmen und Technologien, die sie umsetzen. Beide Kapitel greifen damit unmittelbar ineinander (Abschnitt 2.5.2).

3.1 Überblick und Sezierungs-Prinzip

Die Analyse stützt sich auf 33 tiefenanalysierte Suchalgorithmus-Veröffentlichungen (P01–P33) und einen parallelen Allokator-Korpus aus 23 Arbeiten (A01–A23); die Achsen-Belegung jeder Arbeit ist als XML-Profil in der `comdare-cache-engine (algorithm_profiles/)` persistiert, die vollständigen Bausteine- und Allokator-Matrizen stehen in Anhang D. Die Veröffentlichungen entstammen sechs thematischen *Herkunfts-Clustern* — Trie-Familie, Hybrid- und B⁺-Familie, Layout-Theorie, Prefetching, Synchronisation und den hardware-nahen TU-Dresden-Arbeiten —; diese Cluster dienen hier nur der Herkunfts-Einordnung, nicht als Gliederungsachse.

Der Kernbeitrag dieser Arbeit ist die *Sezierung* (Abschnitt 2.2.3): Jeder Gesamt-Algorithmus (ein „Lebewesen“) wird in seine orthogonalen Organe — die Achsen — zerlegt, und die so gewonnenen Organ-Algorithmen werden *je Achse* einsortiert, mit kurzem Verweis auf das Herkunfts-Lebewesen. Diese achsenweise Sezierung bildet den Kern des Kapitels; die Zuordnung Achse ↔ Algorithmus ↔ Veröffentlichung folgt einer web-verifizierten Provenienz-Map über rund 110 Organ-Algorithmen.

Dass das Achsen-Gerüst auch *nicht-baumartige* Verfahren trägt, belegt eine reine Hash-Tabelle (`abs1::flat_hash_map/SwissTable` [Goo18]) als Gegenprobe innerhalb der `SearchAlgorithm`-Gattung: Sie erfüllt dasselbe `std::map`-Interface über ein *limitiertes Achsen-Subset* — die trie-spezifischen Achsen (Pfadkompression, Knotentyp, Bereichs-Index) entfallen als Durchreich-Variante, während Allokation, Prefetch, Nebenläufigkeit, Speicher-Layout und ISA-SIMD (für das `SwissTable`-Gruppen-Lookup) geteilt bleiben — und erfordert so keine eigene Gattung.

3.1.1 Der Korpus nach Herkunfts-Clustern

Ergänzend zur achsenweisen Sezierung (Abschnitt 3.3) ordnet dieser Überblick die Veröffentlichungen ihren sechs Herkunfts-Clustern zu — die ganzheitliche Sicht auf jedes Lebewesen, bevor es in seine Organe zerlegt wird. Die *Trie-Familie* (Cluster A) umfasst ART (P01) [LKN13], HOT (P02) [BZP⁺18], Masstree (P03) [MKM12], CoCo-Trie (P04) [BFTV24], START (P05) [FJKN20] sowie die succinct-Tries LOUDS (P09) [Jac89] und SuRF (P10) [ZLL⁺18]. Die *Hybrid- und B⁺-Familie* (Cluster B) vereint den B²-Baum (P06) [SSL⁺21], Wormhole (P07) [WNJ19] und die cache-sensitiven CSS-, CSB⁺- und Hankins-Bäume (P11–P13) [RR99, RR00, HP03]. Die *Layout-Theorie* (Cluster C) reicht von Samuel (P14) [SPB05] und Graefe/Larson (P15) [GL01] über die cache-oblivious Entwürfe von Bender et al.

(P16/P17) [BDFC00, BDFC05] — theoretisch fundiert durch deren ESA-2002-Arbeiten zum Baum-Layout in der Speicherhierarchie [BDFC02], zur cache-oblivious Traversierung [BCDFC02] und zum Order-Maintenance-/Packed-Memory-Primitiv [BCD⁺02] — bis zu den layout-invarianten Bäumen von Saikkonen/Soisalon-Soininen (P18/P19) [SSS08, SSS16]. Das *Prefetching* (Cluster D/E) umfasst *B-Trees Are Back* (Mueller 2025, P20) [MBL25], die Prefetching-B⁺-Bäume von Chen et al. (P21/P22) [CGM01, CGM02], Khan (P23) [Kha11], Naderan-Tahan (P24) [NTSA16], Mahling (P25) [MWR25] sowie Zhang (P26/P27) [ZSZ⁺24, ZGH⁺25] und Kuehn (P28) [Kue23]. Die *Synchronisation* (Cluster F) trägt das optimistische Lock-Coupling von ART (P08) [LABN16], RCU (P29) [McK01] und Hazard-Pointer (P30) [Mic04]; die hardware-nahe *TU-Dresden-Linie* schließlich Ungethüm (P31) [UHL⁺17], Schmidt (P32) [SKK⁺25] und VAMPIR (P33) [BH23]. Parallel steht der *Allokator-Korpus* (A01–A23) — von Hoard [BMBW00] und Slab [Bon94] über mimalloc [LZM19], jemalloc [Eva06], TCMalloc [GM05] und sncmalloc [LDPB19] bis zu NUMAalloc [YZL23], StarMalloc [RFP⁺24] und PIM-malloc [VIA26].

3.2 Cache-Konzepte als Instanzen

Die Cache-Hierarchie-Konzepte aus Abschnitt 2.1 treten im Stand der Technik als konkrete Entwürfe und Mechanismen auf; jeder der folgenden Unterabschnitte spiegelt eine Grundlagen-Definition aus Abschnitt 2.1.

3.2.1 Speicher- und Cache-Ebenen

Die mehrstufige Hierarchie (Abschnitt 2.1.1) ist Gegenstand grundlegender Vermessungen: Graefe und Larson (P15, 2001) untersuchen B-Baum-Indizes explizit gegenüber den CPU-Cache-Ebenen, Bender, Demaine und Farach-Colton (P16, 2002; P17, 2005) modellieren ein mehrstufiges Speicher-Modell mit einem Block-Transfer-Parameter und lösen darin das Baum-Layout-Problem auch bei unbekannter Block-Größe nahezu optimal [BDFC02], und Zhang (P27, ASPLOS 2025) bündelt Prefetches hierarchisch über L1/L2/L3.

3.2.2 Cache-Line-bewusstes Layout

Das cache-line-orientierte Layout (Abschnitt 2.1.2) ist der dominante Leitgedanke der Layout-Theorie: Der CSS-Baum (P11, Rao/Ross 1999) führt den Cache-Line-Knoten ein, der CSB⁺-Baum (P12, Rao/Ross 2000) das zusammenhängende Geschwister-Cluster; Hankins und Patel (P13, 2003) vermessen den Effekt der Knotengröße, Samuel et al. (P14, 2005) machen den Knoten prozessor-bewusst. Der Adaptive Radix Tree (P01 [LKN13]) staffelt mit Node4/16/48/256 die Knotengröße cache-line-orientiert, und Schmidt et al. (P32, 2025) untersuchen Stride- gegen Sequenz-Zugriffe auf High-Bandwidth-Memory. Auf der Speicher-Layout-Achse stehen damit konkret: dichtes Array-of-Structs, cache-line-ausgerichtetes AoS, spaltenorientiertes Struct-of-Arrays, bit-gepacktes LOUDS (P09, Jacobson 1989) und das SIMD-gekachelte AoSoA.

3.2.3 Adressübersetzung: TLB und dTLB

Der dTLB (Abschnitt 2.1.3) tritt im Stand der Technik vorwiegend als *Mess-Ziel* auf — die Mess-Architektur führt eine eigene dTLB-Miss-Kategorie (Abschnitt 2.5). Algorithmenseitig adressieren ihn

große Seiten (Huge-Pages), die mehrere Allokatoren (mimalloc, jemalloc, TCMalloc, smlalloc) unterstützen, sowie Schmidt et al. (P32, 2025) empirisch mit 2-MiB-Seiten. Eine *eigene* dTLB-Optimierungs-Achse bleibt in den Suchalgorithmus-Papern jedoch unterbelegt — eine Beobachtung, auf die die Forschungslücke (Abschnitt 3.7) zurückkommt.

3.2.4 Schreib-Protokolle, Update-Heuristiken und Inklusions-Politik

Das in Abschnitt 2.1.4 eingeführte Cache-Update-Verhalten ist je ISA und je Cache-Ebene konkret festgelegt. *Schreib-Protokolle*: ARMv8 wählt Write-Through, Write-Back oder Non-Cacheable sowie Read- bzw. Read-Write-Allocate über Seiten-Attribute [Arm24], während x86 als Normalfall Write-Back mit Write-Allocate verwendet [Int24b]. *Heuristische Ersetzungs-/Update-Strategien*: von LRU und Pseudo-LRU bis zur *Re-Reference Interval Prediction* (RRIP/DRRIP), die mit zwei Bit je Line scan- und thrash-resistente Verdrängung erreicht [JTJE10] und auf der adaptiven Insertions-Politik aufbaut [QJP⁺07]. *Inklusions-Politik*: Intel-Client-Prozessoren führen einen inklusiven L3, Intel-Server ab Skylake-X einen nicht-inklusive, während AMD Zen den L3 als überwiegend exklusiven Victim-Cache betreibt [Int24a, Adv23]. Diese drei Aspekte bestimmen gemeinsam, welche Ebenen ein Schreibzugriff aktualisiert; da sie mikroarchitektonisch fixiert und zur Laufzeit nicht wählbar sind, fließen sie als *vermessene* Plattform-Eigenschaften in die systemoptimierte Binary ein (Abschnitt 2.1.5).

3.2.5 Cache-bewusste und cache-oblivious Entwürfe

Beide Philosophien aus Abschnitt 2.1.5 sind im Stand der Technik instanziiert: Bender et al. (P17, 2005) begründen den cache-oblivious B-Baum über das van-Emde-Boas-Layout — aufbauend auf dem Order-Maintenance-/Packed-Memory-Primitiv [BCD⁺02] und der cache-oblivious Traversierung geordneter Mengen [BCDFC02] —, P16 (2002) erweitert das mehrstufige Layout-Modell, und Saikkonen/Soisalon-Soininen (P18, 2008; P19, 2016) halten eine cache-sensitive Layout-Invariante auch unter Updates; Graefe/Larson (P15, 2001) vergleichen beide Ansätze. Sämtliche praktischen Trie- und B⁺-Indizes (P01–P14, P20) sind hingegen *cache-bewusst* ausgelegt. Die vorliegende Arbeit ordnet sich als dritter Weg ein (Abschnitt 2.1.5): gemessenes Lokaliätswissen statt fest verdrahteter oder vollständig ignorierte Cache-Parameter.

3.3 Achsen-Sezierung der Suchverfahren

Den Kern dieses Kapitels bildet die achsenweise *Sezierung*: Jeder Gesamt-Algorithmus wird in seine Organe zerlegt, und je Achse werden die beitragenden Organ-Algorithmen aufgeführt — mit kurzem Verweis auf das Herkunfts-Lebewesen. Das vorgeschlagene Framework führt *neunzehn Kompositions-Achsen* (T0–T18) sowie drei *Build-Achsen* (Seitentyp, SIMD-Erweiterung, Hardware-Profil); viele Achsen sind in Sub-Achsen verfeinert. Die Sub-Achsen-Kürzel (etwa SA1–SA3 oder 6.1–6.5) bezeichnen die Gliederungs-Ebene der Bausteine-Matrix dieser Arbeit (Anhang D); im Code sind sie als themenweise Varianten-Familien (`axis_<nn>_<name>/`) realisiert. Die Tabellen 3.1 und 3.2 geben den Überblick, welche Veröffentlichungen je Achse ein Organ beitragen; die folgenden Unterabschnitte führen sie thematisch gruppiert aus. Die thematischen Herkunfts-Cluster der Vorarbeiten (Trie-Familie P01–P10,

Hybrid-/B⁺-Familie P03/P06/P07/P11–P13, Layout-Theorie P14–P19, Prefetching P20–P28, Synchronisation P08/P29–P30, TU-Dresden-Hardware P31–P33) erscheinen dabei *aufgelöst* in ihren Organ-Beiträgen.

Tabelle 3.1: Achsen-Sezierung (Teil 1: Kompositions-Achsen T0–T9)

Achse (T)	Sub-Achsen	Beitragende Quellen
T0 search_algo	SA1–SA3	P01,P02,P05,P07,P10; k-ary (TUD 2009), Interpol., Eytzinger, Skip-List, B-Baum, Hash
T1 cache_traversal	3.B	Cache-Line-Default; P17,P21,P27,P32
T2 mapping	3.M	P03,P07,P27; CE-Baseline
T3 path_compression	—	P01,P02,P04,P05,P09
T4 node_type	NT1–NT3	P01,P02,P03,P04,P05,P06,P11,P12,P13
T5 memory_layout	HM1–HM4	P06,P09,P11,P12,P32; Default
T6 allocator	6.1–6.5	A01–A23; P29,P30
T7 prefetch	PF1–PF3	P07,P21,P23,P25,P26,P27
T8 concurrency	8.1,8.2	P08,P29,P30; Dijkstra,Courtois,Herlihy,Michael/Scott
T9 serialization	—	P01,P09,P10; LZ77

Tabelle 3.2: Achsen-Sezierung (Teil 2: T10–T18 + Build-Achsen)

Achse	Sub-Achsen	Beitragende Quellen
T10 telemetry	11.X1–X4	P01,P02,P16,P26,P28; HdrHistogram (Gil Tene)
T11 value_handle	—	P01,P03,P07,P29
T12 isa	—	x86-64/AArch64/Power/RISC-V (Vendor-Spec)
T13 index_organization	—	Bayer/McCreight,Comer,Oracle
T14 io_dispatch	IO1–IO3	Stonebraker,Crotty; Baselines
T15 migration_policy	MG1–MG3	Anti-Caching,NVM-Tiering,LeCaR; None
T16 filter	FT1–FT3	Bloom,Cuckoo,P10,Xor
T17 queuing_q1	—	Disruptor, Bw-Tree, Driscoll, P29, Pugh, LSM, Lamport, Michael/Scott
T18 queuing_q2	—	Eager/Lazy/Watermark/Kafka/RocksDB
Build page_type	PG1–PG3	P01,P02,P04,P05,P06,P10,P11,P12,P13,P20; PRT-ART (26)
Build simd (09b)	—	SSE2/AVX2/AVX-512/NEON/SVE2/RVV/CUDA
Build hardware (12)	12.1–12.5	HW-Strategie je Paper (Tab. 3.3)

3.3.1 Such- und Navigations-Organen

Die *Such-Achse* (T0, Sub-Achsen SA1–SA3) trägt die meisten Organe: als Original-Bindungen ART (P01 [LKN13]), HOT (P02 [BZP⁺18]), START (P05 [FJKN20]), SuRF (P10 [ZLL⁺18]) und Wormhole (P07 [WNJ19]); dazu generische Such-*Methoden*: k-äre Suche (Schlegel/Gemulla/Lehner, DaMoN 2009 — TU-Dresden), Interpolationssuche (Perl/Itai/Avni 1978), Eytzinger-Layout-Suche (Khuong/Morin 2017), Skip-Liste (Pugh 1990), binärer Suchbaum und B-Baum (Bayer/McCreight [BM72]) sowie Open-Addressing-Hashing (Knuth [Knu98]). Die *Traversierungs-Achsen* trennen Algorithmus- und Cache-Sicht: T1 (Cache-Memory-Traversierung) reicht von der Cache-Line-Wanderung über Stride-Prefetch (P21 Chen) und HBM-Nähe (P32 Schmidt) bis zur cache-oblivious-Rekursion (P17 Bender [FLPR99]) und dem L1/L2/L3-Bündel (P27 Zhang); T2 (Mapping) verbindet beide über lineare 1:1-Abbildung, Hash-Redirect (P07) oder Permutations-Index (P03 Masstree) — eine im Stand der Technik dünn, überwiegend per Baseline besetzte Achse. Die *Pfadkompression* (T3) instanziiert die byte-weise Variante (P01), den Single-Bit-Patricia-Split (Morrison 1968 [Mor68]; P02), die kollabierte Macro-Node (P04 CoCo) und das Multi-Byte-Rewiring (P05). Die *Knotentyp-Achse* (T4, NT1–NT3, 13 Bausteine) und die Build-Achse *Seitentyp*

(PG1–PG3, 26 Bausteine) bündeln die adaptiven Layouts: Node4/16/48/256 (P01), HOT-Compound (P02), Internal/Border (P03), Decision/Span (P06 B²-Baum [SSL⁺21]), LOUDS-Dense/Sparse (P10), CSS-Knoten/CSB⁺-Node-Group/Hankins-Wide (P11/P12/P13) sowie die PRT-ART-Familie.

3.3.2 Speicher-, Allokations- und Wert-Organen

Die *Speicher-Layout-Achse* (T5, HM1–HM4, 8 Bausteine) reicht vom cache-line-ausgerichteten AoS über zeigerfreie zusammenhängende Anordnung (P11 CSS) und Geschwister-Cluster (P12 CSB⁺) bis zum eingebetteten Baum (P06), bit-gepacktem LOUDS (P09 Jacobson) und den Huge-Page-Varianten (P32). Die *Allokator-Achse* (T6) ist mit dem Korpus A01–A23 am dichtesten besetzt und in fünf Sub-Achsen verfeinert: 6.1 Allokationsstrategie (Slab, Buddy, Region, Pool, Object-Cache), 6.2 Reklamation (Epoch, RCU P29, Hazard-Pointer P30, QSBR), 6.3 NUMA-Affinität (P31 Ungethüm), 6.4 Huge-Page-Politik und 6.5 Free-List-Organisation (Size-Class, Best-/First-Fit, segregiert). Tabelle 3.7 listet die zehn lauffähigen Allokator-Profilen mit Lizenz und Workload-Affinität. Die *Wert-Achse* (T11, 5 Bausteine) unterscheidet Inline-Wert (P01), Zeiger, Redirect-to-Node bzw. externen Pool (P07 Wormhole), versionierten Zeiger (P03 Masstree) und immutable Shared-Ref (P29 RCU). Die *Serialisierungs-Achse* (T9) trägt Raw-Binary, Block-Kompression (LZ77, Ziv/Lempel 1977; LZ4/Snappy), succinct LOUDS/FST (P09/P10) und Varint (P01).

3.3.3 Hardware-nahe Organen

Die *Prefetch-Achse* (T7, PF1–PF3, 6 Bausteine) reicht von keinem Prefetch über software-festes (P21 Chen) und adaptiv-distanziertes (P23 Khan) bis zu fill-buffer-bewusstem (P25 Mahling), pfad-/jump-pointer-orientiertem (P26 Zhang) und hierarchischem Bündel-Prefetch (P27 Zhang); die explizite Hardware-Prefetch-Instruktion stammt aus Wormhole (P07). Die *ISA-Achse* (T12) und die Build-Achse *SIMD-Erweiterung* (09b) führen die Vendor-Spezifikationen: x86-64, AArch64, Power und RISC-V mit SSE2/AVX2/AVX-512, NEON/SVE2, RVV und CUDA. Zwei übergreifende Strategie-Achsen halten fest, was ein Algorithmus aktiv *nutzt* bzw. wie er Arbeit *verteilt*: die *Hardware-Strategie* (Achse 12; Sub 12.1 SIMD-Familie, 12.2 Cache-Level-Targeting, 12.3 NUMA, 12.4 Prefetch-Hardware, 12.5 Atomic-Familie) und die *Scheduling-Strategie* (Achse 13; Sub 13.1 Worker-Pool-Layout, 13.2 SIMD-Worker-Limit — hardwarebedingt typisch zwei von N Kernen —, 13.3 heterogener P-/E-Kern-Dispatch, 13.4 Co-Routine, 13.5 Batch-Granularität). Tabelle 3.3 ordnet beide je Paper zu. Die *Telemetrie-Achse* (T10, Sub 11.X1–X4, 6 Bausteine) instanziiert den Density-Tracker (P01), den Insert-Zähler (P02), das HdrHistogram (Gil Tene) für p50/p95/p99, den Pfad-Lese-Zähler (P26) und die Probability-Hints (P16/P17 Bender). Die kohärenz-schonenden Zähler-Varianten — Leaf-Only, Sampling und Offline-Recompute — gehen auf Kuehn zurück (P28, persönliche Kommunikation 2023, über den publizierten DaMoN-2023-Stand zum cache-/verteilungsbewussten B⁺-Layout hinaus), während der Zähler in allen (auch inneren) Knoten als *Anti-Pattern* (Cache-Line-Ping-Pong) gilt.

3.3.4 Nebenläufigkeit, I/O, Migration und Pufferung

Die *Nebenläufigkeits-Achse* (T8) gliedert sich in 8.1 Muster und 8.2 Sperr-Modus. Die Muster reichen vom seriellen Nullpunkt über grobes Sperren (Dijkstra 1965) und Leser-/Schreiber-Sperren (Courtois 1971) zum optimistischen Lock-Coupling (P08), zu lock-freien (Michael/Scott 1996) und wait-freien Verfahren

Tabelle 3.3: Hardware- und Scheduling-Strategie pro SOTA-Paper (Auswahl); der Vollkatalog steht in Anhang D.

Paper	Hardware-Strategie (Achse 12)	Scheduling-Strategie (Achse 13)
P01 ART	AVX2, L1-aware, none, CAS	thread-per-core
P02 HOT	AVX2 (Bit-Vektor), L1-aware, CAS	thread-per-core
P03 Masstree	scalar, L1-aware, CAS	thread-per-core + work-stealing
P05 START	AVX2 (Multi-Byte-Span), L1-aware, CAS	thread-per-core
P20 B-Trees Are Back	AVX2, HBM-aware, NUMA-aware, CAS	work-stealing + hybrid
P27 hp-soft	PREFETCH (L1/L2/L3-Bundle), all-cache, CAS	thread-per-core + Co-Routine
P28 Kuehn	PREFETCH (scalar Counter), L1-aware, CAS	thread-per-core + Sampling
P31 Ungethuem	AVX-512 (HBM), HBM-aware, NUMA-aware, CAS	hybrid (P-/E-Kerne)
P32 Schmidt	AVX-512 (SIMD-Stride), HBM-aware, NUMA-aware, CAS	hybrid
PRT-ART	AVX2, L1-aware (TLB-aligned), local NUMA, PREFETCH, CAS (OLC)	thread-per-core + 2-SIMD-Worker-Limit + hybrid + Co-Routine + micro-batch

(Herlihy 1991) sowie zu RCU (P29) und Hazard-Pointern (P30); der Sperr-Modus unterscheidet read-only, read-write, upgradeable und optimistische Validierung. Die *I/O-Achse* (T14, IO1–IO3) instanziiert gepufferte (Stonebraker 1981), direkte und memory-mapped Ein-/Ausgabe (Crotty, CIDR 2022) neben dem In-Memory-Nullpunkt. Die *Migrations-Achse* (T15, MG1–MG3) trägt statische Platzierung, Hot/Cold-Trennung (Anti-Caching 2013), mehrstufige RAM→NVM→SSD-Migration (2018) und ML-getriebene adaptive Migration (LeCaR 2018). Die *Pufferungs-Achsen* (T17/T18) schließlich umfassen je rund ein Dutzend Bausteine: Operations-Puffer von der FIFO-/LIFO-Schlange über den Disruptor-Ringpuffer (2011), die Delta-Kette (Bw-Tree 2013), den Copy-on-Write-Puffer (Driscoll 1989), den Epoch-Puffer (P29), die Skip-Liste (Pugh 1990) und den Tombstone-Puffer (LSM, O’Neil 1996) bis zu lock-freien SPSC- (Lampert 1983) und MPMC-Queues (Michael/Scott 1996); sowie Schreib-Rückführung (Flush) von eager über lazy, schwellwert- und zeitfenster-basiert (Kafka) bis adaptiv (RocksDB-LSM).

3.3.5 Index-Organisation und Filter

Die *Index-Organisations-Achse* (T13) unterscheidet die geclusterte Anordnung (Speicher- gleich Indexordnung; Bayer/McCreight [BM72]), die ungeordnete Heap-Ablage, die index-organisierte Tabelle (Oracle, VLDB 2000) und den nicht-geclusterten Sekundärindex (Comer [Com79]). Die *Filter-Achse* (T16, FT1–FT3) trägt den Bloom-Filter (1970), den Cuckoo-Filter (CoNEXT 2014), den succinct-Bereichsfilter SuRF (P10) und den Xor-Filter (2020).

Diese Sezierung zeigt zugleich die Schieflage der Vorarbeiten: Wenige Achsen sind dicht und vielfach besetzt (Such-, Knotentyp-, Allokator-, Prefetch-Achse), andere ausschließlich mit Baseline-/Lehrbuch-Varianten (Mapping, Cache-Memory-Traversierung) oder als reine Mess-Ziele (dTLB) — kein einziges Verfahren belegt alle Achsen gemeinsam. Genau diese Unvollständigkeit motiviert die Forschungslücke (Abschnitt 3.7).

3.4 Workload-Frameworks als Instanzen

Die in Abschnitt 6.4 eingeführten Begriffe — Last, Workload, Workload-Framework und das *Ziel der Variation* — erhalten hier ihre wissenschaftlichen Instanzen.

3.4.1 Benchmark-Frameworks

Die in der Literatur verwendeten Lasten entstammen *mehreren*, untereinander nicht vergleichbaren Benchmark-Frameworks; Tabelle 3.4 ordnet sie den nutzenden Veröffentlichungen zu. Am verbreitetsten ist der Yahoo! Cloud Serving Benchmark (YCSB) [CST⁺10]; daneben treten der Index-Benchmark SOSD [KMR⁺19], die Transaktions-Benchmarks der TPC-Familie [Tra24], reale System-Integrationen (RocksDB), reale String-Korpora, die CPU-Suite SPEC CPU [Sta17], die Scale-out-Suite CloudSuite [FAK⁺12], der Architektur-Simulator gem5 [BBB⁺11] sowie — für den Allokator-Korpus — mimalloc-bench [Lc21] und die Klassiker Larson, threadtest, shbench und LADDIS auf. Da diese Frameworks unvergleichbare Lasten erzeugen, abstrahiert die Arbeit sie zu einem gemeinsamen Satz von Lastprofilen (Abschnitt 3.4.2); die noch nicht im eigenen Mess-Apparat realisierten Frameworks (TPC, SOSD, SPEC, CloudSuite, gem5, Allokator-Suiten) werden über einen Datensatz-Lader angebunden.

Tabelle 3.4: Benchmark-Frameworks und nutzende Veröffentlichungen

Framework	Typ	Nutzende Veröffentlichungen
YCSB [CST ⁺ 10]	KV-Workload	P02, P03, P07, P20, P26
SOSD [KMR ⁺ 19]	Index-Benchmark	P05
TPC-C / TPC-DS [Tra24]	OLTP / analytisch	P19
Real-String-Korpora	String-Datensätze	P04, P06, P10
RocksDB-Integration	Real-System	P10 (SuRF)
SPEC CPU [Sta17]	CPU-Suite	P23, P24, P27, A18
CloudSuite [FAK ⁺ 12]	Scale-out-Server	P24, P27
gem5 [BBB ⁺ 11]	Architektur-Simulator	P27
IBM DB2	Produktions-DBMS	P22
mimalloc-bench [Lc21]	Allokator-Suite	A04, A07, A09, A10, A18
Larson / threadtest / shbench	Allokator-Klassiker	A01, A03, P08, P30
LADDIS	Kernel-Allokator	A02
AggSum / Memory-Stream	Bandbreite	P32
Custom-Microbenchmarks	per Paper (Zipf/Uniform)	P01, P04, P06, P11–P14, P21, P28

3.4.2 Lastprofile und Workload-Affinität der Verfahren

Aus der Analyse der genannten Frameworks destilliert diese Arbeit *vierzehn* distinkte Lastprofile (LP01–LP14, Tabelle 3.5): Sie geben den heterogenen Lasten ein gemeinsames, uint64-basiertes Operations-Modell — der Op-Mix nennt die Anteile von Einfügen, Lookup, Löschen, Scan und Read-Modify-Write (RMW) — über definierten Schlüssel-Verteilungen, und sind je aus konkreten Veröffentlichungen abgeleitet.

Für jeden SOTA-Algorithmus und jeden Allokator hält ein Profil-XML in `algorithm_profiles/` Metadaten fest, die Achsen-Belegung (`page`, `node`, `traversal`, `value_handle`, `concurrency`, `allocator`,

Tabelle 3.5: Lastprofil-Katalog (14 Profile, aus der 33-Paper-Analyse abgeleitet)

LP	Kennung	Op-Mix	Verteilung	neg%	Abgeleitet aus
LP01	bulk-insert-dense	100/0/0/0/0	uniform-seq	0	P01, P02, P03, P05, P12
LP02	bulk-insert-sparse	100/0/0/0/0	uniform-rand	0	P01,P02,P08
LP03	insert-value-length-sweep	100/0/0/0/0	uniform-seq	0	P01,P08
LP04	static-read-positive	0/100/0/0/0	uniform	0	P02, P05, P11, P12, P16, P25
LP05	static-read-zipfian	0/100/0/0/0	zipfian(.99)	0	P10,P20,P25,P28
LP06	static-read-negsweep	0/100/0/0/0	uniform	0–100	P04,P06,P22
LP07	static-read-string-corpus	0/100/0/0/0	real-corpus	50	P04,P06,P10
LP08	range-scan-heavy	0/0/0/100/0	uniform-seq	0	P01,P10,P12,P32
LP09	insert-lookup-5050	50/50/0/0/0	uniform	0	P03,P29
LP10	delete-heavy-post-build	50/0/50/0/0	uniform-rand	0	P01,P08
LP11	mixed-ycsb-oltp	25/50/0/0/25	zipfian(.99)	0	P10,P19,P20
LP12	concurrent-rmw-stress	25/9/36/9/9	uniform	0	P07,P29
LP13	concurrent-read-scaling	0/100/0/0/0	uniform	0	P07,P08,P25,P28
LP14	dynamic-realistic-trace	9/91/0/0/0	tpcc-nonuniform	0	P19,P20

prefetch, telemetry, isa, layout, reclamation), eine Schlüssel-Wert-Signatur und einen optionalen `<expected_workload>`-Tag, der die ExperimentDriver-Heuristik (Kapitel 6) überschreibt. Alle 30 SOTA-Profil und alle 10 Allokator-Profil sind getaggt. Drei der 33 Korpus-Arbeiten besitzen bewusst kein eigenständiges Profil, sondern gehen nur seziert ein: das optimistische Lock-Coupling von ART (P08) als Nebenläufigkeits-Organ des art-Profil, die succinct LOUDS-Repräsentation (P09, Jacobson 1989) als Speicher-Layout-Organ (u. a. in SuRF, P10) und das Profiling-Werkzeug VAMPIR (P33) als Mess-Technologie (Abschnitt 3.5.1); die übrigen 30 Algorithmen tragen je ein eigenes Profil-XML.

Die Workloads verteilen sich auf $13 \times$ YCSB_C (leselastig), $9 \times$ YCSB_A (Zipf-verteiltes Lesen+Aktualisieren), $6 \times$ YCSB_E (Bereichs-Scans) und $2 \times$ YCSB_B (95/5 Lesen/Aktualisieren). PRT-ART selbst ist mit YCSB_F (Read-Modify-Write, für DensityTracker-Aktualisierungen) getaggt. Tabelle 3.7 listet die zehn implementierten Allokator-Profil.

3.4.3 Bias und Vergleichbarkeit

Der Kern-Befund der Lastprofil-Analyse ist ein systematischer *Bias*: Jede Veröffentlichung wählt das Lastprofil, unter dem ihr eigener Entwurf am besten abschneidet. Statische, read-only-orientierte Tries und Filter (HOT, CoCo, B²-Baum, SuRF, CSS) rahmen „build-once, 100 % positive Lookups und meiden Aktualisierungen; komprimierte Tries und Filter (CoCo, B²-Baum, SuRF) machen den Negativ-Anfrage-Anteil zur Primärachse, weil sie bei Misses früh abbrechen; zipfian-zentrierte Arbeiten (B-Trees Are Back, Mahling, Kuehn) begünstigen cache-bewusste Reordering-Layouts; schreib- und nebenläufigkeitslastige Arbeiten (ART, Masstree, Wormhole, OLC-ART, RCU) treten umgekehrt nur dort an, wo statische Strukturen versagen; und prefetch- bzw. hardware-nahe Arbeiten (CSB⁺, Chen, Schmidt) vermessen über Knotengröße und Prefetch eher Cache und TLB als die Algorithmus-Logik. Vierzehn der Veröffentlichungen liefern überhaupt kein eigenes Index-Lastprofil (Survey-, Theorie- und Hardware-Arbeiten). Erst wenn *alle* Lastprofile gegen *alle* Lebewesen-Binaries (und die Container-Hüllen) laufen — der Workload als dynamische Achse (Kapitel 6) —, lässt sich der „Sieger nur im selbstgewählten Heimprofil-Entlarven und bias-freie Vergleichbarkeit gewinnen. Diese Unvollständigkeit der Vorarbeiten ist Teil der Forschungslücke

Tabelle 3.6: SOTA-Algorithmus-Profile mit Workload-Affinität

Profil	Paper-Ref.	expected_workload
art	P01 (Leis 2013)	YCSB_C
hot	P02 (Binna 2018)	YCSB_C
masstree	P03 (Mao 2012)	YCSB_A
coco_trie	P04 (Boffa 2024)	YCSB_C
start	P05 (Fent 2020)	YCSB_C
b2tree	P06 (Schmeisser 2022)	YCSB_E
wormhole	P07 (Wu 2019)	YCSB_A
surf	P10 (Zhang 2018)	YCSB_A
css_tree	P11 (Rao/Ross 1999)	YCSB_C
csb_tree	P12 (Rao/Ross 2000)	YCSB_E
hankins	P13 (Hankins 2003)	YCSB_C
samuel	P14 (Samuel 2005)	YCSB_C
graefe	P15 (Graefe 2001)	YCSB_C
bender_treelayout	P16 (Bender 2002)	YCSB_C
bender_cacheoblivious	P17 (Bender 2005)	YCSB_C
saikkonen_multilevel	P18 (Saikkonen 2008)	YCSB_E
saikkonen_layoutinvariant	P19 (Saikkonen 2016)	YCSB_E
btreesareback	P20 (Mueller 2025)	YCSB_E
chen_prefetch	P21 (Chen 2001)	YCSB_A
chen_fractal	P22 (Chen 2002)	YCSB_A
khan_adaptive	P23 (Khan 2010)	YCSB_A
naderan_tahan	P24 (Naderan-Tahan 2016)	YCSB_A
mahling	P25 (Mahling 2025)	YCSB_E
zhang_fgcs	P26 (Zhang 2024)	YCSB_A
zhang_asplos	P27 (Zhang 2025)	YCSB_A
kuehn	P28 (Kuehn 2023)	YCSB_C
rcu	P29 (McKenney 2001)	YCSB_B
hazard_pointers	P30 (Michael 2004)	YCSB_B
ungethuem	P31 (Ungethuem 2017)	YCSB_C
to_stride	P32 (Schmidt 2025)	YCSB_C

Tabelle 3.7: Allokator-Profile mit Lizenz und Workload-Affinität

Profil	Familie	Lizenz	expected_workload
hoard	A01	Apache-2.0	YCSB_A
michael_lockfree	A03	BSD-3	YCSB_B
mimalloc	A04	MIT	YCSB_A
jemalloc	A05	BSD-2	YCSB_B
tcmalloc	A06	BSD-3	YCSB_C
snmalloc	A07	MIT	YCSB_A
scalloc	A08	BSD-3	YCSB_A
rpmalloc	A10	Public Domain	YCSB_C
lrmalloc	A11	BSD-3	YCSB_B
dlmalloc	A20	CC0	YCSB_C

(Abschnitt 3.7).

3.5 Mess-Technologien als Instanzen

Die in Abschnitt 2.5 eingeführten Güte-Kriterien und Mess-Muster erhalten hier ihre wissenschaftlich nachweisbaren Instanzen — die Technologien, die sie umsetzen.

3.5.1 Erhebungs-Technologien

Hardware-Performance-Zähler (PMC, über `perf`) sind im Stand der Technik nahezu universell und liefern Cache-Miss-Raten (L1/L2/L3), dTLB-Misses, Branch-Misses sowie IPC/CPI — die Mehrzahl der Arbeiten P01–P28 berichtet solche Zähler. *Profiler* wie OTF2/VAMPIR (P33, TU-Dresden) und VTune erheben trace-basierte Laufzeitprofile, *Simulatoren* wie gem5 [BBB⁺11] liefern hardware-genaue Schätzungen, wo reale Zähler fehlen (P25 Mahling, P27 Zhang), und das *HdrHistogramm* (Gil Tene) erhebt die Latenz-Perzentile (p50/p95/p99) und instanziiert damit das Streuungs-Kriterium.

3.5.2 Statistische Methodik

Auf der Erhebung baut die Auswertung auf (Abschnitt 2.5.1, Streuung): Statt arithmetischer Mittel werden Median und Perzentile berichtet, Unterschiede mit dem verteilungsfreien *Mann-Whitney-U-Test* (mit Holm-Korrektur der Familienfehlerrate) auf Signifikanz geprüft und ihre Stärke über das Effektmaß *Cliff's δ* quantifiziert. Diese Methodik folgt den Empfehlungen für wissenschaftliches Benchmarking nach Hoefler und Belli [HB15].

3.5.3 TU-Dresden-Hardware-Messmethodik

Eine eigene methodische Linie bildet die hardware-nahe Messmethodik der TU Dresden (Habich-Gruppe): Ungethüm et al. (P31) systematisieren die Hardware-Optimierungen für Datenbank-Maschinen, Schmidt et al. (P32) vermessen Stride- gegen Sequenz-Zugriffe auf HBM mit dTLB- und L2/L3-Zählern, und das VAMPIR-Werkzeug (P33) profiliert nicht-funktionale Eigenschaften heterogener Speicher. Diese Linie bindet die vorliegende Arbeit unmittelbar an das Datenbank- und Software-Engineering-Umfeld ihrer Hochschule an.

3.5.4 Mess-Kategorien und Brücke zur eigenen Apparatur

Aus diesen Technologien ergibt sich der Satz der Mess-Kategorien, den die eigene Apparatur führt: Cache-Line-Auslastung, Cache-Misses (L1/L2/L3), dTLB-Misses, Speicher-Fußabdruck, Branch-Misses, IPC/CPI, Latenz (Mittel und Perzentile p50–p999), Durchsatz, Energie und Fill-Buffer-Auslastung. Davon löst die eigene Apparatur gegenwärtig die zeit- und observer-basierten Kategorien auf (Wall-Clock, HdrHistogramm, Per-Achsen-Observer, die Statistik-Triade, eine Zwei-Phasen-Operationsschleife und ein Konformitäts-Gatter gegen `std::map`); die zählerbasierten PMC-Kategorien sind vorgesehen, ihre Erhebung jedoch an Hardware-Zähler-Zugriff gebunden — die vollständige Mess-Architektur entwickelt Kapitel 6. Welche dieser Instanzen das vorgeschlagene Framework übernimmt und welche es als überholt

ersetzt, analysiert Kapitel 4 dialektisch (These–Antithese–Synthese): jede Vorarbeit (These) trifft im bias-freien Quervergleich auf ihre Grenzen (Antithese), woraus die tieferen Ergebnisse der eigenen Apparatur als Synthese hervorgehen. Damit schließt der Stand der Technik unmittelbar an die Forschungslücke an (Abschnitt 3.7).

3.6 Architektur-Design und Software-Engineering von Suchstrukturen

Über die technischen Mechanismen hinaus ist das in dieser Arbeit vorgeschlagene Framework primär eine *Architektur-Design*-Erfindung. Dieser Abschnitt ordnet es in den nicht-technischen Entwurfs- und Software-Engineering-Raum ein — die Definitions-Klassen aus Abschnitt 2.6 (Design-Pattern, Metaprogrammierung) erhalten hier ihre wissenschaftlichen Instanzen.

3.6.1 Datenstrukturen als Entwurfsraum

Die Idee, Datenstrukturen nicht einzeln, sondern als *Entwurfsraum* zu betrachten, prägt Idreos’ *Periodic Table of Data Structures* [IZA⁺18] und den *Data Calculator* [IZH⁺18]: Strukturen entstehen aus der Kombination weniger Entwurfs-Primitive, deren Kosten sich synthetisieren lassen. Das deckt sich unmittelbar mit der Achsen-Sezierung dieser Arbeit (Abschnitt 3.3) — die klassischen Familien sind bloße Lese-Gruppierungen orthogonaler Achsen-Belegungen, ein konkretes Verfahren ein Punkt im kartesischen Produkt der Achsen.

3.6.2 Selbst-designende und gelernte Indizes

Eine zweite Linie verlagert den Entwurf in die Laufzeit: *Database Cracking* [IKM07] und seine robuste Variante [HIKY12] bauen den Index als Nebenprodukt der Anfrageverarbeitung adaptiv auf, während *gelernte Indizes* [KBC⁺18] die Struktur durch ein Modell der Schlüsselverteilung ersetzen; jüngere Arbeiten (VEGA, SIGMOD 2025; AirIndex) führen dies aktiv-tunend fort. Diese Ansätze zeigen denselben Trend — weg von der fest verdrahteten Einzelstruktur, hin zu konfigurierbaren, sich anpassenden Entwürfen —, dem die vorliegende Arbeit mit der permutierbaren Achsen-Komposition eine systematische, mess-getriebene Form gibt.

3.6.3 Design-Pattern-getriebene Architektur als Beitrag

Der eigentliche Beitrag liegt damit weniger in einer einzelnen Struktur als in einem *Werkzeug*: Statt das Vergleichsproblem direkt zu lösen, konstruiert diese Arbeit zuerst die „Schaufel– den Permutations- und Mess-Apparat —, mit der sich beliebige Achsen-Kombinationen erzeugen und fair vermessen lassen. Software-technisch ist dieser Apparat eine Komposition benannter Entwurfsmuster (Abschnitt 2.6.1) und zweier neuer Metaprogrammier-Muster (Abschnitt 2.6.3). Wissenschaftlich ordnet er sich in das Software-Engineering der *Komposition* ein: die invasive Software-Komposition aus Gray-Box-Komponenten [Aßm03] (TU Dresden), die generative Programmierung [CE00] und die feature-orientierten Software-Produktlinien [ABKS13, BO92] — eine Achse entspricht dabei einem Feature, ein Lebewesen einer Feature-Komposition.

Provenienz des Achsen-Konzepts. Das Konzept der Achsen-Architektur selbst ist keine Neuentwicklung dieser Arbeit, sondern aus einer Vorarbeit des Autors im Rahmen der von ihm gehaltenen Marke Comdare bzw. der BEP Venture UG übernommen, wo es ursprünglich — im Kontext der UltiHash-Datenbank — der Optimierung von Deduplikations-Verfahren diente. Der Beitrag der vorliegenden Arbeit ist die Übertragung und Ausformung dieses Konzepts auf cache-bewusste Suchstrukturen sowie seine design-pattern-getriebene, mess-orientierte Realisierung. *Welche* fremden Instanzen das Framework übernimmt und welche es als überholt ersetzt, entwickelt Kapitel 4 dialektisch (vgl. Abschnitt 3.5.4).

3.7 Forschungslücke

Die Sezierung in Abschnitt 3.3 legt die zentrale Lücke offen: Jede der untersuchten Arbeiten optimiert nur wenige Achsen isoliert, und *kein einziges Verfahren belegt alle neunzehn Achsen gemeinsam*. Manche Achsen sind im Stand der Technik ausschließlich mit Baseline- oder Lehrbuch-Varianten besetzt (das Mapping zwischen Algorithmus- und Cache-Traversierung), andere treten nur als *Mess-Ziel* auf (der dTLB), und die räumliche Such-Klasse fehlt ganz; Achsen wie Ein-/Ausgabe, Migration, Filter und Pufferung sind zwar je einzeln belegt, aber nie zusammen mit den Such-Achsen in einem Entwurf.

Hinzu kommt die methodische Lücke: Die Ergebnisse werden in nicht vergleichbaren Settings berichtet, und der Paper-Bias — jede Veröffentlichung wählt ihr Heimspiel-Lastprofil (Abschnitt 3.4.3) — verhindert einen fairen Quervergleich. Es fehlt eine bias-freie Vermessung *aller* Lastprofile gegen *alle* Verfahren auf einer gemeinsamen Basis.

Was fehlt — und was diese Arbeit beiträgt — ist daher die systematische, orthogonale *Achsen-Zerlegung* dieser Entwürfe samt einer *permutierten Mess-Architektur*, die sie als explizite Konfigurationen rekonstruiert und auf einer gemeinsamen Basis vermisst. Welche der hier katalogisierten Instanzen das vorgeschlagene Framework übernimmt und welche es als überholt ersetzt, entwickelt das folgende Kapitel dialektisch (These–Antithese–Synthese, Abschnitt 3.6.3); daraus gehen die tieferen Ergebnisse der eigenen Apparatur hervor.

4 Konzept und Architektur

Dieses Kapitel präsentiert den Kernbeitrag der Arbeit: ein Achsen-Bibliotheks-Framework, das cache-bewusste Suchalgorithmen in austauschbare, orthogonale Bausteine-Achsen zerlegt, die geschichtete Engine-Architektur, die es realisiert, die ABI-stabile Modul-Schnittstelle, die neunzehn-Achsen-Permutationsmatrix, den Prüfling PRT-ART und den Builder, der die Drei-Stufen-Prüfung orchestriert.

4.1 Das Achsen-Bibliotheks-Framework

Die zentrale Idee besteht darin, einen Algorithmus nicht als Monolith zu behandeln, sondern als *Komposition orthogonaler Achsen*, wobei jede Achse eine Teilentscheidung eines cache-bewussten Index repräsentiert (wie Seiten angeordnet werden, wie Knoten verzweigen, wie Speicher alloziert wird, wie Nebenläufigkeit behandelt wird usw.). Ein konkreter Algorithmus ist dann ein Punkt im kartesischen Produkt aller Achsen, und ein Stand-der-Technik-Entwurf wird als eine explizite Konfiguration rekonstruiert. Die entscheidende Eigenschaft ist die *modulare Austauschbarkeit*: Forschende können eine neue Achsen-Variante isoliert untersuchen — und ihren Effekt pro Achse und für den gesamten Algorithmus messen — ohne alle anderen neu zu implementieren. Das macht den Vergleich zweier `std::map`-Implementierungen (Abschnitt 2.3.1) zu einem kontrollierten, reproduzierbaren Experiment.

Diese Zerlegung beruht auf zwei Beobachtungen. Erstens teilen Suchalgorithmen im großen Bild dieselbe *Baum-Anatomie* in unterschiedlicher Ausprägung — von ausgewogenen Suchbäumen über Tries und Radix-Bäume bis zu eigenentwickelten Konstrukten; selbst eine sortierte Schlüssel-Wert-Liste ist streng genommen ein einzelnes Blatt eines Baumes, dessen Werte über Verweise weiter auf Daten-Blätter zeigen. Die Baum-Struktur ist damit universell über alle Suchalgorithmus-Architekturen hinweg und bildet die Leinwand, auf der die Achsen platziert werden. Nicht-baumartige Suchverfahren — etwa eine Hash-Tabelle — fügen sich in dasselbe Achsen-System ein, indem sie die baum-spezifischen Achsen (Pfadkompression, Knotentyp, Bereichs-Index) als Durchreich-Varianten belegen und nur das gemeinsame Achsen-Subset tragen; sie bleiben dieselbe Gattung mit eingeschränktem Profil. Eine eigene Gattung entstünde erst, wenn sich der Basis-Achsen-Satz grundlegend unterschiede. Zweitens fasst eine anschauliche Metapher einen konkreten Algorithmus als *Lebewesen* auf, dessen *Organe* die Achsen sind: Jeder Algorithmus besitzt ein Knoten-Layout-Organ, ein Traversal-Organ, ein Allokations-Organ und so weiter, jeweils in anderer Ausprägung. Die Permutation der Achsen gleicht dann einem genetischen Experiment — der Austausch von Organen zwischen Lebewesen erzeugt neue Komposit-Algorithmen mit bisher unbekanntem Cache-Line-Verhalten. Das Forschungsziel ist es, diese gemeinsame Anatomie und im resultierenden Entwurfsraum (*design space*) [IZA⁺18, IZH⁺18] das schnellste Komposit für ein gegebenes datentyp-spezifisches Lastmuster zu finden.

4.2 Vier-Subsystem-Trennung (M-Modell)

Die Diplomarbeit, der Cache-Engine-Builder, die Cache-Engine und der Prüfling sind *vier formal getrennte Subsysteme* mit unterschiedlichen Verantwortlichkeiten. Die Trennung ist orthogonal zur Drei-Repository-Aufteilung (Abschnitt 5.1): Builder und Engine wohnen im selben Repository, sind dort aber eine ausführbare Datei bzw. eine Bibliothek.

1. **messung_driver** (Diplomarbeit/Code) — der Auswertungs-Orchestrator (äußere Schleife): liest die drei Messreihen-XML-Konfigurationen (A/B/C), triggert pro Reihe den Builder und sammelt die Ergebnisse für den Anhang.
2. **CacheEngineBuilder** (`cache-engine/apps`) — ein autonomes Plattform-Ausmess-System, das die Sieben-Phasen-Pipeline implementiert (discover, measure, classify, publish, bind, execute, compare) und je registrierter Engine den Permutationsraum enumeriert.
3. **CacheEngine** (`cache-engine/libs`) — die Werkzeug-Bibliothek: sie stellt für jede der neunzehn Bausteine-Achsen (Abschnitt 4.5) die Standard-Implementierungen und die zugehörigen Cache-Strategie-Familien als C++23-Concept-API bereit, aus denen der Builder die Permutations-Konfigurationen zusammensetzt.
4. **Prüfling (PRT-ART)** (`prt-art`) — eine bei der Cache-Engine als `IExecutingEngine` registrierte Algorithmus-Implementierung; sie implementiert die Such-Engine-Schicht für jede Achse und konsumiert Cache-Engine-Dienste während ihrer eigenen Lookup-/Insert-/Scan-Operationen.

Die definierende Eigenschaft des M-Modells ist die *bidirektionale* Cache-Engine ↔ Prüfling-Beziehung: (a) die Cache-Engine instantiiert den Prüfling je Permutations-Konfiguration (Builder-Phase 5, BIND), und (b) der Prüfling ruft zur Laufzeit Cache-Engine-Dienste (Telemetry, Prefetch, Heuristik) während seiner eigenen Lookups auf (Phase 6, EXECUTE). Diese Symmetrie ermöglicht die *Multi-Prüfling*-Fähigkeit: Messreihe A vergleicht PRT-ART gegen viele SOTA-Adapter in einer einzigen Builder-Pipeline.

4.3 Drei-Schichten-Hierarchie

```
CacheEngine           (Schicht 1: Stand der Technik, Werkzeug-Bibliothek)
  ^ erbt
ExecutionEngine<ProcessingStrategy> (Schicht 2: OS-Primitive)
  ^ erbt
SearchEngine<Collection, Config>    (Schicht 3: Such-Muster)
  ^ spezialisiert
PrtArtSearchEngineAdapter           (Prüfling-Anbindung)
```

Schicht 1 (CacheEngine) stellt die Standard-Bausteine der neunzehn Achsen — etwa Memory-Layout, Allokator-Familien, Prefetch und Telemetry — als Bibliothek bereit. **Schicht 2 (ExecutionEngine)** kapselt diese zu höheren OS-Primitiven (cache-line-aligned Allokation, NUMA-konformes Read-Pin, coherence-aware Write). **Schicht 3 (SearchEngine)** bietet such-spezifische Muster (Trie-Walk, B⁺-Bereichs-Scan, Hot-Path-Lookup) auf den Execution-Primitiven.

4.4 ABI-stabiles C++23-Modul-Interface

Das ABI folgt dem Prinzip variadischer Template-Parameter: ein Parameter $\Rightarrow$ `std::vector-API`, zwei Parameter $\Rightarrow$ `std::map-API`, $N > 2 \Rightarrow$ `std::map<K, std::tuple<V1...VN>>`. Komplexe Schlüssel werden von einer Fingerprint-Komponente auf 16-Byte-Binärstrings fester Länge reduziert. Die Modulgrenze ist binär-stabil, sodass der Builder vorkompilierte Permutations-Module laden (LoadLibrary/dlopen) und den ABI-Vertrag je Modul verifizieren kann.

4.5 Die Bausteine-Achsen

Nach der N-Phasen-Erweiterung und der Konsolidierung zur Suchalgorithmus-Anatomie umfasst die Matrix **neunzehn Hauptachsen plus etwa siebenundfünfzig Sub-Achsen**: (1) Search-Algorithm, (2) Cache-Traversal, (3) Mapping, (4) Path-Compression, (5) Node-Type, (6) Memory-Layout, (7) Allocator (Allocation, Reclamation, NUMA, Huge-Page, Free-List), (8) Prefetch, (9) Concurrency (Pattern, Locking-Mode), (10) Serialization, (11) Telemetry, (12) Value-Handle, (13) ISA, (14) Index-Organization, (15) I/O-Dispatch, (16) Migration, (17) Filter, (18) Queuing-Buffer und (19) Flush-Policy — die siebzehn Such-Achsen plus die beiden Queuing-Achsen, genau die neunzehn Organe, die das `IsComposition`-Concept je Algorithmus erzwingt. Je Achse enumeriert ein `std::variant` alle Konkretisierungen (allein die Allocator-Achse bietet rund zwei Dutzend); das kartesische Produkt ergibt weit mehr als 10^{11} Konfigurationen (Abschnitt 6.7).

4.6 PRT-ART als Prüfling

Der `PrtArtSearchEngineAdapter` implementiert das `SearchEngine-ABI` durch *Komposition* statt direkter Vererbung, sodass die hybride `PrtArtSearchEngine<Ts...>-API` unberührt bleibt; das Adapter-Muster ermöglicht einen Compile-Time-Fallback auf Cache-Engine-Bausteine (`resolve_baustein.hpp`) dort, wo der Prüfling eine Achse nicht überschreibt. PRT-ART steuert eigene Implementierungen in den Achsen Page, Layout, Free-List, Prefetch, Concurrency-Pattern und Measurement bei (4+2-Pool-Allokator, Distance-Estimator-Prefetch, OLC mit reservierten Wert-Blöcken sowie die H1/H2/H3-Metriken); für die übrigen Achsen verwendet er bestehende SOTA-Bausteine über die Permutations-Konfiguration wieder.

4.7 CacheEngineBuilder und Drei-Stufen-Prüfung

Der Builder orchestriert eine Drei-Stufen-Prüfung: **Stufe 1** (nur Cache-Engine) nutzt die Standard-Variante je Achse; **Stufe 2** (nur Prüfling) ersetzt die überschriebenen Achsen durch die Prüfling-Varianten und behält sonst die Cache-Engine-Standards; **Stufe 3** (Full Join, nicht-redundant) kombiniert die Standard- und alle Prüfling-Varianten. Jede Stufe erzeugt tausende nicht-redundante Permutations-Binaries, die zur Laufzeit jeweils als ABI-stabiles C++23-Modul geladen werden. Die Stufen bilden direkt die drei Pflicht-Messreihen aus Kapitel 6 ab. Nach Auswertung der feingliedrigen Messergebnisse wählt der Builder den besten Gesamtalgorithmus aus und liefert ihn als eigenständige, ABI-stabile Binary hinter dem einheitlichen Suchalgorithmus-Interface aus.

5 Implementierung

Dieses Kapitel beschreibt die Implementierung thematisch (nicht als chronologisches Entwicklungsprotokoll): die Drei-Repository-Aufteilung, die SOTA- und Allokator-Adapter, die Permutations-Codegenerierung und das Flag-System, die Concept-/CRTP-Realisierung der Achsen, Nebenläufigkeit und Speicherfreigabe, die cache-kohärenz-bewusste Telemetrie und die Mess-Pipeline.

5.1 Drei-Repository-Architektur

Die Implementierung verteilt sich auf drei Git-Repositoryn mit klar getrennten Verantwortlichkeiten:

- `comdare-cache-engine` — **wie** gemessen wird: die Werkzeug-Bibliothek und der Builder. Kernkomponenten: der Builder (XML-Config-Parser, `generate_module_from_profile`-Codegenerierung, Permutations-Schleife, Modul-Loader und der sieben-phasige `ExperimentDriver`); die ABI-Header (`search_engine`, `execution_engine`, `baustein_variants`, `resolve_baustein`); 30 SOTA-Profil unter `algorithm_profiles/sota`; und die Test-Suites. Der Quellbaum folgt einem Pitchfork-Layout (`apps/` für ausführbare Dateien, `libs/` für Bibliotheken, `libs/common` für querschnittlichen Code).
- `comdare-prt-art` — der **Prüfling**: die hybride `PrtArtSearchEngine` (Vector/Map/Tuple-API), ihre ABI-Adapter, ihre Bausteine (4+2-Allokator-Pools, OLC mit reservierten Blöcken, MultiLevel-Layout, pfadorientiertes Prefetch, DensityTracker, Hypothesen-Metriken H1/H2/H3) und die prüflingseitigen Re-Implementierungen.
- `probst-Diplomarbeit-cache-engine` — **was** getestet wird: der `messung_driver`, die Experiment-Konfigurationen, die Auswertungs-Pipeline und das Manuskript.

5.2 Adapter für Stand-der-Technik-Algorithmen und Allokatoren

Jeder SOTA-Entwurf und jeder Allokator wird über einen dünnen Adapter integriert, der die Original-Implementierung hinter dem Engine-ABI bereitstellt. Die Adapter folgen einem einheitlichen Muster: ein `COMDARE_HAVE_<X>`-Compile-Flag schützt einen `#if defined`-Block, der die Original-API verdrahtet, mit einem sicheren Fallback (z. B. `std::malloc` oder eine typisierte Ausnahme), wenn das Original nicht verfügbar ist. Das hält den Build eigenständig lauffähig und erlaubt zugleich, die Original-Quellen je Abhängigkeit zu aktivieren. Die Integration des hierarchischen Prefetching (P27) liefert zusätzlich einen Call-Graph-Analyzer zur Übersetzungszeit (`p27_bundle_finder`, ein C++23-Port des Original-Werkzeugs `hp_soft`), der Funktionen mit Subtree-Footprint oberhalb eines Schwellwerts als Bundle-Einstiegspunkte identifiziert.

5.3 Permutations-Codegenerierung und Flag-System

Der Builder überführt jedes Profil in ein vorkompiliertes C++23-Modul (`generate_module_from_profile`). Jede Permutation ist durch eine bit-gepackte *Permutations-Flag*-Struktur identifiziert. Mit der N-Phasen-Verfeinerung der Matrix wächst das Flag-System von 9 Bänken (~50 Bit) auf einen breiteren, mehrbänkigen bit-gepackten *Permutations-Identifizier* mit Sub-Bank-Bitfeldern, der die neunzehn Achsen und ihre Sub-Achsen widerspiegelt. Ein Constraint-Filter schließt ungültige Achsen-Kombinationen vor jeder Modulerzeugung aus.

5.4 Concept-/CRTP-Realisierung der Achsen

Jede Achse ist ein Topic-Verzeichnis, das ein zweischichtiges Concept (ein breites Topic-Concept und ein enges Achsen-Concept) sowie eine durch eine `requires`-Klausel abgesicherte CRTP-Basisklasse enthält. Die Prüfling-gegen-Standard-Auswahl je Achse wird zur Übersetzungszeit über `resolve_baustein.hpp` (`std::conditional_t` über Tag-Spezialisierungen) aufgelöst, sodass der Hot-Path weder `virtual`-Aufrufe noch Laufzeit-Switches enthält — die zero-cost abstraction aus Abschnitt 2.6.

5.5 Nebenläufigkeit und Speicherfreigabe

Nebenläufigkeit ist selbst eine Achse (8.1 Pattern, 8.2 Locking-Mode), realisiert über einen Concurrency-Manager, der die Disziplinen OLC, HTM, STM, Lock-Free, RCU und Hazard Pointers koordiniert. Die Speicherfreigabe (Reclamation) ist eine Sub-Achse des Allokators (6.2): epoch-basierte, RCU-, Hazard-Pointer- und QSBR-Strategien sind je Permutation wählbar. PRT-ART steuert OLC mit reservierten Wert-Blöcken als Nebenläufigkeits-Variante bei.

5.6 Telemetrie gegen Cache-Kohärenz-Effekte

Per-Node-Zähler verursachen auf Mehr-Kern-Systemen Cache-Line-Ping-Pong, was genau jene Messungen verfälscht, die sie erheben. Der Kuehn-Forschungslinie (P28) folgend bietet die Telemetrie-Achse daher cache-kohärenz-bewusste Strategien: *Leaf-Only-Zähler* (die Hauptvariante), *Sampling* (jeder N -te Blattzugriff) und *Offline-Recompute* (Bottom-up-Aggregation vor dem Reordering); der klassische Per-Node-Zähler bleibt nur als gekennzeichnetes Anti-Pattern zum Vergleich erhalten.

5.7 Mess-Pipeline

Die Auswertungs-Werkzeugkette ist eine Folge kleiner Programme: `sample_data_generator` (deterministische Testdaten ohne reale Hardware) → `messung_driver` (steuert die Reihen) → `binary_to_csv` (deserialisiert die binären Mess-Records) → `csv_to_latex` (booktabs-Tabellen mit Algorithmus-Steckbriefen pro Profil) → `diagram_generator` (A4-bewusste TikZ-Plots) → `latex_to_pdf`. Der Mess-Pfad in der Execution-Engine wird nur unter

`-DCOMDARE_EXPERIMENT_MODE=ON` mitkompiliert, sodass der Produktiv-Pfad keinen Mess-Overhead trägt. Continuous Integration (GitLab CI plus ein gespiegelter GitHub-Workflow) läuft über alle drei Repositorien.

6 Evaluationsmethodik

Dieses Kapitel beschreibt, wie gemessen wird; die konkreten Ergebnisse folgen in Kapitel 7.

6.1 Drei Messreihen

Die Architektur sieht drei Messreihen vor, die alle vom `messung_driver` über die `ExperimentDriver`-Bibliothek ausgeführt werden, konfiguriert entweder als drei feste Reihen oder über eine externe `messreihen.xml`.

Reihe A — PRT-ART vs. Stand der Technik. Vergleicht den Prüfling gegen die dokumentierten SOTA-Algorithmen. Zwei Modi: *A_defined* (schnelle Verifikation gegen die 8 Rang-1-Profile) und *A_full* (alle 30 SOTA-Profile; der Standard, gemäß der Direktive, so vollständig wie möglich zu messen).

Reihe B — Cache-Engine-Permutationen. Die reine “Stand der Technik”-Vergleichsbasis: alle SOTA-Profile ohne den PRT-ART-Prüfling.

Reihe C — Merge alt/neu. Konkrete Merge-Punkte zwischen PRT-ART-Bausteinen und Cache-Engine-Permutationen, z. B. PRT-ART OLC + `tcmalloc` + ART-Page; PRT-ART 4+2-Pool + HOT-Page; PRT-ART Path-Prefetch + B²-Baum-Page; PRT-ART MultiLevel-Layout + Wormhole-Page.

Jede dieser drei Reihen wird in drei Granularitäten erhoben (Aufgabenstellung-Methodik): Das *Micro-Benchmarking* vergleicht jeden einzelnen Entwurfsbestandteil (Achse) isoliert über das gemeinsame Interface seiner Kategorie gegen Lastprofile; das *Makro-Benchmarking* misst die einzelnen `std::map`-Interface-Operationen (Einfügen, Punkt- und Bereichssuche, Aktualisieren, Löschen) gegen Beispiel-Lasten; und das *Gesamt-Benchmarking* fährt die etablierten YCSB-Lastprofile über jede Rekombination der Bausteine. Aus dem Ranking über diese drei Granularitäten ergibt sich, welche Entscheidung an welcher Stelle für welches Lastmuster vorteilhaft ist.

6.2 ExperimentDriver-Phasen

Der `ExperimentDriver` durchläuft sieben Phasen je Reihe: (1) **Enumerate** (XML-Configs parsen und die Permutationsliste aufbauen); (2) **Codegen** (ein `comdare_perm_<fp>.cpp` je Permutation, plus `generate_module_from_profile` für die SOTA-Profile); (3) **Compile**; (4) **Load** (LoadLibrary/dlopen aller Module); (5) **Execute** (`create_instance` + profilbewusstes `run_workload`); (6) **Measure** (die binären Mess-Records aggregieren); (7) **Persist** (`measurements.csv` + `.json`). Zwei opt-in-Phasen kompilieren fehlende Module hot und verifizieren den ABI-Vertrag je Modul.

6.3 Hypothesen

H1 (Page-Type-Kosten) Die mittleren Zugriffskosten je Page-Type-Variante unterscheiden sich systematisch nach Workload: dichte Page-Typen gewinnen erwartungsgemäß unter Zipf-Verteilung (YCSB-A), bereichsoptimierte Pages unter Bereichs-Scans (YCSB-E).

H2 (Code-Qualität pro Quelle) Der vom Original-Quell-Repository geerbte Code-Qualitäts-Score korreliert messbar mit dem erreichten Durchsatz.

H3 (Inline- vs. External- vs. Chain-Verteilung) Die beobachtete Verteilung der drei ValueHandle-Varianten korreliert mit der Page-Dichte: dichte Pages bevorzugen Inline, spärliche Pages External und PRT-ART-Heuristiken ChainRef.

6.4 Workloads und Datensätze

Phase 5 leitet den YCSB-Workload je Permutation aus dem Traversal-Tag des Profils ab (Tabelle 6.1); Module ohne Profil verwenden den globalen Default-Workload.

Tabelle 6.1: Profil-bewusstes Workload-Routing

Traversal-Tag	YCSB-Workload	Charakteristik
RANGE_SCAN / RANGE_HOT_PATH	YCSB-E	Bereichsabfragen
HOT_PATH / PRTART_HOT_PATH	YCSB-A	50/50 Lesen/Aktualisieren, Zipf
STANDARD (Default)	YCSB-C	nur Lesen

Als Datengrundlage dienen reale String-Datensätze unterschiedlicher Charakteristik (Tabelle 6.2); jeder trägt eine Reproduzierbarkeits-Akte aus Quelle, Prüfsumme, Zeilenzahl, Vorverarbeitung und Seed-Regel für die synthetischen Miss-, Update- und Delete-Schlüssel.

Tabelle 6.2: Reale Datensätze und ihre Charakteristik

Datensatz	Charakteristik
url	sehr lange Schlüssel, lange gemeinsame Präfixe, mittleres Alphabet
dna	kleines Alphabet, hohe Wiederholung, tiefe Präfix-Entropie
protein	biologische Sequenzen, kleines bis mittleres Alphabet
xml	strukturierte Strings mit wiederkehrenden Präfixen
tpcds-id	datenbanknahe IDs, sortiert und unsortiert
trec-terms	Wörterbuch-/Suchindex-Terme mit definierten Miss-Schlüsseln

6.5 Versuchsplattformen

Die Messungen zielen auf eine lokale Workstation zur Entwicklungs-Verifikation und den TU-Dresden-ZIH-Cluster (Barnard CPU, Capella GPU) für die vollständigen Sweeps, plus die Rang-3-Plattformen (Ryzen 9 9950X3D, Intel i9 14900KS und eine HBM-Plattform). Ein `IPlatformProbe` (Phase 1)

meldet je Plattform die lauffähige Teilmenge der Achsen (z. B. AVX-512 nur dort, wo verfügbar, HBM-aware nur auf HBM-Hardware). Je Plattform werden ein Scalar-, ein AVX2- und — auf Barnard — ein AVX-512-Build vorbereitet; auf Hybrid-CPU's werden P- und E-Cores getrennt vermessen (eigene Zähler-Domänen `cpu_core/cpu_atom`) statt zusammenaggregiert.

Experiment-Betriebssysteme. Auf den Produktions-Zielmaschinen wird jede Messung unter zwei Betriebssystem-Regimes erhoben, um *Produktions-Treue* von *Mess-Tiefe* zu trennen. Ein *immutable* Betriebssystem (Talos) spiegelt den Produktiv-Einsatz und erzwingt boot- bzw. übersetzungsstatische Cache-Einstellungen (Abschnitt 2.1.5); ein *root-Linux mit vollem Hardware-Zähler-Zugriff* (`perf/MSR`) macht hingegen die zählerbasierten Mess-Kategorien — Cache-, dTLB- und Branch-Misses sowie IPC/CPI (Abschnitt 3.5.4) — und die laufzeit-konfigurierbaren cache-nahen Achsen-Parameter (Prefetch-Distanz, Hardware-Prefetcher) überhaupt erst zugänglich. Die zeit- und observer-basierten Kategorien (Wall-Clock, HdrHistogramm, Per-Achsen-Observer) laufen unter beiden Regimes identisch und sichern die Vergleichbarkeit; die PMC-Kategorien werden nur unter dem privilegierten Regime erhoben und je Plattform mit Betriebssystem, Kernel und Zähler-Domäne protokolliert.

6.6 Fairness und Reproduzierbarkeit

Faire Vergleichbarkeit ist über feste Regeln gesichert. Jeder Vergleich trennt einen gemeinsamen Minimalmodus (Common-Denominator: externe Werte-Handles, keine PRT-ART-Spezialpfade) vom PRT-ART-Native-Modus (Inline-Umschaltung, Cache-Engine, Seitentyp-Scheduler). Alle Verfahren erhalten identische, normalisierte Byte-Schlüssel sowie identische Miss-, Update- und Delete-Mengen; Warmup, Randomisierung und Thread-Pinning werden je Plattform und Workload protokolliert. Fehlende Fähigkeiten einer Baseline — etwa Bereichs-Scans einer Hash-Tabelle — werden als N/A ausgewiesen statt über Hilfsstrukturen verdeckt emuliert; für jede Fremdbibliothek werden Compiler, Flags, ISA-Pfad, Allokator und Commit-Hash mitgeloggt. Für die Reproduzierbarkeit wird jede Konfiguration in mehreren getrennten Läufen vermessen, deren Streuung Teil der Aussage bleibt: Perzentile werden über HDR-Histogramme bestimmt statt gemittelt.

6.7 Permutations-Explosion und Reduktion

Mit der N-Phasen-Erweiterung von 11 auf 19 Achsen plus ~ 57 Sub-Achsen wächst der Raum von $\sim 5,5$ Milliarden auf weit mehr als 10^{11} Permutationen. Drei Mechanismen dämpfen die Explosion: (1) der `<expected_workload>`-Profil-Filter routet nur kompatible Workloads; (2) `MessreihenMode::Defined` führt nur die deklarierten Tupel aus (~ 100 – 1000 je Reihe), die in das Manuskript einfließen; (3) `Full` (und das realistischere `Full-Sampled`, z. B. jede 1000. Permutation) läuft nur auf dem ZIH-Cluster, unter Beachtung der vom Plattform-Probe gemeldeten Achsen-Kompatibilität.

7 Ergebnisse und Auswertung

Dieses Kapitel präsentiert die Mess-Ergebnisse. Konkrete Diagramme und Tabellen werden aus den Mess-Läufen generiert und aus den (sprachneutralen) Verzeichnissen `tikz/` und `tabellen/` eingebunden; siehe die Platzhalter unten. Zum Zeitpunkt der Erstellung sind die Methodik und die Auswertungs-Pipeline vorhanden und auf Beispieldaten verifiziert; die Läufe auf realer Hardware stehen aus (Abschnitt 8.2).

7.1 Auswertungs-Pipeline

Pro Reihen-Lauf schreibt der `messung_driver` unter `Code/_runs/<date>/<spec_id>/` die Dateien `measurements.csv` (eine Zeile je Permutation: Cycles, L1/L2/L3-Misses, Branches, Durchsatz), `measurements.json` und optionale binäre Records. Die Auswertung nutzt `binary_to_csv` (deserialisieren und um Profil-ID, Paper-Ref und Achsen anreichern), `csv_to_latex` (booktabs-Tabellen mit Steckbriefen je Profil) und `diagram_generator` (A4-bewusste TikZ-Balkendiagramme, Scatter-Plots, Heatmaps). Das Manuskript bindet diese über `\input{tikz/<spec_id>/...}` und `\input{tabellen/<spec_id>...}` ein.

7.2 Messreihe A: PRT-ART vs. Stand der Technik

Die Hypothesen H1–H3 (Abschnitt 6.3) werden hier ausgewertet, sobald die Mess-Daten verfügbar sind.

7.3 Messreihe B: Cache-Engine-Permutationen

7.4 Messreihe C: Full Join

7.5 Achsen-Sensitivitätsanalyse

Anhand der Records je Permutation quantifiziert dieser Abschnitt auf der Micro-Granularität, welche Achse am meisten zur Performance-Varianz beiträgt; das Makro- und das Gesamt-Benchmarking ergänzen die Operations- und die YCSB-Lastprofil-Ebene. Aus der Rangbildung über die drei Granularitäten folgt die Empfehlung von Standard-Konfigurationen je Workload-Klasse.

7.6 Diskussion

Die Ergebnisse werden vor dem Hintergrund des Leitmotivs der Arbeit diskutiert: dem Übergang vom heuristischen zum informierten Cache-Management, d. h. ob telemetrie-getriebene, automatisch adaptierte Konfigurationen statisch gewählte übertreffen.

8 Fazit und Ausblick

8.1 Beantwortung der Forschungsfragen

FF0 (Hauptfrage). Wie stark aktive, messgetriebene cache-bewusste Entscheidungen ihre passiven, statisch ausgelegten Gegenstücke übertreffen, lässt sich abschließend erst nach den End-to-End-Läufen je Plattform beziffern (Abschnitt 8.3). Vorbereitet ist die Apparatur, die genau diese Differenz isoliert: dieselbe achsen-konstante Struktur wird einmal aktiv (messgetriebene Layout-, Prefetch- und Value-Ablage-Entscheidung) und einmal passiv (statisch ausgelegt) permutiert und unter identischer Compiler- und Flag-Basis auf Hybrid-CPU und Sapphire-Rapids gegenübergestellt.

FF1 (Komposition). Die neunzehn Permutations-Achsen (Search-Algorithm, Cache-Traversal, Mapping, Path-Compression, Node-Type, Memory-Layout, Allocator, Prefetch, Concurrency, Serialization, Telemetry, Value-Handle, ISA, Index-Organization, I/O-Dispatch, Migration, Filter und die beiden Queuing-Achsen) spannen einen Konfigurationsraum von weit mehr als 10^{11} Permutationen auf. Standard-Technik-Entwürfe werden über Algorithmus-Profil-XMLs eindeutig rekonstruiert: 30 SOTA-Profile (8 Rang-1 + 22 Rang-2/3) zeigen, dass die Achsen-Zerlegung P01–P32 trägt. Die variadische API-Spezialisierung (1/2/N Parameter auf `std::vector/std::map/Tuple`) macht sowohl algorithmus- als auch container-orientierte Muster auf einer Engine ausdrückbar. Die nicht voll-orthogonalen Achsen-Kopplungen (Knotentyp $\times$ SIMD $\times$ Pfadkompression) werden dabei als solche markiert und bei der Permutation explizit eingeschränkt.

FF2 (Mess-Methodik). Die Messung ist in einen Produktiv-Modus (`COMDARE_EXPERIMENT_MODE=OFF`, kein Overhead) und einen Experiment-Modus (der ResultAggregator ist integraler ExecutionEngine-Member) geteilt. Profil-bewusstes Workload-Routing wählt den Workload je Permutation aus dem Traversal-Tag und sichert so faire Vergleiche zwischen bereichsabfrage- und punktzugriffsorientierten Entwürfen. Jede Messreihe wird in drei Granularitäten erhoben — Micro-, Makro- und Gesamt-Benchmarking —, sodass sowohl einzelne Entwurfsbestandteile als auch ganze Algorithmen vergleichbar werden. Widersprüchliche Literaturbefunde zur optimalen Knoten-/Cache-Line-Größe (eine gegenüber bis zu sechzehn Cache-Lines) werden so nicht aus inkompatiblen Original-Setups übernommen, sondern achsen-isoliert ceteris paribus nachgemessen.

FF3 (Prüfling-Vergleich). Der konkrete Vergleich von PRT-ART gegen die 30 SOTA-Profile steht nach den ersten Mess-Läufen aus (Abschnitt 8.3). Die Architektur ist vollständig vorbereitet: das PRT-ART-Profil, die drei Pflicht-Messreihen-Templates, der `ExperimentDriver` mit Auto-Pickup der SOTA-Profile und der ABI-konforme Adapter mit allen `std::map`-Operationen und Notify-Hooks. Auch die günstigste lokale Seitendarstellung je Verzweigungspunkt (Punkt- gegenüber Kompaktseite) und ihre plattform-kalibrierten Umschaltregeln sind Teil des PRT-ART-Profils.

FF4 (Compile-Time vs. Laufzeit). Die Trennung ist im Build-System verankert: plattform-spezifische Achsen (ISA, Hardware-Konstanten) werden zur Übersetzungszeit fixiert und erzeugen provenienz-nachweisbare Binaries je Permutation (SHA256-validierte Original-Bodies, `experiment_compiler()`-Kennung), während Layout- und Seiten-Selektion über einen bit-codierten Permutations-Identifizier zur Laufzeit wählbar bleiben, ohne die gemeinsame Compiler- und Flag-Basis zu verletzen.

8.2 Limitierungen

- **Modul-Bodies sind derzeit Stubs:** die Codegenerierungs-Pipeline erzeugt ABI-konforme Skelett-Module je Profil; die eigentliche Such-Algorithmus-Implementierung je Permutation bleibt für eine Folge-Phase offen.
- **Hardware-Validierung ausstehend:** die Konfiguration ist lokal grün, aber der vollständige `ctest`-Lauf und die End-to-End-messung_`driver`-Läufe über die BlockAO-Plattformen (AVX2/AVX-512, ARM NEON/SVE) stehen aus.
- **Ergebnis-Kapitel:** aktuell methodisch; konkrete Messungen folgen.

8.3 Ausblick

Kurzfristig: der End-to-End-Lauf der drei Pflicht-Messreihen auf realer Hardware, die Generierung der Mess-Diagramme und die Vervollständigung der Modul-Bodies je Permutation. Mittel- bis langfristig: ein GPU-Adapter (Grace-Hopper-Cluster, ZIH), die PIM-Allokator-Integration (A16) und die formale Verifikation ausgewählter Permutations-Klassen (StarMalloc-Stil, A13). Die Drei-Repository-Architektur erlaubt es künftigen Studierenden, weitere Prüfling-Algorithmen analog zu PRT-ART einzubringen, ohne die Cache-Engine-Werkzeug-Bibliothek zu modifizieren — ein zentrales Designprinzip für die Skalierbarkeit der Forschungsinfrastruktur. Über die hier behandelten baumartigen Suchstrukturen hinaus ist das Anatomie-Prinzip über höherwertige Abstraktionen auf die weiteren Strukturklassen aus Abschnitt 2.2 erweiterbar — hash-basierte, räumliche sowie mengen- und sequenzorientierte Strukturen —, sodass derselbe Mess- und Permutationsapparat ohne Neubau auf ganz neue Klassen angewendet werden kann; darin liegt ein wesentlicher Hebel für künftigen experimentellen Fortschritt.

Literaturverzeichnis

- [ABKS13] APEL, Sven ; BATORY, Don ; KÄSTNER, Christian ; SAAKE, Gunter: *Feature-Oriented Software Product Lines: Concepts and Implementation*. 1. Springer, 2013
- [Adv23] ADVANCED MICRO DEVICES. *Software Optimization Guide for the AMD Zen Microarchitecture (Family 19h)*. Advanced Micro Devices, Santa Clara, CA, USA. 2023
- [Ale01] ALEXANDRESCU, Andrei: *Modern C++ Design: Generic Programming and Design Patterns Applied*. 1. Addison-Wesley, 2001
- [App24] APPLE INC. *Apple Silicon CPU Optimization Guide*. Apple Inc., Cupertino, CA, USA, Version 4. 2024
- [Arm24] ARM LIMITED. *Arm Architecture Reference Manual for A-profile Architecture (DDI 0487)*. Arm Limited, Cambridge, UK. 2024
- [Aßm03] ASSMANN, Uwe: *Invasive Software Composition*. 1. Springer, 2003
- [AVL62] ADELSON-VELSKY, Georgy ; LANDIS, Evgenii: An Algorithm for the Organization of Information. In: *Doklady Akademii Nauk SSSR* 146 (1962), S. 263–266
- [Bay72] BAYER, Rudolf: Symmetric Binary B-Trees: Data Structure and Maintenance Algorithms. In: *Acta Informatica* 1 (1972), Nr. 4, S. 290–306
- [BBB⁺11] BINKERT, Nathan ; BECKMANN, Bradford ; BLACK, Gabriel ; REINHARDT, Steven K. ; SAIDI, Ali ; BASU, Arkaprava ; HESTNESS, Joel ; HOWER, Derek R. ; KRISHNA, Tushar ; SARDASHTI, Somayeh ; SEN, Rathijit ; SEWELL, Korey ; SHOAIB, Muhammad ; VAISH, Nilay ; HILL, Mark D. ; WOOD, David A.: The gem5 Simulator. In: *ACM SIGARCH Computer Architecture News* 39 (2011), Nr. 2, S. 1–7
- [BCD⁺02] BENDER, Michael A. ; COLE, Richard ; DEMAINE, Erik D. ; FARACH-COLTON, Martin ; ZITO, Jack: Two Simplified Algorithms for Maintaining Order in a List. In: *Algorithms — ESA 2002, 10th Annual European Symposium* Bd. 2461, Springer, 2002, S. 152–164
- [BCDFC02] BENDER, Michael A. ; COLE, Richard ; DEMAINE, Erik D. ; FARACH-COLTON, Martin: Scanning and Traversing: Maintaining Data for Traversals in a Memory Hierarchy. In: *Algorithms — ESA 2002, 10th Annual European Symposium* Bd. 2461, Springer, 2002, S. 139–151
- [BDFC00] BENDER, Michael A. ; DEMAINE, Erik D. ; FARACH-COLTON, Martin: Cache-Oblivious B-Trees. In: *Proceedings of the 41st Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, IEEE, 2000, S. 399–409

- [BDFC02] BENDER, Michael A. ; DEMAINE, Erik D. ; FARACH-COLTON, Martin: Efficient Tree Layout in a Multilevel Memory Hierarchy. In: *Algorithms — ESA 2002, 10th Annual European Symposium* Bd. 2461, Springer, 2002, S. 165–173
- [BDFC05] BENDER, Michael A. ; DEMAINE, Erik D. ; FARACH-COLTON, Martin: Cache-Oblivious B-Trees. In: *SIAM Journal on Computing* 35 (2005), Nr. 2, S. 341–358
- [Ben75] BENTLEY, Jon L.: Multidimensional Binary Search Trees Used for Associative Searching. In: *Communications of the ACM* 18 (1975), Nr. 9, S. 509–517
- [BFTV24] BOFFA, Antonio ; FERRAGINA, Paolo ; TOSONI, Francesco ; VINCIGUERRA, Giorgio: CoCo-trie: Data-aware Compression and Indexing of Strings. In: *Information Systems* 120 (2024), S. 102316
- [BH23] BERTHOLD, Johannes ; HABICH, Dirk: VAMPIR/OTF2 Profiling and Performance Analysis for Database Workloads. In: *Proceedings of the 19th International Workshop on Data Management on New Hardware (DaMoN)*, 2023. – Poster presentation
- [BM72] BAYER, Rudolf ; MCCREIGHT, Edward M.: Organization and Maintenance of Large Ordered Indexes. In: *Acta Informatica* 1 (1972), Nr. 3, S. 173–189
- [BMBW00] BERGER, Emery D. ; MCKINLEY, Kathryn S. ; BLUMOF, Robert D. ; WILSON, Paul R.: Hoard: A Scalable Memory Allocator for Multithreaded Applications. In: *Proceedings of the Ninth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS IX)*, ACM, 2000, S. 117–128
- [BO92] BATORY, Don ; O’MALLEY, Sean: The Design and Implementation of Hierarchical Software Systems with Reusable Components. In: *ACM Transactions on Software Engineering and Methodology* 1 (1992), Nr. 4, S. 355–398
- [Bon94] BONWICK, Jeff: The Slab Allocator: An Object-Caching Kernel Memory Allocator. In: *Proceedings of the USENIX Summer 1994 Technical Conference* USENIX, 1994, S. 87–98
- [Bri59] DE LA BRIANDAIS, René: File Searching Using Variable Length Keys. In: *Western Joint Computer Conference*, 1959, S. 295–298
- [BZP⁺18] BINNA, Robert ; ZANGERLE, Eva ; PICHL, Martin ; SPECHT, Günther ; LEIS, Viktor: HOT: A Height Optimized Trie Index for Main-Memory Database Systems. In: *ACM SIGMOD International Conference on Management of Data*, 2018, S. 521–534
- [CE00] CZARNECKI, Krzysztof ; EISENECKER, Ulrich W.: *Generative Programming: Methods, Tools, and Applications*. 1. Addison-Wesley, 2000
- [CGM01] CHEN, Shimin ; GIBBONS, Phillip B. ; MOWRY, Todd C.: Improving Index Performance through Prefetching. In: *Proceedings of the 2001 ACM SIGMOD International Conference on Management of Data*, ACM, 2001, S. 235–246

- [CGM02] CHEN, Shimin ; GIBBONS, Phillip B. ; MOWRY, Todd C.: Fractal Prefetching B+-Trees. In: *Proceedings of the 2002 ACM SIGMOD International Conference on Management of Data*, ACM, 2002, S. 157–168
- [Com79] COMER, Douglas: The Ubiquitous B-Tree. In: *ACM Computing Surveys* 11 (1979), Nr. 2, S. 121–137
- [CST⁺10] COOPER, Brian F. ; SILBERSTEIN, Adam ; TAM, Erwin ; RAMAKRISHNAN, Raghu ; SEARS, Russell: Benchmarking Cloud Serving Systems with YCSB. In: *1st ACM Symposium on Cloud Computing (SoCC)*, 2010, S. 143–154
- [Dre07] DREPPER, Ulrich: What Every Programmer Should Know About Memory / Red Hat, Inc. 2007. – Forschungsbericht. White paper
- [EB75] VAN EMDE BOAS, Peter: Preserving Order in a Forest in Less Than Logarithmic Time. In: *16th Annual Symposium on Foundations of Computer Science (FOCS)*, 1975, S. 75–84
- [Eva06] EVANS, Jason: A Scalable Concurrent malloc(3) Implementation for FreeBSD. In: *Proceedings of BSDCan 2006* BSDCan, 2006
- [FAK⁺12] FERDMAN, Michael ; ADILEH, Almutaz ; KOCBERBER, Onur ; VOLOS, Stavros ; ALISAFEE, Mohammad ; JEVDJIC, Djordje ; KAYNAK, Cansu ; POPESCU, Adrian D. ; AILAMAKI, Anastasia ; FALSAFI, Babak: Clearing the Clouds: A Study of Emerging Scale-out Workloads on Modern Hardware. In: *17th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2012, S. 37–48
- [FB74] FINKEL, Raphael A. ; BENTLEY, Jon L.: Quad Trees: A Data Structure for Retrieval on Composite Keys. In: *Acta Informatica* 4 (1974), Nr. 1, S. 1–9
- [FJKN20] FENT, Philipp ; JUNGMAIR, Michael ; KIPF, Andreas ; NEUMANN, Thomas: START: Self-Tuning Adaptive Radix Tree. In: *36th IEEE International Conference on Data Engineering Workshops (ICDEW)*, 2020, S. 147–153
- [FLPR99] FRIGO, Matteo ; LEISERSON, Charles E. ; PROKOP, Harald ; RAMACHANDRAN, Sridhar: Cache-Oblivious Algorithms. In: *40th Annual Symposium on Foundations of Computer Science (FOCS)*, 1999, S. 285–297
- [Fre60] FREDKIN, Edward: Trie Memory. In: *Communications of the ACM* 3 (1960), Nr. 9, S. 490–499
- [FW93] FREDMAN, Michael L. ; WILLARD, Dan E.: Surpassing the Information Theoretic Bound with Fusion Trees. In: *Journal of Computer and System Sciences* 47 (1993), Nr. 3, S. 424–436
- [GHJV94] GAMMA, Erich ; HELM, Richard ; JOHNSON, Ralph ; VLISSIDES, John: *Design Patterns: Elements of Reusable Object-Oriented Software*. 1. Addison-Wesley, 1994

- [GL01] GRAEFE, Goetz ; LARSON, Per-Åke: B-Tree Indexes and CPU Caches. In: *Proceedings of the 17th IEEE International Conference on Data Engineering (ICDE)*, IEEE, 2001, S. 349–358
- [GM05] GHEMAWAT, Sanjay ; MENAGE, Paul: TCMalloc: Thread-Caching Malloc / Google. 2005. – Forschungsbericht. Design documentation
- [Goo18] GOOGLE. *Abseil Swiss Tables (flat_hash_map)*. <https://abseil.io/about/design/swisstable>. 2018
- [GS78] GUIBAS, Leo J. ; SEDGEWICK, Robert: A Dichromatic Framework for Balanced Trees. In: *19th Annual Symposium on Foundations of Computer Science (FOCS)*, 1978, S. 8–21
- [Gut84] GUTTMAN, Antonin: R-Trees: A Dynamic Index Structure for Spatial Searching. In: *ACM SIGMOD International Conference on Management of Data*, 1984, S. 47–57
- [HB15] HOEFLER, Torsten ; BELLI, Roberto: Scientific Benchmarking of Parallel Computing Systems: Twelve Ways to Tell the Masses When Reporting Performance Results. In: *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2015
- [HIKY12] HALIM, Felix ; IDREOS, Stratos ; KARRAS, Panagiotis ; YAP, Roland H. C.: Stochastic Database Cracking: Towards Robust Adaptive Indexing in Main-Memory Column-Stores. In: *Proceedings of the VLDB Endowment* 5 (2012), Nr. 6, S. 502–513
- [HP03] HANKINS, Richard A. ; PATEL, Jignesh M.: Effect of Node Size on the Performance of Cache-Conscious B+-Trees. In: *Proceedings of the 2003 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems*, ACM, 2003, S. 120–131
- [HP19] HENNESSY, John L. ; PATTERSON, David A.: *Computer Architecture: A Quantitative Approach*. 6th. Morgan Kaufmann, 2019
- [IKM07] IDREOS, Stratos ; KERSTEN, Martin L. ; MANEGOLD, Stefan: Database Cracking. In: *3rd Biennial Conference on Innovative Data Systems Research (CIDR)*, 2007, S. 68–78
- [Int24a] INTEL CORPORATION. *Intel 64 and IA-32 Architectures Optimization Reference Manual*. Intel Corporation, Santa Clara, CA, USA. 2024
- [Int24b] INTEL CORPORATION. *Intel 64 and IA-32 Architectures Software Developer's Manual, Volume 3A: System Programming Guide*. Intel Corporation, Santa Clara, CA, USA. 2024
- [ISO24] ISO/IEC. *ISO/IEC 14882:2024 — Programming Languages — C++*. International Organization for Standardization, Geneva, Switzerland. 2024
- [IZA⁺18] IDREOS, Stratos ; ZOUMPATIANOS, Kostas ; ATHANASSOULIS, Manos ; DAYAN, Niv ; HENTSCHEL, Brian ; KESTER, Michael S. ; GUO, Demi ; MAAS, Lukas M. ; QIN, Wilson ; WASAY, Abdul ; SUN, Yiyu: The Periodic Table of Data Structures. In: *IEEE Data Engineering Bulletin* 41 (2018), Nr. 3, S. 64–75

- [IZH⁺18] IDREOS, Stratos ; ZOUMPATIANOS, Kostas ; HENTSCHEL, Brian ; KESTER, Michael S. ; GUO, Demi: The Data Calculator: Data Structure Design and Cost Synthesis from First Principles, and Learned Cost Models. In: *ACM SIGMOD International Conference on Management of Data*, 2018, S. 535–550
- [Jac89] JACOBSON, Guy J.: Space-Efficient Static Trees and Graphs. In: *Proceedings of the 30th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, IEEE, 1989, S. 549–554
- [JTJE10] JALEEL, Aamer ; THEOBALD, Kevin B. ; JR., Simon C. S. ; EMER, Joel: High Performance Cache Replacement Using Re-Reference Interval Prediction (RRIP). In: *37th Annual International Symposium on Computer Architecture (ISCA)*, 2010, S. 60–71
- [KBC⁺18] KRASKA, Tim ; BEUTEL, Alex ; CHI, Ed H. ; DEAN, Jeffrey ; POLYZOTIS, Neoklis: The Case for Learned Index Structures. In: *ACM SIGMOD International Conference on Management of Data*, 2018, S. 489–504
- [Kha11] KHAN, Minhaj A.: Data Cache Prefetching With Dynamic Adaptation. In: *The Computer Journal* 54 (2011), Nr. 5, S. 815–823
- [KMR⁺19] KIPF, Andreas ; MARCUS, Ryan ; VAN RENEN, Alexander ; STOIAN, Mihail ; KEMPER, Alfons ; KRASKA, Tim ; NEUMANN, Thomas: SOSD: A Benchmark for Learned Indexes. In: *NeurIPS Workshop on Machine Learning for Systems*, 2019
- [Knu98] KNUTH, Donald E.: *The Art of Computer Programming, Volume 3: Sorting and Searching*. 2nd. Addison-Wesley, 1998
- [Kue23] KUEHN, Lena: Towards Data-Based Cache Optimization of B+-Trees. In: *Proceedings of the 19th International Workshop on Data Management on New Hardware (DaMoN)*, ACM, 2023, S. 8:1–8:8
- [LABN16] LEIS, Viktor ; ANDERSEN, Michael ; BIRKEDAL, Erik ; NEUMANN, Thomas: The ART of Practical Synchronization. In: *Proceedings of the International Workshop on Data Management on New Hardware (DaMoN)*, ACM, 2016, S. 8:1–8:8
- [LC86] LEHMAN, Tobin J. ; CAREY, Michael J.: A Study of Index Structures for Main Memory Database Management Systems. In: *12th International Conference on Very Large Data Bases (VLDB)*, 1986, S. 294–303
- [Lc21] LEIJEN, Daan ; CONTRIBUTORS. *mimalloc-bench: Suite for Benchmarking Malloc Implementations*. <https://github.com/daanx/mimalloc-bench>. 2021
- [LDPB19] LIETAR, Paul ; DREYER, Derek ; PARKINSON, Adam ; BORNAT, Richard: snmalloc: A Message Passing Allocator. In: *Proceedings of the 2019 ACM SIGPLAN International Symposium on Memory Management (ISMM)*, ACM, 2019, S. 122–135
- [LKN13] LEIS, Viktor ; KEMPER, Alfons ; NEUMANN, Thomas: The Adaptive Radix Tree: ARTful Indexing for Main-Memory Databases. In: *29th IEEE International Conference on Data Engineering (ICDE)*, 2013, S. 38–49

- [LZM19] LEIJEN, Daan ; ZORN, Benjamin ; DE MOURA, Leonardo: Mimalloc: Free List Sharding in Action. In: *Proceedings of the 17th Asian Symposium on Programming Languages and Systems (APLAS)*, Springer, 2019, S. 172–192
- [MBL25] MÜLLER, Marcus ; BENSON, Lawrence ; LEIS, Viktor: B-Trees Are Back: Engineering Fast and Pageable Node Layouts. In: *Proceedings of the ACM on Management of Data* 3 (2025), Nr. 1
- [McK01] MCKENNEY, Paul E.: Read-Copy Update. In: *Proceedings of the 2001 Ottawa Linux Symposium (OLS)*, 2001, S. 338–367
- [MF02] MANZINI, Giovanni ; FERRAGINA, Paolo: Engineering a Lightweight Suffix Array Construction Algorithm. In: *Algorithms — ESA 2002, 10th Annual European Symposium Bd.* 2461, Springer, 2002, S. 698–710
- [Mic04] MICHAEL, Maged M.: Hazard Pointers: Safe Memory Reclamation for Lock-Free Objects. In: *IEEE Transactions on Parallel and Distributed Systems* 15 (2004), Nr. 6, S. 491–504
- [MKM12] MAO, Yandong ; KOHLER, Eddie ; MORRIS, Robert T.: Cache Craftiness for Fast Multicore Key-Value Storage. In: *7th ACM European Conference on Computer Systems (EuroSys)*, 2012, S. 183–196
- [Mor68] MORRISON, Donald R.: PATRICIA — Practical Algorithm To Retrieve Information Coded in Alphanumeric. In: *Journal of the ACM* 15 (1968), Nr. 4, S. 514–534
- [MWR25] MAHLING, Fabian ; WEISGUT, Marcel ; RABL, Tilmann: Fetch Me If You Can: Evaluating CPU Cache Prefetching and Its Reliability on High Latency Memory. In: *Proceedings of the 21st International Workshop on Data Management on New Hardware (DaMoN)*, ACM, 2025
- [NTSA16] NADERAN-TAHAN, Mahmood ; SARBAZI-AZAD, Hamid: Why Does Data Prefetching Not Work for Modern Workloads? In: *The Computer Journal* 59 (2016), Nr. 2, S. 244–259
- [PR04] PAGH, Rasmus ; RODLER, Flemming F.: Cuckoo Hashing. In: *Journal of Algorithms* 51 (2004), Nr. 2, S. 122–144
- [QJP⁺07] QURESHI, Moinuddin K. ; JALEEL, Aamer ; PATT, Yale N. ; JR., Simon C. S. ; EMER, Joel: Adaptive Insertion Policies for High Performance Caching. In: *34th Annual International Symposium on Computer Architecture (ISCA)*, 2007, S. 381–391
- [RFP⁺24] REITZ, Chris ; FROMHERZ, Aymeric ; PROTZENKO, Jonathan ; CHEN, Boniface ; FIEDLER, Talia ; BICHHAWAT, Ananya ; ALHANAHNAH, Dalia ; OU, Elaine ; BHARGAVAN, Karthikeya: StarMalloc: Verifying a Modern, Hardened Memory Allocator. In: *Proceedings of the 2024 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, ACM, 2024. – Also available as arXiv:2403.09435
- [RR99] RAO, Jun ; ROSS, Kenneth A.: Cache Conscious Indexing for Decision-Support in Main Memory. In: *Proceedings of the 25th International Conference on Very Large Data Bases (VLDB)*, 1999, S. 78–89

- [RR00] RAO, Jun ; ROSS, Kenneth A.: Making B+-Trees Cache Conscious in Main Memory. In: *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, ACM, 2000, S. 475–486
- [SKK⁺25] SCHMIDT, Lennart ; KÜHN, Roland ; KRAUSE, Matti ; TEUBNER, Jens ; LEHNER, Wolfgang ; HABICH, Dirk: To Stride or Not to Stride the Memory Access? In: *Proceedings of the 3rd Workshop on Disruptive Memory Systems (DIMES)*, ACM, 2025, S. 19–26
- [SPB05] SAMUEL, Curt ; PEDERSEN, Torben B. ; BONNET, Philippe: Processor-Conscious Indexing. In: *Proceedings of the 2005 ACM SIGMOD International Conference on Management of Data*, ACM, 2005, S. 235–246
- [SSL⁺21] SCHMEISSER, Josef ; SCHÜLE, Maximilian E. ; LEIS, Viktor ; NEUMANN, Thomas ; KEMPER, Alfons: The B²-Tree: Cache-Friendly String Indexing within B-Trees. In: *Datenbanksysteme für Business, Technologie und Web (BTW)*, 2021, S. 39–58
- [SSS08] SAIKKONEN, Riku ; SOISALON-SOININEN, Eljas: Cache-Sensitive Memory Layout for Binary Trees. In: *Fifth IFIP International Conference on Theoretical Computer Science (TCS 2008)* Bd. 273, Springer, 2008
- [SSS16] SAIKKONEN, Riku ; SOISALON-SOININEN, Eljas: Cache-Sensitive Memory Layout for Dynamic Binary Trees. In: *The Computer Journal* 59 (2016), Nr. 5, S. 630–649
- [Sta17] STANDARD PERFORMANCE EVALUATION CORPORATION. *SPEC CPU 2017 Benchmark Suite*. <https://www.spec.org/cpu2017/>. 2017
- [Tra24] TRANSACTION PROCESSING PERFORMANCE COUNCIL. *TPC Benchmark C and TPC-DS Specifications*. Transaction Processing Performance Council, San Francisco, CA, USA. 2024
- [UHL⁺17] UNGETHUEM, Sigmund ; HABICH, Dirk ; LEHNER, Wolfgang [u. a.]: Hardware Acceleration for Database Systems: A Comprehensive Survey. In: *ACM Computing Surveys* 50 (2017), Nr. 1, S. 3:1–3:35
- [VIA26] VIA RESEARCH: PIM-malloc: A Fast and Scalable Dynamic Memory Allocator for Processing-In-Memory Systems. In: *Proceedings of the IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, IEEE, 2026. – Accepted to HPCA 2026; also available as arXiv:2505.13002 (2025)
- [Wil83] WILLARD, Dan E.: Log-Logarithmic Worst-Case Range Queries are Possible in Space $\Theta(N)$. In: *Information Processing Letters* 17 (1983), Nr. 2, S. 81–84
- [WNJ19] WU, Xingbo ; NI, Fan ; JIANG, Song: Wormhole: A Fast Ordered Index for In-Memory Data Management. In: *14th ACM European Conference on Computer Systems (EuroSys)*, 2019
- [YZL23] YANG, Zhaoxin ; ZHAO, Hui ; LIU, Yunxin: NUMAlloc: A Faster NUMA Memory Allocator. In: *Proceedings of the 2023 ACM SIGPLAN International Symposium on Memory Management (ISMM)*, ACM, 2023, S. 134–147

- [ZGH⁺25] ZHANG, Tingji ; GROT, Boris ; HE, Wenjian ; LV, Yashuai ; QU, Peng ; SU, Fang ; WANG, Wenxin ; ZHANG, Guowei ; ZHANG, Xuefeng ; ZHANG, Youhui: Hierarchical Prefetching: A Software-Hardware Instruction Prefetcher for Server Applications. In: *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), Volume 2*, ACM, 2025, S. 529–544
- [ZLL⁺18] ZHANG, Huanchen ; LIM, Hyeontaek ; LEIS, Viktor ; ANDERSEN, David G. ; KAMINSKY, Michael ; KEETON, Kimberly ; PAVLO, Andrew: SuRF: Practical Range Query Filtering with Fast Succinct Tries. In: *ACM SIGMOD International Conference on Management of Data*, 2018, S. 323–336
- [ZSZ⁺24] ZHANG, Qian ; SONG, Haoyun ; ZHOU, Kaiyan ; WEI, Jianhao ; XIAO, Chuqiao: A Prefetching Indexing Scheme for In-Memory Database Systems. In: *Future Generation Computer Systems* 156 (2024), S. 179–190

Abbildungsverzeichnis

A.1	Zyklen je Lebewesen-Permutation (v5_pipeline_demo)	58
A.2	Zyklen je Lebewesen-Permutation (cartesian_smoke43)	60
A.3	Heatmap: Gesamt-Latenz (ns/op) (Suchalgorithmus x Workload)	61
A.4	Heatmap: Insert-Latenz p50 (ns/op) (Suchalgorithmus x Workload)	62
A.5	Heatmap: Lookup-Latenz p50 (ns/op) (Suchalgorithmus x Workload)	63
A.6	Heatmap: Erase-Latenz p50 (ns/op) (Suchalgorithmus x Workload)	64
A.7	Heatmap: Scan-Latenz p50 (ns/op) (Suchalgorithmus x Workload)	65
A.8	Heatmap: RMW-Latenz p50 (ns/op) (Suchalgorithmus x Workload)	66

Tabellenverzeichnis

3.1	Achsen-Sezierung (Teil 1: Kompositions-Achsen T0–T9)	20
3.2	Achsen-Sezierung (Teil 2: T10–T18 + Build-Achsen)	20
3.3	Hardware- und Scheduling-Strategie pro SOTA-Paper (Auswahl); der Vollkatalog steht in Anhang D.	22
3.4	Benchmark-Frameworks und nutzende Veröffentlichungen	23
3.5	Lastprofil-Katalog (14 Profile, aus der 33-Paper-Analyse abgeleitet)	24
3.6	SOTA-Algorithmus-Profile mit Workload-Affinität	25
3.7	Allokator-Profile mit Lizenz und Workload-Affinität	25
6.1	Profil-bewusstes Workload-Routing	38
6.2	Reale Datensätze und ihre Charakteristik	38
A.1	Messreihe v5_pipeline_demo: vorkonfigurierte Lebewesen-Permutationen (Pfad A, run_workload)	57
A.2	Messreihe cartesian_smoke43: vorkonfigurierte Lebewesen-Permutationen (Pfad A, run_workload)	59
A.3	Bias-Bruch-Matrix (Median ns/op, Suchverfahren x Lastprofil)	59
A.4	Achsen-Austauschbarkeit: search_algo (Geschwister-Paar-Diffs, Median rel. Δ ns/op bzgl. v ; 480 Geschwister-Paare; Konfundierung: am wenigsten konfundiert)	67
A.5	Achsen-Austauschbarkeit: node_type (Geschwister-Paar-Diffs, Median rel. Δ ns/op bzgl. v ; 480 Geschwister-Paare; Konfundierung: Vorbehalt)	68
A.6	Achsen-Austauschbarkeit: memory_layout (Geschwister-Paar-Diffs, Median rel. Δ ns/op bzgl. v ; 640 Geschwister-Paare; Konfundierung: Vorbehalt)	70
A.7	Achsen-Austauschbarkeit: prefetch (Geschwister-Paar-Diffs, Median rel. Δ ns/op bzgl. v ; 480 Geschwister-Paare; Konfundierung: am wenigsten konfundiert)	75
A.8	Ehrliche Limitierungen: nicht-gefixte Vorbehalte (kein Befund verfällt still)	76
F.1	Vollständige <code>std::map</code> -Schnittstelle (C++23)	115
F.2	Vollständige <code>std::vector</code> -Schnittstelle (C++23)	118
F.3	Zusammengesetzte Funktionen („unter der Haube“)	120

A Vollständige Messreihen

Dieser Anhang enthält die vollständigen Messreihen der vorkonfigurierten Lebewesen-Permutationen. Tabelle A.1 listet die je Permutation gemessenen Kennzahlen (Operationen, Taktzyklen, Cache-Misses L1–L3); Abbildung A.1 stellt die Taktzyklen grafisch gegenüber.

Permutation	Ops	Zyklen	L1	L2	L3
ArtComposition_0	2000	825053	0	0	0
HotComposition_1	2000	740176	0	0	0
StartComposition_2	2000	805750	0	0	0
SurfComposition_3	2000	873976	0	0	0
WormholeComposition_4	2000	696350	0	0	0
MasstreeComposition_5	2000	770871	0	0	0

Tabelle A.1: Messreihe v5_pipeline_demo: vorkonfigurierte Lebewesen-Permutationen (Pfad A, run_workload)

A.1 Kartesische Smoke-Messreihe (43 Permutationen, echte DLL-Messung)

Die folgende Messreihe entstand aus dem vollständigen Ketten-Durchlauf: die gebauten Permutations-DLLs (43 kartesische SIMD×Layout×Allokator-Kombinationen) wurden am Prüf-Dock gemessen, als binäre Mess-Records persistiert, via Pipeline-Stufe 03 in das 16-Spalten-Schema konvertiert und automatisch hier eingebunden. Tabelle A.2 und Abbildung A.2 zeigen die Ergebnisse.

A.2 Bias-Bruch-Matrix

Tabelle A.3 fasst die je Suchverfahren und Lastprofil gemessene Median-Latenz (ns/op, nearest-rank) über alle dynamischen Einstellungen und Wiederholungen zusammen; berücksichtigt sind ausschließlich die als `two_phase_valid` markierten Läufe. Die Matrix macht sichtbar, bei welchen Lastprofilen das jeweilige Verfahren seinen systematischen Bias bricht.

A.3 Latenz-Surfaces je Schnittstellen-Funktion

Die folgenden sechs Heatmaps stellen die Latenz-Oberfläche (Suchalgorithmus × Lastprofil) je Schnittstellen-Funktion dar: die Gesamt-Latenz (`ns_per_op`) sowie die p50-Latenzen der Einzeloperationen `insert`, `lookup`, `erase`, `scan` und `rmw`.

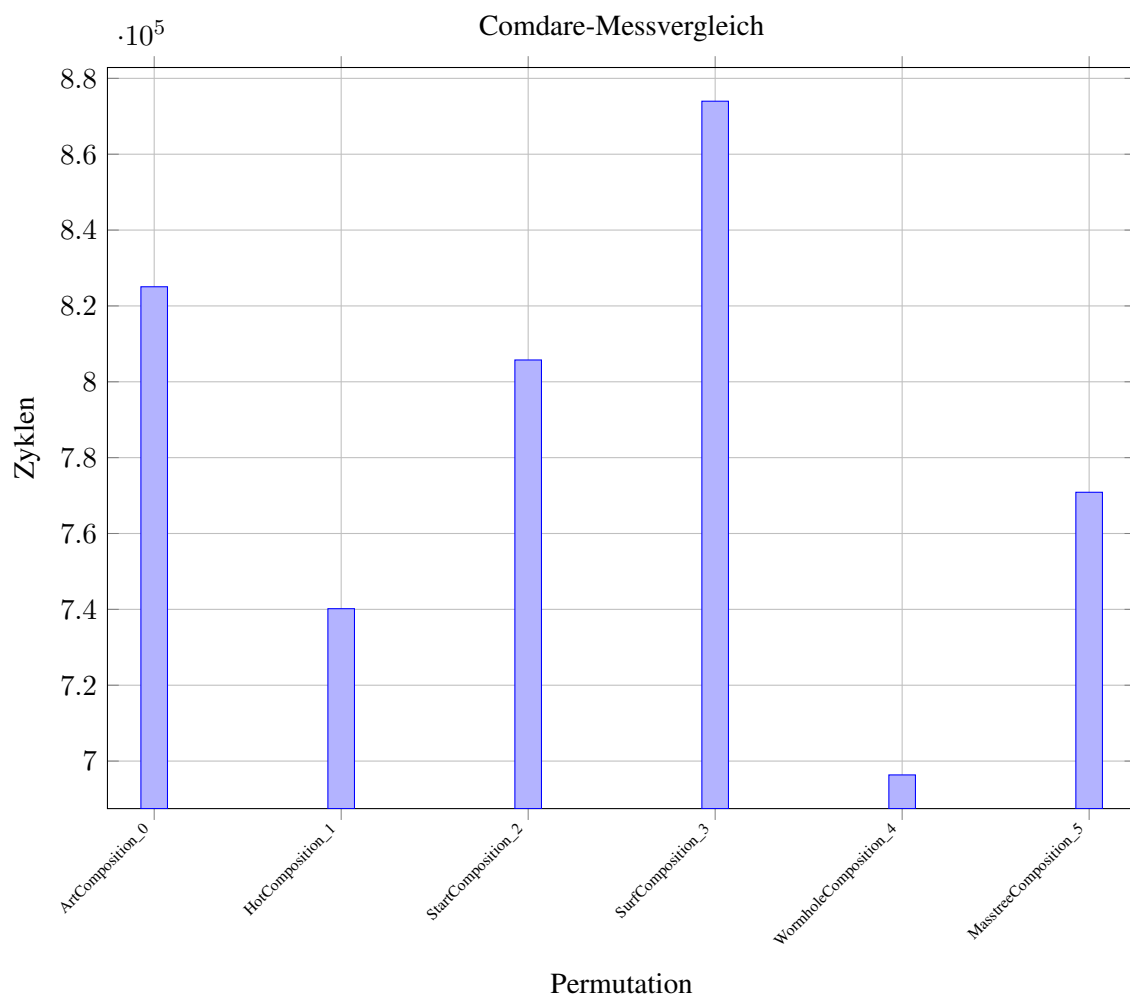


Abbildung A.1: Zyklen je Lebewesen-Permutation (v5_pipeline_demo)

Permutation	Ops	Zyklen	L1	L2	L3
avx2_aos_jemalloc	1000	8025	0	0	0
avx2_aos_mimalloc	1000	7995	0	0	0
avx2_aos_std	1000	7995	0	0	0
avx2_hybrid_jemalloc	1000	315164	0	0	0
avx2_hybrid_mimalloc	1000	243930	0	0	0
avx2_hybrid_std	1000	230759	0	0	0
avx2_soa_jemalloc	1000	423930	0	0	0
avx2_soa_mimalloc	1000	400634	0	0	0
avx2_soa_std	1000	398265	0	0	0
scalar_aos_jemalloc	1000	6975	0	0	0
scalar_aos_mimalloc	1000	7005	0	0	0
scalar_aos_std	1000	6975	0	0	0
scalar_hybrid_jemalloc	1000	1366319	0	0	0
scalar_hybrid_mimalloc	1000	1279365	0	0	0
scalar_hybrid_std	1000	1356329	0	0	0
scalar_soa_jemalloc	1000	440835	0	0	0
scalar_soa_mimalloc	1000	394200	0	0	0
scalar_soa_std	1000	438960	0	0	0
sse4_aos_jemalloc	1000	16845	0	0	0
sse4_aos_mimalloc	1000	14160	0	0	0
sse4_aos_std	1000	14324	0	0	0
sse4_hybrid_jemalloc	1000	1281180	0	0	0
sse4_hybrid_mimalloc	1000	1275404	0	0	0
sse4_hybrid_std	1000	1279350	0	0	0
sse4_soa_jemalloc	1000	397095	0	0	0
sse4_soa_mimalloc	1000	392294	0	0	0
sse4_soa_std	1000	396780	0	0	0
pa_compact_lazy_binary_leaf_only	1000	348105	0	0	0
pa_compact_lazy_binary_sampling	1000	375600	0	0	0
pa_compact_lazy_linear_leaf_only	1000	445815	0	0	0
pa_compact_lazy_linear_sampling	1000	423090	0	0	0
pa_compact_none_binary_leaf_only	1000	360765	0	0	0
pa_compact_none_binary_sampling	1000	346980	0	0	0
pa_compact_none_linear_leaf_only	1000	398939	0	0	0
pa_compact_none_linear_sampling	1000	391875	0	0	0
pa_wide_lazy_binary_leaf_only	1000	309300	0	0	0
pa_wide_lazy_binary_sampling	1000	311805	0	0	0
pa_wide_lazy_linear_leaf_only	1000	445605	0	0	0
pa_wide_lazy_linear_sampling	1000	492495	0	0	0
pa_wide_none_binary_leaf_only	1000	307110	0	0	0
pa_wide_none_binary_sampling	1000	311925	0	0	0
pa_wide_none_linear_leaf_only	1000	445140	0	0	0
pa_wide_none_linear_sampling	1000	478635	0	0	0

Tabelle A.2: Messreihe cartesian_smoke43: vorkonfigurierte Lebewesen-Permutationen (Pfad A, run_workload)

Suchverfahren	csso-p04neg0	csso-p04neg100	csso-p04neg25	csso-p04neg50	csso-p04neg75	ib	ib	ib_addressed_500	ib_bulk_insert	ip_casement_jmva	ip_dedup_heavy	ip_dynamic_trace	ip_mixed_oltp	ip_zmqgc_scan	ip_zmqgc_uniform	psdb.a	psdb.b	psdb.c	psdb.d	psdb.e	psdb.f
eytzipper	4663	9367	5755	6924	8129	175317	12536	97590	235805	91687	134523	19093	49297	33	4651	98559	12663	4730	16086	7794	8205
interpolation	3650	7202	4557	5444	6339	153540	11475	86204	207310	88743	115834	17004	42893	91811	3607	85569	11425	3704	15762	99954	6283
k_ary	2817	5473	3467	4113	4824	92827	7134	52027	125172	51927	76934	10594	26662	23	2802	51911	7208	2851	9141	4187	4820
linear_scan	11506	23486	14487	17282	20557	48913	12990	30947	63537	96624	88800	14029	25348	168720	11385	31086	13001	11652	21425	166863	21763

Tabelle A.3: Bias-Bruch-Matrix (Median ns/op, Suchverfahren x Lastprofil)

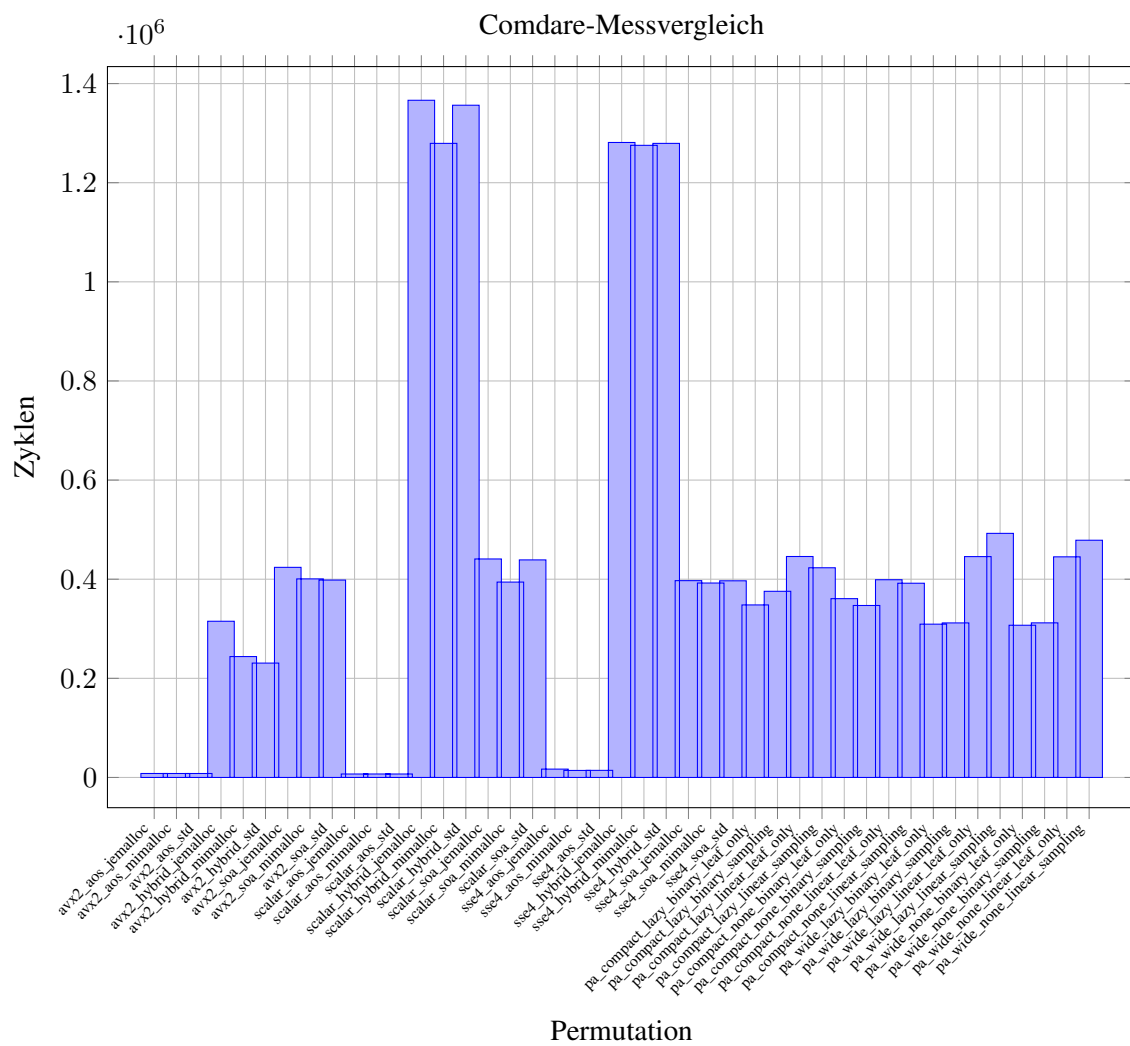


Abbildung A.2: Zyklen je Lebewesen-Permutation (cartesian_smoke43)

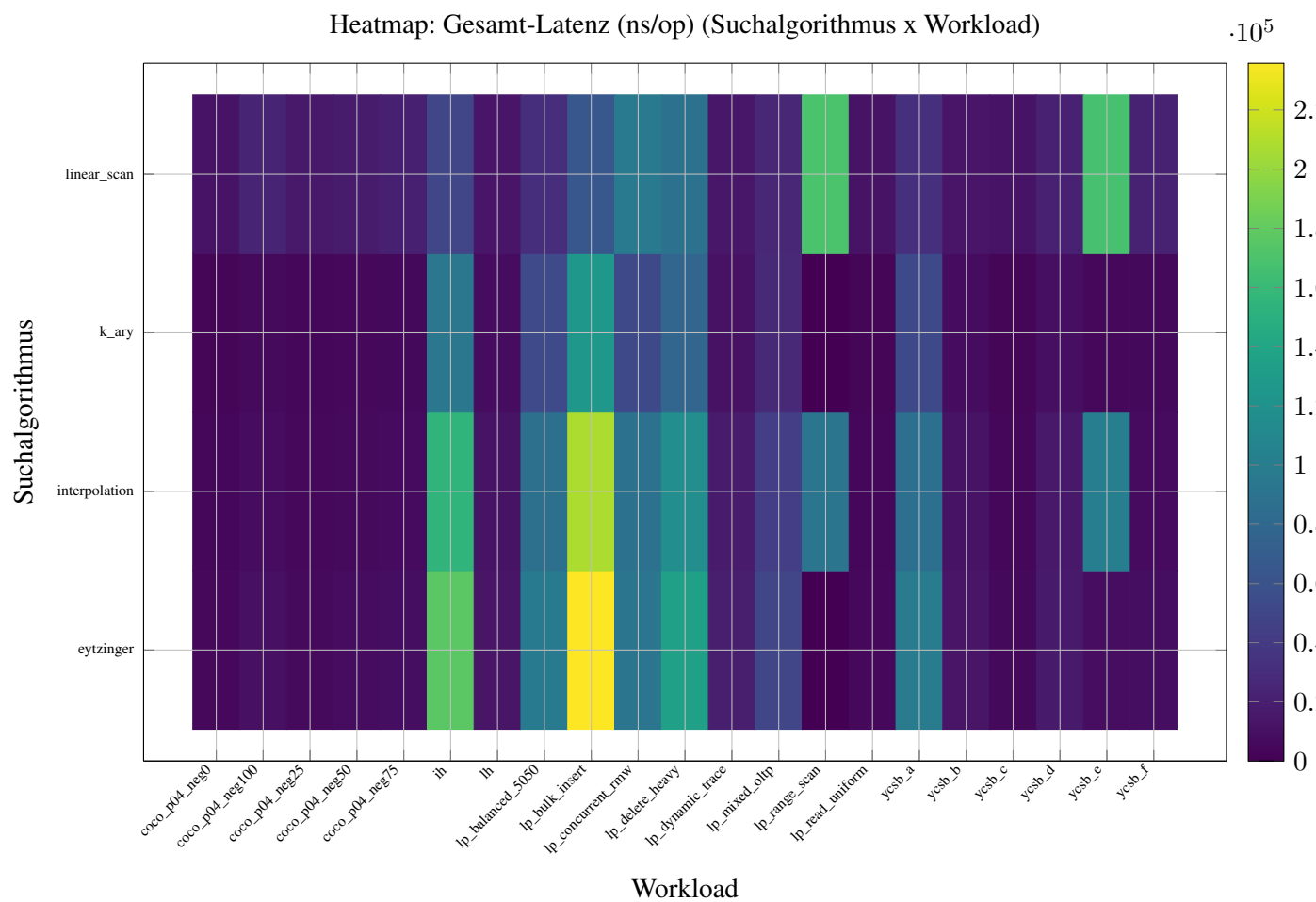


Abbildung A.3: Heatmap: Gesamt-Latenz (ns/op) (Suchalgorithmus x Workload)

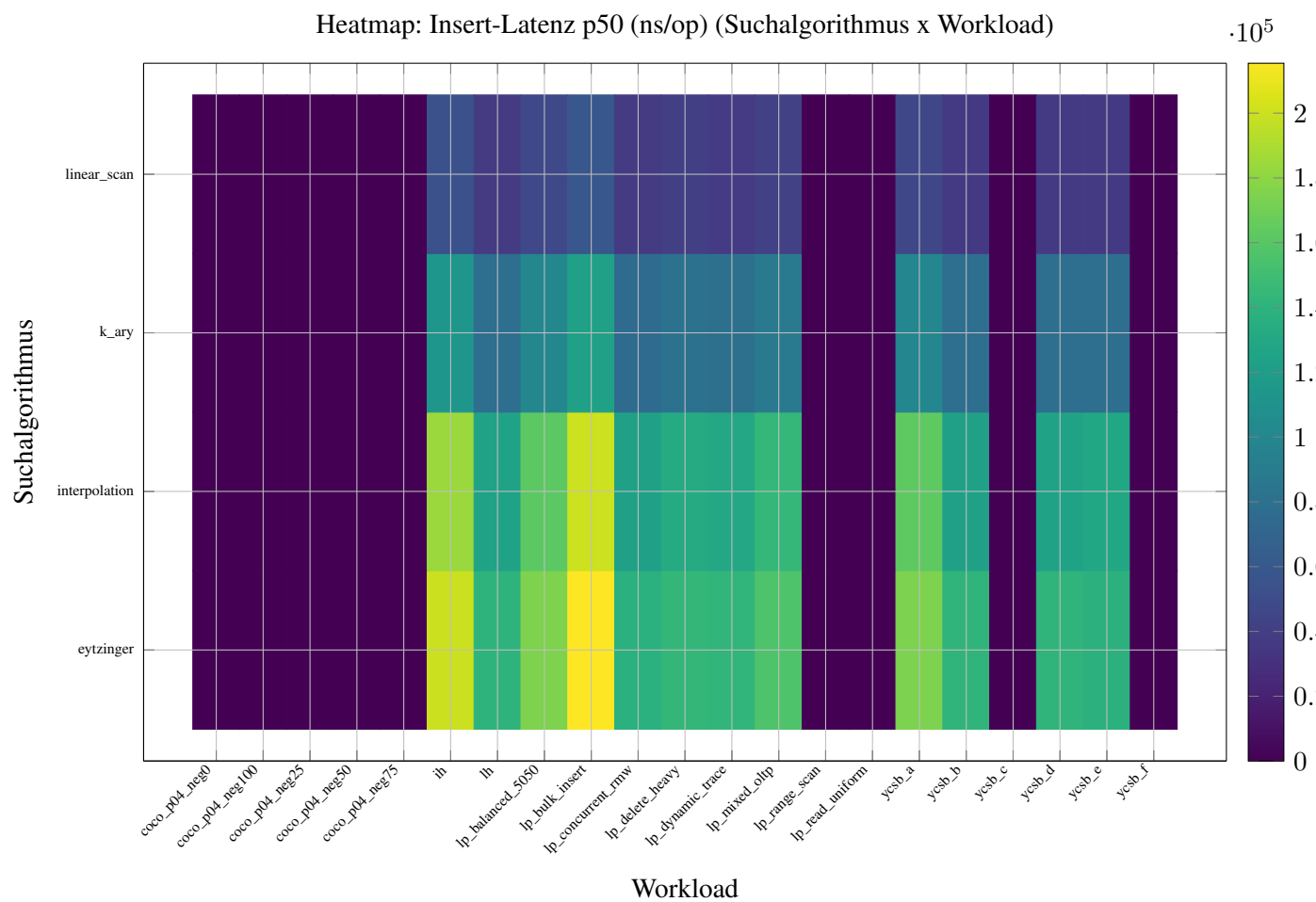


Abbildung A.4: Heatmap: Insert-Latenz p50 (ns/op) (Suchalgorithmus x Workload)

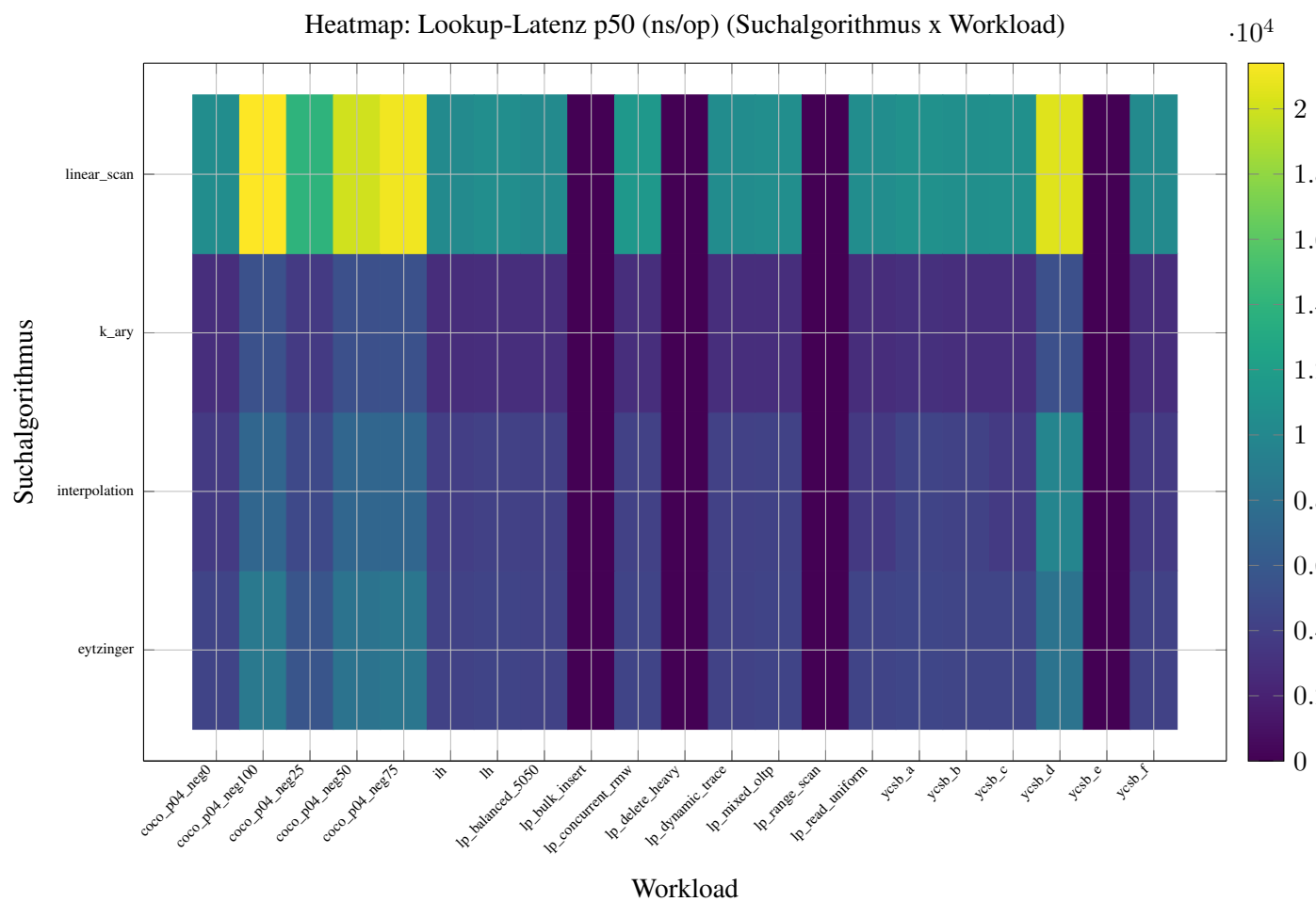


Abbildung A.5: Heatmap: Lookup-Latenz p50 (ns/op) (Suchalgorithmus x Workload)

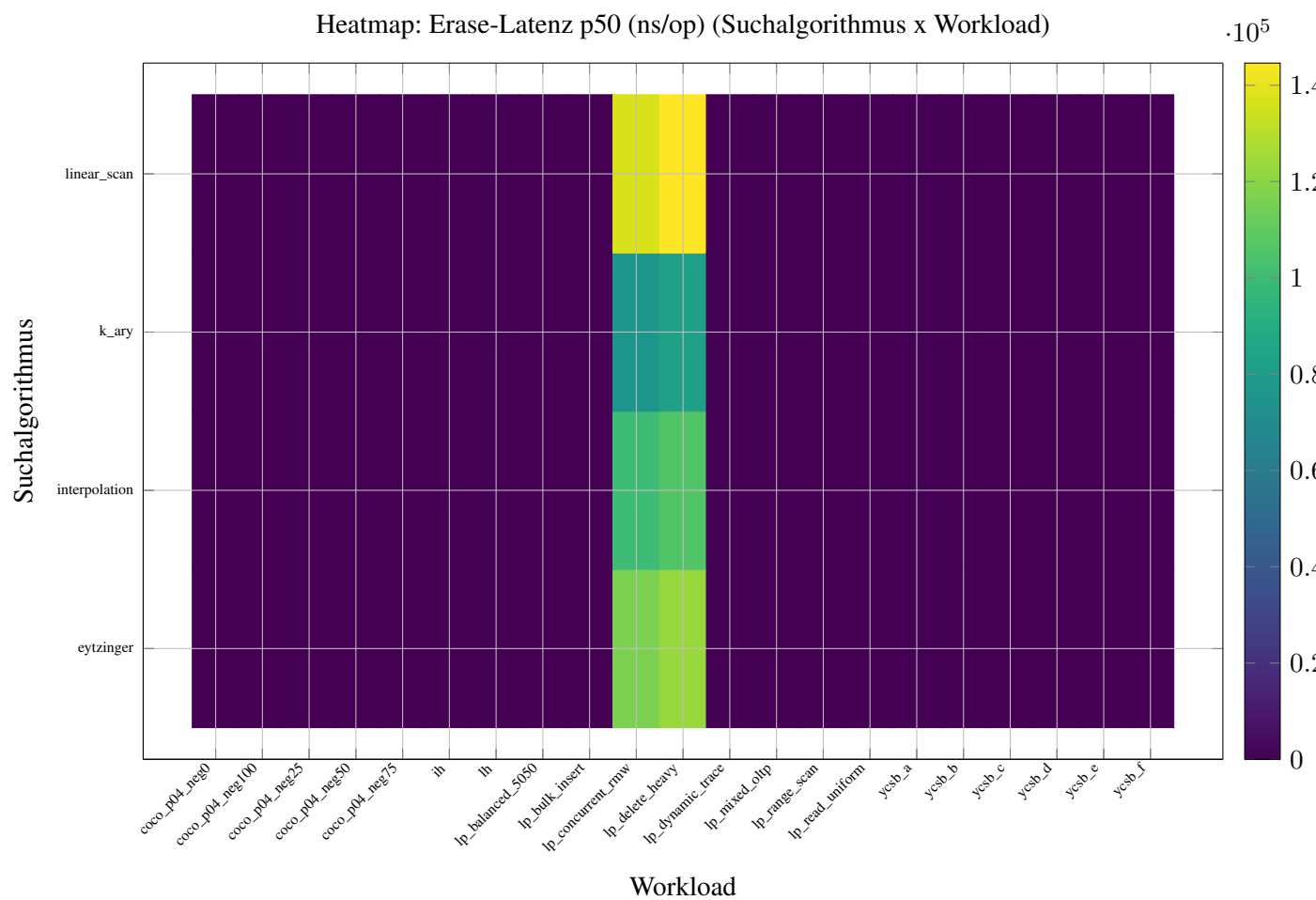


Abbildung A.6: Heatmap: Erase-Latenz p50 (ns/op) (Suchalgorithmus x Workload)

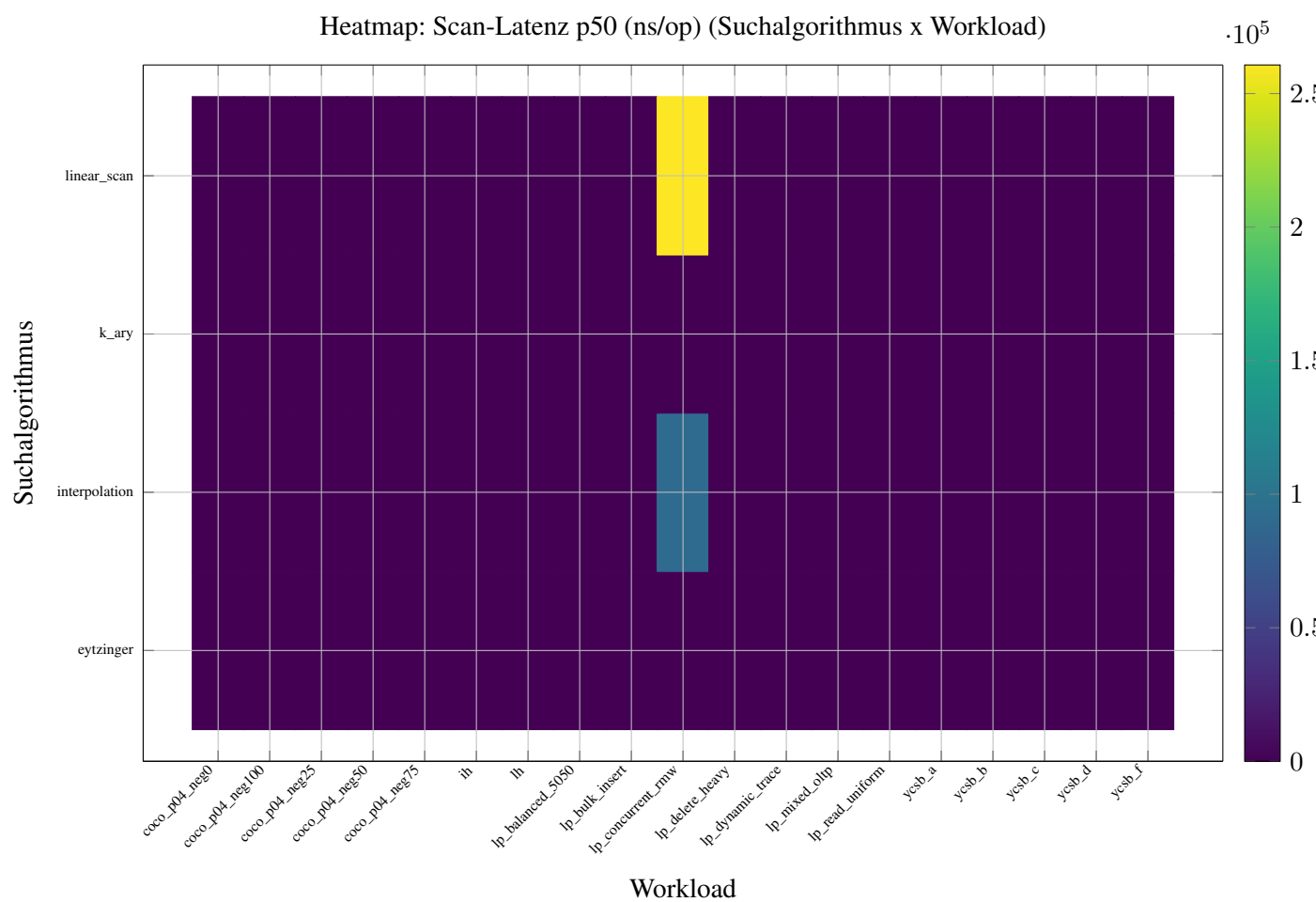


Abbildung A.7: Heatmap: Scan-Latenz p50 (ns/op) (Suchalgorithmus x Workload)

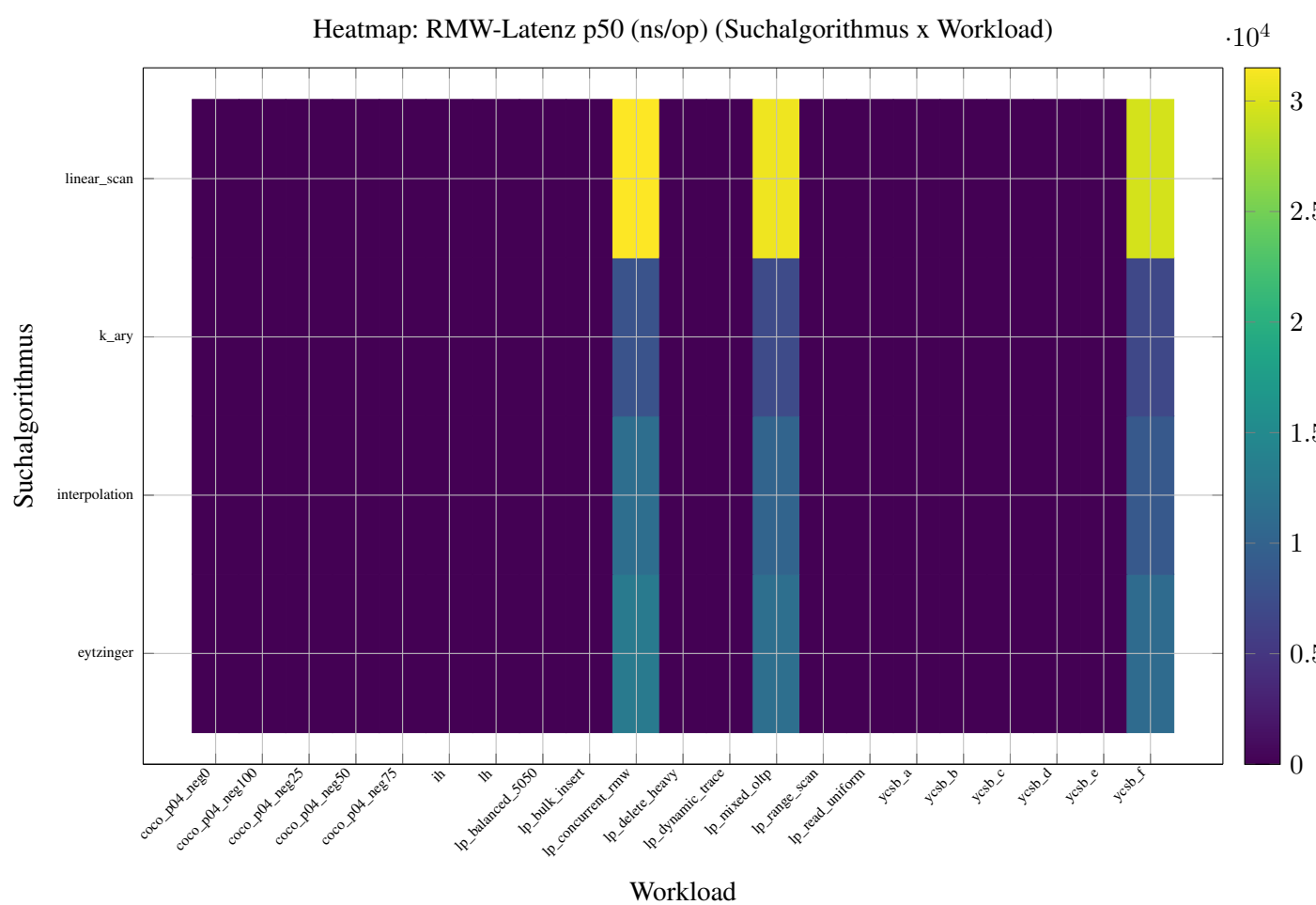


Abbildung A.8: Heatmap: RMW-Latenz p50 (ns/op) (Suchalgorithmus x Workload)

A.4 Achsen-Austauschbarkeit

Die vier folgenden Langtabellen belegen die Austauschbarkeit je variabler Achse anhand der Geschwister-Paar-Differenzen (Median rel. Δ ns/op bezüglich v): `search_algo` (Tabelle A.4), `node_type` (Tabelle A.5), `memory_layout` (Tabelle A.6) und `prefetch` (Tabelle A.7). Am wenigsten konfundiert sind `search_algo` und `prefetch`.

Tabelle A.4: Achsen-Austauschbarkeit: `search_algo` (Geschwister-Paar-Diffs, Median rel. Δ ns/op bzgl. v ; 480 Geschwister-Paare; Konfundierung: am wenigsten konfundiert)

v	v'	Funktion	Median abs. Δ (ns)	Median rel. Δ	IQR rel.	n	Diagnose
eytzinger	interpolation	ns_per_op	-1493	-0.145	0.574	1680	verschiedener Organ-Pfad
eytzinger	interpolation	insert	0	-0.103	0.399	960	verschiedener Organ-Pfad
eytzinger	interpolation	lookup	-200	-0.146	0.436	1360	verschiedener Organ-Pfad
eytzinger	interpolation	erase	0	-0.148	0.344	160	verschiedener Organ-Pfad
eytzinger	interpolation	scan	0	+1177.000	309.000	11	verschiedener Organ-Pfad
eytzinger	interpolation	rmw	0	-0.171	0.403	240	verschiedener Organ-Pfad
eytzinger	k_ary	ns_per_op	-6145	-0.420	0.145	1680	verschiedener Organ-Pfad
eytzinger	k_ary	insert	-40500	-0.428	0.166	960	verschiedener Organ-Pfad
eytzinger	k_ary	lookup	-1500	-0.357	0.125	1360	verschiedener Organ-Pfad
eytzinger	k_ary	erase	0	-0.357	0.218	160	verschiedener Organ-Pfad
eytzinger	k_ary	scan	0	-1.000	0.000	11	verschiedener Organ-Pfad
eytzinger	k_ary	rmw	0	-0.374	0.167	240	verschiedener Organ-Pfad
eytzinger	linear_scan	ns_per_op	2587	+0.189	2.026	1680	verschiedener Organ-Pfad
eytzinger	linear_scan	insert	-77100	-0.755	0.151	960	verschiedener Organ-Pfad
eytzinger	linear_scan	lookup	5600	+1.347	0.777	1360	verschiedener Organ-Pfad
eytzinger	linear_scan	erase	0	+0.146	0.383	160	verschiedener Organ-Pfad
eytzinger	linear_scan	scan	0	+2758.000	107.000	11	verschiedener Organ-Pfad
eytzinger	linear_scan	rmw	0	+1.587	0.842	240	verschiedener Organ-Pfad
interpolation	k_ary	ns_per_op	-6981	-0.340	0.275	1680	verschiedener Organ-Pfad
interpolation	k_ary	insert	-28100	-0.372	0.187	960	verschiedener Organ-Pfad
interpolation	k_ary	lookup	-900	-0.289	0.235	1360	verschiedener Organ-Pfad
interpolation	k_ary	erase	0	-0.243	0.211	160	verschiedener Organ-Pfad
interpolation	k_ary	scan	0	-1.000	0.000	80	verschiedener Organ-Pfad
interpolation	k_ary	rmw	0	-0.278	0.256	240	verschiedener Organ-Pfad
interpolation	linear_scan	ns_per_op	4390	+0.338	2.065	1680	verschiedener Organ-Pfad
interpolation	linear_scan	insert	-55300	-0.735	0.178	960	verschiedener Organ-Pfad
interpolation	linear_scan	lookup	6200	+1.596	1.277	1360	verschiedener Organ-Pfad
interpolation	linear_scan	erase	0	+0.395	0.446	160	verschiedener Organ-Pfad
interpolation	linear_scan	scan	0	+1.777	1.149	80	verschiedener Organ-Pfad
interpolation	linear_scan	rmw	0	+2.000	1.488	240	verschiedener Organ-Pfad
k_ary	linear_scan	ns_per_op	8960	+1.322	3.446	1680	verschiedener Organ-Pfad
k_ary	linear_scan	insert	-18200	-0.586	0.322	960	verschiedener Organ-Pfad
k_ary	linear_scan	lookup	7800	+2.815	1.095	1360	verschiedener Organ-Pfad
k_ary	linear_scan	erase	0	+0.774	0.409	160	verschiedener Organ-Pfad
k_ary	linear_scan	rmw	0	+3.254	1.326	240	verschiedener Organ-Pfad

Tabelle A.5: Achsen-Austauschbarkeit: node_type (Geschwister-Paar-Diffs, Median rel. Δ ns/op bzgl. v ; 480 Geschwister-Paare; Konfundierung: Vorbehalt)

v	v'	Funktion	Median abs. Δ (ns)	Median rel. Δ	IQR rel.	n	Diagnose
node16	node256	ns_per_op	1010	+0.126	0.592	1680	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
node16	node256	insert	0	-0.033	0.441	960	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
node16	node256	lookup	500	+0.207	0.493	1360	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
node16	node256	erase	0	-0.025	0.354	160	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
node16	node256	scan	0	+0.339	0.419	40	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
node16	node256	rmw	0	+0.188	0.473	240	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
node16	node4	ns_per_op	6164	+0.573	0.943	1680	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
node16	node4	insert	12900	+0.993	0.540	960	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
node16	node4	lookup	0	+0.058	0.349	1360	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
node16	node4	erase	0	+1.085	0.749	160	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
node16	node4	scan	0	+0.275	0.677	40	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
node16	node4	rmw	0	+0.127	0.455	240	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
node16	node48	ns_per_op	-160	-0.027	0.325	1680	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich

Fortsetzung nächste Seite

Tabelle A.5 – Fortsetzung

v	v'	Funktion	Median abs. Δ (ns)	Median rel. Δ	IQR rel.	n	Diagnose
node16	node48	insert	0	-0.135	0.407	960	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
node16	node48	lookup	0	+0.037	0.245	1360	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
node16	node48	erase	0	-0.151	0.225	160	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
node16	node48	scan	0	+0.052	0.183	40	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
node16	node48	rmw	0	+0.018	0.257	240	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
node256	node4	ns_per_op	4066	+0.268	0.947	1680	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
node256	node4	insert	6700	+1.065	1.332	960	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
node256	node4	lookup	0	-0.024	0.326	1360	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
node256	node4	erase	0	+1.436	0.754	160	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
node256	node4	scan	0	-0.010	0.447	49	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
node256	node4	rmw	0	+0.042	0.411	240	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
node256	node48	ns_per_op	-1410	-0.132	0.382	1680	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
node256	node48	insert	0	-0.063	0.352	960	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
node256	node48	lookup	-300	-0.140	0.317	1360	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich

Fortsetzung nächste Seite

Tabelle A.5 – Fortsetzung

v	v'	Funktion	Median abs. Δ (ns)	Median rel. Δ	IQR rel.	n	Diagnose
node256	node48	erase	0	-0.100	0.278	160	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
node256	node48	scan	0	-0.299	0.368	49	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
node256	node48	rmw	0	-0.114	0.386	240	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
node4	node48	ns_per_op	-5104	-0.360	0.567	1680	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
node4	node48	insert	0	-0.586	0.262	960	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
node4	node48	lookup	0	-0.012	0.264	1360	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
node4	node48	erase	0	-0.596	0.103	160	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
node4	node48	scan	0	-0.151	0.230	40	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
node4	node48	rmw	0	-0.084	0.277	240	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich

Tabelle A.6: Achsen-Austauschbarkeit: memory_layout (Geschwister-Paar-Diffs, Median rel. Δ ns/op bzgl. v ; 640 Geschwister-Paare; Konfundierung: Vorbehalt)

v	v'	Funktion	Median abs. Δ (ns)	Median rel. Δ	IQR rel.	n	Diagnose
memory_layout_aos_stm	memory_layout_aosoa	ns_per_op	1138	+0.092	0.309	1344	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
memory_layout_aos_stm	memory_layout_aosoa	insert	0	+0.211	0.326	768	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
memory_layout_aos_stm	memory_layout_aosoa	lookup	0	+0.000	0.255	1088	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich

Fortsetzung nächste Seite

Tabelle A.6 – Fortsetzung

v	v'	Funktion	Median abs. Δ (ns)	Median rel. Δ	IQR rel.	n	Diagnose
memory_layout_aos_strict	memory_layout_aosoa	erase	0	+0.209	0.244	128	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4
memory_layout_aos_strict	memory_layout_aosoa	scan	0	-0.045	0.351	35	search_organ_-Beschattung, Apparat-Artefakt moeglich verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4
memory_layout_aos_strict	memory_layout_aosoa	rmw	0	+0.097	0.257	192	search_organ_-Beschattung, Apparat-Artefakt moeglich verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4
memory_layout_aos_strict	memory_layout_cache_line	aligned	1081	+0.077	0.223	1344	search_organ_-Beschattung, Apparat-Artefakt moeglich verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4
memory_layout_aos_strict	memory_layout_cache_line	insertaligned	0	+0.230	0.259	768	search_organ_-Beschattung, Apparat-Artefakt moeglich verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4
memory_layout_aos_strict	memory_layout_cache_line	lookupaligned	0	+0.000	0.124	1088	search_organ_-Beschattung, Apparat-Artefakt moeglich verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4
memory_layout_aos_strict	memory_layout_cache_line	erasealigned	0	+0.229	0.213	128	search_organ_-Beschattung, Apparat-Artefakt moeglich verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4
memory_layout_aos_strict	memory_layout_cache_line	scanaligned	0	+0.020	0.096	35	search_organ_-Beschattung, Apparat-Artefakt moeglich verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4
memory_layout_aos_strict	memory_layout_cache_line	rmwaligned	0	+0.099	0.139	192	search_organ_-Beschattung, Apparat-Artefakt moeglich verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4
memory_layout_aos_strict	memory_layout_packed_doublemap	pop	-45	-0.008	0.225	1344	search_organ_-Beschattung, Apparat-Artefakt moeglich verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4
memory_layout_aos_strict	memory_layout_packed_doublemap	insert	0	-0.009	0.234	768	search_organ_-Beschattung, Apparat-Artefakt moeglich verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4
memory_layout_aos_strict	memory_layout_packed_doublemap	lookup	0	+0.000	0.265	1088	search_organ_-Beschattung, Apparat-Artefakt moeglich verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4
memory_layout_aos_strict	memory_layout_packed_doublemap	erase	0	+0.005	0.220	128	search_organ_-Beschattung, Apparat-Artefakt moeglich verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4
memory_layout_aos_strict	memory_layout_packed_doublemap	scan	0	-0.099	0.254	35	search_organ_-Beschattung, Apparat-Artefakt moeglich verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4

Fortsetzung nächste Seite

Tabelle A.6 – Fortsetzung

v	v'	Funktion	Median abs. Δ (ns)	Median rel. Δ	IQR rel.	n	Diagnose
memory_layout_aos_strict	memory_layout_packed	hitmap	0	+0.000	0.306	192	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
memory_layout_aos_strict	memory_layout_soa	ns_per_op	1170	+0.096	0.303	1344	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
memory_layout_aos_strict	memory_layout_soa	insert	0	+0.277	0.284	768	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
memory_layout_aos_strict	memory_layout_soa	lookup	0	+0.000	0.148	1088	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
memory_layout_aos_strict	memory_layout_soa	erase	0	+0.253	0.302	128	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
memory_layout_aos_strict	memory_layout_soa	scan	0	-0.014	0.186	35	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
memory_layout_aos_strict	memory_layout_soa	rmw	0	+0.107	0.185	192	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
memory_layout_aosoa	memory_layout_cache_linealigned	insertaligned	-221	-0.025	0.265	1344	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
memory_layout_aosoa	memory_layout_cache_linealigned	insertaligned	0	-0.030	0.275	768	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
memory_layout_aosoa	memory_layout_cache_linealigned	lookupaligned	0	-0.020	0.243	1088	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
memory_layout_aosoa	memory_layout_cache_linealigned	erasealigned	0	-0.038	0.259	128	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
memory_layout_aosoa	memory_layout_cache_linealigned	scanaligned	0	+0.010	0.358	32	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
memory_layout_aosoa	memory_layout_cache_linealigned	rmwaligned	0	-0.014	0.317	192	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
memory_layout_aosoa	memory_layout_packed	hitmap	-1439	-0.105	0.241	1344	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich

Fortsetzung nächste Seite

Tabelle A.6 – Fortsetzung

v	v'	Funktion	Median abs. Δ (ns)	Median rel. Δ	IQR rel.	n	Diagnose
memory_layout_aosoa	memory_layout_packed	insert	0	-0.201	0.176	768	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
memory_layout_aosoa	memory_layout_packed	lookup	0	+0.000	0.168	1088	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
memory_layout_aosoa	memory_layout_packed	listen	0	-0.171	0.147	128	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
memory_layout_aosoa	memory_layout_packed	shiftmap	0	-0.019	0.200	32	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
memory_layout_aosoa	memory_layout_packed	trimmap	0	-0.085	0.141	192	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
memory_layout_aosoa	memory_layout_soa	ns_per_op	98	+0.015	0.272	1344	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
memory_layout_aosoa	memory_layout_soa	insert	0	+0.010	0.280	768	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
memory_layout_aosoa	memory_layout_soa	lookup	0	+0.020	0.257	1088	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
memory_layout_aosoa	memory_layout_soa	erase	0	+0.020	0.256	128	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
memory_layout_aosoa	memory_layout_soa	scan	0	+0.025	0.412	32	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
memory_layout_aosoa	memory_layout_soa	rmw	0	+0.027	0.300	192	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
memory_layout_cache	memory_layout_packed	shiftmap	-1157	-0.110	0.326	1344	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
memory_layout_cache	memory_layout_packed	insert	0	-0.209	0.239	768	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
memory_layout_cache	memory_layout_packed	lookup	0	+0.000	0.416	1088	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich

Fortsetzung nächste Seite

Tabelle A.6 – Fortsetzung

v	v'	Funktion	Median abs. Δ (ns)	Median rel. Δ	IQR rel.	n	Diagnose
memory_layout_cache_inlined	memory_layout_packed	listenmap	0	-0.179	0.234	128	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
memory_layout_cache_inlined	memory_layout_packed	schinmap	0	-0.085	0.285	32	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
memory_layout_cache_inlined	memory_layout_packed	tribinmap	0	-0.097	0.363	192	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
memory_layout_cache_inlined	memory_layout_soa	ns_per_op	-32	-0.005	0.198	1344	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
memory_layout_cache_inlined	memory_layout_soa	insert	0	-0.007	0.209	768	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
memory_layout_cache_inlined	memory_layout_soa	lookup	0	+0.000	0.185	1088	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
memory_layout_cache_inlined	memory_layout_soa	erase	0	+0.005	0.209	128	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
memory_layout_cache_inlined	memory_layout_soa	scan	0	-0.043	0.144	32	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
memory_layout_cache_inlined	memory_layout_soa	rmw	0	-0.014	0.188	192	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
memory_layout_packed	memory_layout_soa	ns_per_op	1763	+0.157	0.356	1344	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
memory_layout_packed	memory_layout_soa	insert	2700	+0.288	0.337	768	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
memory_layout_packed	memory_layout_soa	lookup	0	+0.020	0.216	1088	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
memory_layout_packed	memory_layout_soa	erase	0	+0.251	0.246	128	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich
memory_layout_packed	memory_layout_soa	scan	0	+0.009	0.294	35	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich

Fortsetzung nächste Seite

Tabelle A.6 – Fortsetzung

v	v'	Funktion	Median abs. Δ (ns)	Median rel. Δ	IQR rel.	n	Diagnose
memory_layout_packed	memory_layout_soa	rmw	0	+0.127	0.260	192	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_-Beschattung, Apparat-Artefakt moeglich

Tabelle A.7: Achsen-Austauschbarkeit: prefetch (Geschwister-Paar-Diffs, Median rel. Δ ns/op bzgl. v ; 480 Geschwister-Paare; Konfundierung: am wenigsten konfundiert)

v	v'	Funktion	Median abs. Δ (ns)	Median rel. Δ	IQR rel.	n	Diagnose
prefetch_distance_estimate	prefetch_hardware	ns_per_op	-13	-0.002	0.088	1680	verschiedener Organ-Pfad
prefetch_distance_estimate	prefetch_hardware	insert	0	-0.002	0.081	960	verschiedener Organ-Pfad
prefetch_distance_estimate	prefetch_hardware	lookup	0	+0.000	0.081	1360	verschiedener Organ-Pfad
prefetch_distance_estimate	prefetch_hardware	erase	0	-0.004	0.076	160	verschiedener Organ-Pfad
prefetch_distance_estimate	prefetch_hardware	scan	0	-0.005	0.121	43	verschiedener Organ-Pfad
prefetch_distance_estimate	prefetch_hardware	rmw	0	+0.000	0.092	240	verschiedener Organ-Pfad
prefetch_distance_estimate	prefetch_none	ns_per_op	-18	-0.003	0.075	1680	verschiedener Organ-Pfad
prefetch_distance_estimate	prefetch_none	insert	0	-0.010	0.087	960	verschiedener Organ-Pfad
prefetch_distance_estimate	prefetch_none	lookup	0	+0.000	0.064	1360	verschiedener Organ-Pfad
prefetch_distance_estimate	prefetch_none	erase	0	-0.009	0.054	160	verschiedener Organ-Pfad
prefetch_distance_estimate	prefetch_none	scan	0	-0.014	0.071	43	verschiedener Organ-Pfad
prefetch_distance_estimate	prefetch_none	rmw	0	-0.007	0.093	240	verschiedener Organ-Pfad
prefetch_distance_estimate	prefetch_path_oriented	ns_per_op	50	+0.007	0.123	1680	verschiedener Organ-Pfad
prefetch_distance_estimate	prefetch_path_oriented	insert	0	+0.005	0.128	960	verschiedener Organ-Pfad
prefetch_distance_estimate	prefetch_path_oriented	lookup	0	+0.000	0.118	1360	verschiedener Organ-Pfad
prefetch_distance_estimate	prefetch_path_oriented	erase	0	-0.001	0.127	160	verschiedener Organ-Pfad
prefetch_distance_estimate	prefetch_path_oriented	scan	0	-0.010	0.156	43	verschiedener Organ-Pfad
prefetch_distance_estimate	prefetch_path_oriented	rmw	0	+0.014	0.165	240	verschiedener Organ-Pfad
prefetch_hardware	prefetch_none	ns_per_op	-10	-0.003	0.103	1680	verschiedener Organ-Pfad
prefetch_hardware	prefetch_none	insert	0	-0.002	0.118	960	verschiedener Organ-Pfad
prefetch_hardware	prefetch_none	lookup	0	+0.000	0.084	1360	verschiedener Organ-Pfad
prefetch_hardware	prefetch_none	erase	0	+0.000	0.098	160	verschiedener Organ-Pfad
prefetch_hardware	prefetch_none	scan	0	+0.000	0.119	44	verschiedener Organ-Pfad
prefetch_hardware	prefetch_none	rmw	0	+0.000	0.138	240	verschiedener Organ-Pfad
prefetch_hardware	prefetch_path_oriented	ns_per_op	36	+0.005	0.081	1680	verschiedener Organ-Pfad
prefetch_hardware	prefetch_path_oriented	insert	0	+0.003	0.076	960	verschiedener Organ-Pfad
prefetch_hardware	prefetch_path_oriented	lookup	0	+0.000	0.070	1360	verschiedener Organ-Pfad
prefetch_hardware	prefetch_path_oriented	erase	0	+0.003	0.070	160	verschiedener Organ-Pfad
prefetch_hardware	prefetch_path_oriented	scan	0	-0.003	0.053	44	verschiedener Organ-Pfad
prefetch_hardware	prefetch_path_oriented	rmw	0	+0.012	0.123	240	verschiedener Organ-Pfad
prefetch_none	prefetch_path_oriented	ns_per_op	77	+0.009	0.116	1680	verschiedener Organ-Pfad
prefetch_none	prefetch_path_oriented	insert	0	+0.010	0.116	960	verschiedener Organ-Pfad
prefetch_none	prefetch_path_oriented	lookup	0	+0.000	0.109	1360	verschiedener Organ-Pfad
prefetch_none	prefetch_path_oriented	erase	0	+0.004	0.090	160	verschiedener Organ-Pfad
prefetch_none	prefetch_path_oriented	scan	0	-0.005	0.107	42	verschiedener Organ-Pfad
prefetch_none	prefetch_path_oriented	rmw	0	+0.026	0.115	240	verschiedener Organ-Pfad

A.5 Limitierungen

Tabelle A.8 führt alle nicht-gefixten Vorbehalte der Messreihen auf, damit kein Befund still verfällt; Zeile 1 betrifft die fehlende PMC-Kernmetrik (Cache-Misses).

Tabelle A.8: Ehrliche Limitierungen: nicht-gefixte Vorbehalte (kein Befund verfällt still)

#	Vorbehalt	Status / Konsequenz
1	Cache-Misses (Kernmetrik): L1/L2/L3 + dTLB + Coherence + Energy = 0 / nicht erhoben	NullPmcSource, available=false; Intel-PCM-Windows-Treiber + Linux-PAPI ausstehend (#26/P4); Cache-Verhalten nur indirekt über Wall-Clock-Proxy (seg_memory_layout_ns/ns_per_op).
2	15 gepinnte Achsen = 0 Austauschbarkeits-Belege	Nur 4 Achsen variieren (search_algo, node_type, memory_layout, prefetch). Gepinnt (je 1 Wert): cache_traversal, mapping, path_compression, allocator, concurrency, serialization, telemetry, value_handle, isa, index_organization, io_dispatch, migration_policy, filter, queuing_q1, queuing_q2.
3	Observer-Proben-Zähler (*_find/*_probe/*_get) by-design ≈ 1	Erwartetes Verhalten, kein Phantom: je Lookup eine Probe. Keine Aussage über interne Iterationen.
4	Honest-0 inaktiver Sub-Features gepinnter Strategien	concurrency/migration/io/value_handle melden 0 (Feature inaktiv), nicht Mess-Fehler.
5	Wall-Clock nicht bit-reproduzierbar	Seed steuert Keys (deterministisch), NICHT das CPU-Timing; Wall-Clock variiert lauf-zu-lauf.
6	RC-Dimension nur nominal	K1: $\times 18 \rightarrow \times 3$ degeneriert; RC-Achse liefert keine volle Ausprägungs-Breite.
7	2/21 Scan-Profile = No-Op (ausgeschlossen)	ycsb_e und lp_range_scan führen keinen echten Scan aus; bei scan-Diffs ausgeschlossen (19 valide Lastprofile).
8	Insert-Profile messen Upserts; stat_*-Spalten enthalten Load-Phase	Insert-Pfad ist faktisch Upsert; die stat_*-Zähler akkumulieren die Lade-Phase mit.
9	prefetch misst Key-Werte als Pseudo-Adressen (K9)	Prefetch-Achse interpretiert Schlüssel-Werte als Adressen; kein echter Speicher-Prefetch.
10	node_type + memory_layout: Q2-Schritt-4-Beschattung	Apparat-Artefakt möglich (search_organ_-Beschattung); search_algo + prefetch sind am wenigsten konfundiert.
11	memory_layout-Effekt teils sub-noise	Teil der Layout-Differenzen liegt unter dem Mess-Rauschen $\rightarrow$ PMC extern-gated (#26).

B Code-Struktur

C Glossar

Dieses Glossar fasst die in der Arbeit verwendeten Abkürzungen und zentralen Begriffe zusammen.

ABI Application Binary Interface — die binär-stabile Schnittstelle, über die der Builder die Permutations-Module (Lebewesen) lädt.

Achse Eine orthogonale Teilentscheidung („Organ“) eines Suchalgorithmus; das Framework führt neunzehn Kompositions-Achsen.

ART Adaptive Radix Tree [LKN13]; Referenzentwurf mit adaptiven Knotengrößen (Node4/16/48/256).

AVX, AVX-512 x86-SIMD-Erweiterungen (256- bzw. 512-Bit).

CLU Cache-Line-Utilization — Anteil genutzter Bytes je geladener Cache-Line.

CPI, IPC Cycles per Instruction bzw. Instructions per Cycle.

CRTP Curiously Recurring Template Pattern — C++-Idiom für statischen Polymorphismus.

dTLB Daten-Translation-Lookaside-Buffer; cacht Daten-Adressübersetzungen.

Gattung Ein nach außen sichtbares Interface (SearchAlgorithm, Container, Graph) — die oberste Ebene des Frameworks.

GoF „Gang of Four“— die kanonischen Entwurfsmuster nach Gamma et al. [GHJV94].

HBM High-Bandwidth Memory.

HOT Height Optimized Trie [BZP⁺18].

Lebewesen Ein vollständiger Suchalgorithmus als Komposition aller Achsen (metaphorischer Code-Begriff; technischer Identifier: SearchAlgorithm-Instanz).

LOUDS Level-Ordered Unary Degree Sequence — succinct Baum-Kodierung (Jacobson 1989).

MESI, MOESI, MESIF Cache-Kohärenzprotokolle (Grund-Protokoll bzw. AMD-/Intel-Varianten).

NINE Non-Inclusive-Non-Exclusive — Inklusions-Politik des L3 in Intel-Server-Prozessoren.

NUMA Non-Uniform Memory Access.

OLC Optimistic Lock Coupling — optimistische Nebenläufigkeits-Technik (ARTSync, P08).

PMC (Hardware) Performance Monitoring Counter.

Prüf-Dock Die Builder-Seite, die genau eine Gattung lädt, durch ihr Interface treibt, vermisst und persistiert.

Prüfling Der testweise einzugliedernde neue Suchalgorithmus (hier PRT-ART); wird vom Builder gegen die Kernbestandteile der Cache-Engine kompiliert.

PRT-ART Probst-Redirect-Tree-/ART-Hybrid — der Prüfling dieser Arbeit.

Rang Zentralitäts-Einordnung der Stand-der-Technik-Veröffentlichungen (Rang-1/2/3); erklärt in Abschnitt 1.3.

RCU Read-Copy-Update — Reklamations-Schema (McKenney, P29).

RRIP, DRRIP (Dynamic) Re-Reference Interval Prediction — cache-Ersetzungs-Heuristik [JTJE10].

SIMD Single Instruction, Multiple Data.

SOSD Search On Sorted Data — Index-Benchmark [KMR⁺19].

SOTA State of the Art — der Stand der Technik.

SuRF Succinct Range Filter [ZLL⁺18].

SVE, NEON, RVV ARM- (SVE, NEON) bzw. RISC-V- (RVV) SIMD-Erweiterungen.

TLB Translation Lookaside Buffer.

YCSB Yahoo! Cloud Serving Benchmark [CST⁺10].

D Bausteine-Matrix

Dieser Anhang enthält die vollständige Bausteine-Matrix, auf die der Stand der Technik (Kapitel 3) verweist. Je Achse sind der Zweck, die *statischen* Sub-Achsen (übersetzungszeitlich wählbare Varianten-Familien) und die *dynamischen* Sub-Achsen (Laufzeit-Parameter) sowie die konkreten Baustein-Varianten mit ihrer Quelle (Paper-Kennung oder Code/Baseline) aufgeführt. Die Angaben sind aus dem Code der `comdare-cache-engine` und gegen die Paper-Provenienz-Map (Doc 18) verifiziert.

D.1 T0 Suchalgorithmen-Achse T0 (Such-Strukturen und Such-Methoden)

Systematische Permutation von 17 konkreten Suchalgorithmen und Such-Strukturen (Array-basiert, sortierte Vektoren, Original-Paper-Wrapper, Such-Methoden wie k-ary/Interpolation, sowie geordnete Strukturen wie Skip-Liste/B-Baum) zur messbaren Quantifizierung des Einflusses der Such-Paradigma-Wahl auf Performance-Charakteristiken im einheitlichen `std::map`-Interface. Sub-Achsen klassifizieren nach Zugriffsdichte (SA1 dense, SA2 sparse, SA3/SA4 multilevel), ermöglichen Compliance-Nachweise über Paper-Mixins (Habich-Anforderung: `is_original`-Markierung pro Funktion/Wrapper), und identifizieren SIMD-fähige sowie store-traversierbare Varianten.

Statische Sub-Achsen

ID	Name	Beschreibung
SA1	Direkt-adressiert hohe Dichte (Dense Access)	Array-Familie: direkt-adressiert ($O(1)$ lookup), hohe Füllungsdichte $\geq 50\%$. Umfasst <code>Array256SearchAlgo</code> (256-Slot compact array, ART Node256 Re-Impl) und <code>Array65535SearchAlgo</code> (uint16-basiert, mid-density 25-50 % direkt-adressiert).
SA2	Sparse/Geordnet (Sparse Access)	Sortierte Vektoren, geordnete Such-Methoden und tree/hash-Strukturen: niedrige Füllungsdichte, Range-Scan-Support, variable $O(\log n)$ - $O(1)$ Lookup. Umfasst <code>VectorU8U8SearchAlgo</code> , Original-Paper-Wrapper (ART/HOT/START/Wormhole/SuRF), Such-Methoden (k-ary, Interpolation, Eytzinger, Hash), sowie Strukturen (SkipList, BST, B-Baum).
SA3	Multi-Byte-Diskriminatoren (Multilevel Access)	Mehrbyteige Such-Strukturen mit indizierten Sub-Diskriminatoren (z.B. START mit Cost-DP-Mehrebenen). <code>VectorU16U16SearchAlgo</code> (u16-Keys, sorted lower_bound) und <code>OriginalStartSearchAlgo</code> (Paper P05) gehören hier.
SA4	Direkt-adressiert mittlere Dichte (Direct Multibyte)	Teilt die Direkt-Adressierungs-Zugriffsmuster (wie SA1) mit Multi-Byte-Granularität (uint16-Größe) und mittlerer Füllungsdichte 25–50 % (im Gegensatz zu SA1 $\geq 50\%$). Beispiel: prt-art <code>Array65535</code> (REV 6 §5.17). Diese Subachse wurde hinzugefügt, um die SA1-Abstraktion nicht zu brechen, aber die Density-Variante zu erfassen.

Bausteine

Variante	Beschreibung	Quelle
Array256SearchAlgo (S01)	ART Node256 direkt-adressierte Re-Implementierung: <code>std::array<std::optional<u64>, 256></code> mit $O(1)$ Lookup/Insert/Erase. Cache-Line-aligned, SIMD-fähig, kein Heap-Allocation. Baseline für dichte Zugriffe.	Code/Baseline (Leis ICDE 2013 ART Paper P01)
VectorU8U8SearchAlgo (S02)	Sparse sortiertes Key-Value-Vektor-Paar: <code>std::vector<u8> keys + std::vector<u64> values</code> mit <code>std::lower_bound</code> . $O(\log n)$ Lookup, $O(n)$ Insert/Erase. Konzept-äquivalent zu HOT-k-Constrained. SIMD-fähig (Bitmask-Scan).	Code/Baseline (Binna SIGMOD 2018 HOT Paper P02)
VectorU16U16SearchAlgo (S03)	Sparse sortiertes u16-Vektor-Paar: <code>std::vector<u16> keys + std::vector<u64> values</code> . Vereinfachtes START-Konzept (Multi-Byte-Diskriminatoren ohne Cost-DP). $O(\log n)$ Lookup, $O(n)$ Insert/Erase.	Code/Baseline (Fent ICDEW 2020 START Paper P05)
Array65535SearchAlgo (S09)	Mid-Density direkt-adressiert uint16-basiert (25–50 % Füllungsdichte). Aus prt-art Legacy-Code (REV 6 §5.17) migriert. $O(1)$ Lookup aber mit Speicher-Overhead vs. SA1 denser Arrays.	Code/Baseline (prt-art F.6-Migration)
OriginalArtSearchAlgo (S04)	ART Paper-Bindung (<code>unodb::db::ist_original_module()=true</code> (4/4 Funktionen original)). Paper-Mixin via SHA256-validierte Tool-Codegenerierung. Habich-Compliance: <code>get_compiler()='gcc-9.5'</code> , all insert/lookup/erase/clear marked original.	P01 (Leis/Kemper/Neumann ICDE 2013, Apache-2.0 unodb)
OriginalHotSearchAlgo (S05)	HOT Paper-Bindung (HOTRowex): <code>is_original_module()=false</code> (2/4 original). insert/lookup original, aber erase/clear Lücken (HotRowex ist append-only; Re-Impl fallback). Paper P02.	P02 (Binna et al. SIGMOD 2018, ISC ISC)
OriginalStartSearchAlgo (S06)	START Paper-Bindung (sosd-Adapter): <code>is_original_module()=false</code> (2/4 original). insert/lookup original, erase/clear Lücken (kein remove im sosd-Adapter). Paper P05, Multi-Byte Diskriminatoren.	P05 (Fent/Jungmair/Kipf/Neumann ICDEW 2020, MIT)
OriginalWormholeSearchAlgo (S07)	Wormhole Paper-Bindung: <code>is_original_module()=false</code> (3/4 original). put/get/del original, clear Lücke. Hybrid Hash+Trie+B+ Index. GPL-3.0-lizenziert (blockiert Original-Code-Linking für Thesis).	P07 (Wu/Ni/Jiang EuroSys 2019, GPL-3.0)
OriginalSurfSearchAlgo (S08)	SuRF Paper-Bindung (FST mit LOUDS-Bitmap): <code>is_original_module()=false</code> (1/4 original, nur lookup). Succinct Range Filter, Multi-Header-Bibliothek. Paper P10.	P10 (Zhang/Lim/Andersen SIGMOD 2018, Apache-2.0)
KArySearchAlgo (S10)	k-ary Search Such-METHODE (Schlegel/Gemulla/Lehner DaMoN 2009): K-Wege-Partition statt Halbierung bei der Binärsuche. K ist iterable_aspect ($K \in \{2,4,8,16\}$). $O(\lceil \log_{K+1} n \rceil)$ Iterationen, parallele ILP-Vergleiche. Array-Familie aber kein treues dediziertes Traversal-Organ (SA2-Klassifikation). C++23-Re-Impl, <code>is_original=false</code> .	Paper (Schlegel/Gemulla/Lehner DaMoN 2009 pseudocode study)
InterpolationSearchAlgo (S11)	Interpolationssuche Such-METHODE (Perl/Itai/Avni CACM 1978): verteilungsbewusste Positionsschätzung statt blinder Halbierung. $O(\log \log N)$ avg bei Gleichverteilung, $O(N)$ worst-case. Nicht SIMD-vektorisierbar (datenabhängiger Index). C++23-Re-Impl.	Paper (Perl/Itai/Avni CACM 1978, pseudocode)

Variante	Beschreibung	Quelle
EytzingerSearchAlgo (S12)	Eytzinger/BFS-Layout branch-free Suche (Khuong/Morin JEA 2017): Cache-Layout-Such-METHODE mit branch-free Kern. Keys in BFS-Ordnung des impliziten Suchbaums. Fuer große Arrays laut Paper schnellste Allzweck-Layout. C++23-Re-Impl (Experiment-Harness ohne Standard-Lizenz).	Paper (Khuong/Morin JEA 2017 arXiv:1509.05053)
SkipListSearchAlgo (S13)	Skip-Liste probabilistische geordnete STRUKTUR (Pugh CACM 1990): $O(\log n)$ erwartet search/insert/erase ohne explizites Rebalancing. Jeder Knoten hat 1..kMaxLevel Forward-Zeiger (Münzwurf Probability 0.5). Index-basiert (<code>std::vector<Node></code> mit uint32 indices, value-semantisch). Lehrbuch-Algorithmus, C++23-Re-Impl.	Paper (Pugh CACM 1990, pseudocode)
HashSearchAlgo (S14)	Open-Addressing-Hashtabelle UNGEORDNET (Knuth TAOCP 3 §6.4): Fibonacci-Hash, Linear Probing, Tombstone-Erase bei Resize. $O(1)$ erwartet lookup/insert bei <code>load_factor < 0.7</code> . Keine Range-Scan-Unterstützung (distinkte Klassifikation). Korrekte Tombstone-Markierung zur Probe-Ketten-Integrität. C++23-Re-Impl.	Paper (Knuth TAOCP 3 §6.4 1998, textbook)
LinearScanSearchAlgo (S15)	Unsortierter linearer Scan Baseline (Leis ICDE 2013 ART Node4): $O(n)$ lookup auf unsortiertem KV-Array. Einfachste Vergleichs-Nullpunkt-Struktur: insert $O(1)$ -amortisiert, erase $O(1)$ via swap-and-pop. Store-traversierbar (Array-Familie). Keine Range-Scan-Unterstützung.	Code/Baseline (Leis ICDE 2013 ART Paper P01 Node4)
BinarySearchTreeSearchAlgo (S16)	Unbalancierter Binärer Suchbaum (Knuth TAOCP 3 §6.2.2 Hibbard-Deletion): deterministische Vergleichs-Baum-Baseline (vs. SkipList probabilistisch). Index-basiert (<code>std::vector<Node></code> mit uint32 indices, Free-List). Hibbard-erase (3 Fälle: Blatt/1 Kind/2 Kinder via In-Order-Nachfolger). $O(n)$ worst-case bei sortiertem Input.	Paper (Knuth TAOCP 3 §6.2.2 1998, textbook)
BTreeSearchAlgo (S17)	Balancierter Block-orientierter Mehrwege-B-Baum (Bayer/McCreight Acta Inf. 1972 / CLRS Kap. 18): $t=4$, daher [3..7] Keys pro Knoten, bis zu 8 Kinder. IMMER balanciert ($O(\log n)$ guaranteed). Index-basiert, <code>alignas(64)</code> -Knoten (Cache-Line-Alignment). Top-Down Splitten beim Insert, Borrow/Merge beim Erase. Das DB-Index-Arbeitspferd.	Paper (Bayer/McCreight Acta Informatica 1972 / CLRS Ch. 18, textbook)

Beitragende Paper: P01, P02, P05, P07, P10.

D.2 T1 Cache-Speicher-Traversierung (Fanout-Lookup)

Die Achse `axis_03b` definiert drei distinkte Strategien zur Schlüssel-Auflösung in kleinen Fanout-Arrays (typisch: B+-Tree oder Trie Inner Nodes). Sie spannt ein Spektrum von linearer Suche ($O(N)$, cache-freundlich, klein), über multiplikatives Hashing mit Open-Addressing ($O(1)$ amortisiert, Knuth-Fibonacci), bis zur sortierten Binärsuche ($O(\log N)$, geordnete Traversierung, B+-Tree-Standard). Alle drei sind Lehrbuch-Baselines ohne Original-Code-Linking, implementiert als reine C++23-Re-Implementierungen.

Statische Sub-Achsen

ID	Name	Beschreibung
CT1	Lineare Zugriffe	Linear-scan-basierte Lookup-Methoden auf kleinen, unsortierten Fanout-Arrays. Charakterisiert durch $O(N)$ -Auflösung mit einfachen Vergleichen, typisch für Nodes mit Fanout < 32 in B+-Trees. Sub-Achse für <code>LinearFanout</code> und <code>BinarySearchFanout</code> (beide: <code>axis_tag = linear_access_tag</code>).
CT2	Hash-basierte Zugriffe	Hash-Lookup mit Open-Addressing und linearem Probing. $O(1)$ amortisierte Auflösung, charakterisiert durch Fibonacci-Multiplikator $(11400714819323198485 \approx 2^{64}/\varphi)$, Power-of-2 Bucket-Count, dynamisches Rehashing bei Load-Factor > 0.7 . Nur Hash Lookup-Wrapper. Sub-Achse: <code>axis_tag = hash_access_tag</code> .

Bausteine

Variante	Beschreibung	Quelle
<code>LinearFanout</code>	Unsortierte Fanout-Traversal via <code>std::find_if</code> und linearem Scan. $O(N)$ Lookup, kein Hash-Overhead, cache-freundlich für kleine N (< 32). Klassische B-Tree-Baseline (Bayer/McCreight 1972). Gehört zu Sub-Achse CT1.	Code/Baseline — Bayer/McCreight 1972 (Acta Informatica)
<code>HashLookup</code>	Fibonacci-Hash open-addressing mit linearem Probing (Knuth TAOCP Vol. 3, §6.4). $O(1)$ amortisiert, dynamisches Rehashing (verdoppelt Capacity bei load ≥ 0.7), iterable Laufzeit-Kapazität (<code>kIterableInitialCapacities: {8, 16, 64, 256, 1024}</code>). Standalone C++23-Re-Impl, keine Original-Code-Bindung. Gehört zu Sub-Achse CT2.	Code/Baseline — Knuth 1998 (The Art of Computer Programming Vol.3)
<code>BinarySearchFanout</code>	Sortierte Fanout-Traversal via <code>std::lower_bound</code> . $O(\log N)$ Lookup bei sortierter Ordnung, bewahrt Schlüssel-Sequenzierung für Range-fähige Routing-Layer. Klassische B+-Tree-Inner-Node-Suche (Bayer/McCreight 1972), dritte distinkte Strategie der Achse. C++23-Re-Impl (R7.2, 2026-05-29), <code>is_original=false</code> . Gehört zu Sub-Achse CT1.	Code/Baseline — Bayer/McCreight 1972 (Acta Informatica)

Beitragende Paper: P17, P21, P27, P32.

D.3 T2 Slot-zu-Offset-Mapping (Traversal-Sublayer)

Vermittlung zwischen logischen Slot-Indizes und physischen Speicher-Offsets in Trie-Inner-Nodes und Speicher-Pools. Die Achse bietet zwei grundsätzlich verschiedene Zuordnungsstrategien: direkte absolute Offsets (lineare Packing) oder pool-relative Offsets mit automatischem Rebase-Mechanismus zur Unterstützung von Speicher-Umallokation und Persistent-Data-Structures.

Statische Sub-Achsen

ID	Name	Beschreibung
MP1	Direct-Access-Mapping (MP1)	Direkter Slot-zu-Absolute-Offset-Mapping ohne Indirection. Wrapper-Familie für klassisches Array-of-Pointers-Layout wie in B-Tree Inner-Nodes (Bayer/McCreight 1972). Einfache Cache-Locality, O(N) Lookup per <code>std::find_if</code> .
MP2	Pool-Relative-Mapping (MP2)	Slot-zu-Pool-Relativem-Offset-Mapping mit Based-Pointer-Arena-Pattern. Alle Offsets relativ zu dynamischem <code>pool_base_address</code> gespeichert. Ermöglicht O(1) Rebase bei Speicher-Umallokation — zentrale Eigenschaft für Persistent-Data-Structures (Driscoll et al. 1989) und CustomAligned Structure-Pattern in prt-art.

Bausteine

Variante	Beschreibung	Quelle
DirectPlacement	Wrapper für direktes Slot-zu-Absolute-Offset-Mapping (lineares Packing). <code>std::vector<pair<slot_index_type, offset_type>; resolve_offset/register_slot</code> via <code>linear std::find_if</code> . Verwendungskontext: prt-art <code>INode::placement_page_ + Mapping-Table</code> . Speicher: 16-bit Slot-Index + <code>size_t</code> Offset pro Entry. Keine Pool-Basis-Abhängigkeit.	CE-Baseline
PoolRelative	Wrapper für Pool-Relatives-Offset-Mapping mit O(1)-Rebase-Fähigkeit. <code>std::vector<pair<slot_index_type, relative_offset> + mutable pool_base_address</code> . Bei Pool-Umallokation: <code>rebase(new_pool_base)</code> setzt nur <code>pool_base_neu</code> — alle relativen Offsets bleiben gültig. Verwendet bei prt-art CustomAligned Structure + Versioning-Szenarien (Persistent Data Structures nach Driscoll/Sarnak/Sleator/Tarjan 1989).	CE-Baseline

Beitragende Paper: P03, P07, P27.

D.4 T3 Pfadkompression

Definiert die Kompressions-Granularität und -Strategie für Schlüsselpfade in Trie-Strukturen. Steuert, wie innere Knoten redundante Byte-Sequenzen oder signifikante Bits gemeinsamer Präfixe speichern und navigieren, um Speicher- und Cache-Effizienz zu optimieren.

Statische Sub-Achsen

ID	Name	Beschreibung
pc1	Kompressions-Granularität (PC1)	Definiert die minimale Einheit der Pfad-Kompression: Bit-weise (Patricia), Byte-weise (ART) oder keine Kompression (raw path). Bestimmt die Trie-Höhe und navigationsoptimale Codeblöcke im Traversal.
pc2	Sprung-Strategie (PC2)	Definiert das Navigations-Pattern bei inneren Knoten: Linear über 8 Bytes (ByteWise), Single-Bit-Vergleich (Patricia) oder direkter Sprung ohne Zwischenschritte. Beeinflusst Branch-Dichte und Prefetch-Verhalten.
pc3	Decode-Komplexität (PC3)	Klassifiziert die Laufzeit-Komplexität beim Extrahieren von Präfix-Informationen: O(1) für direkte Extraktion, O(log n) für struktur-basierte Suche, O(n) für sequenzielle Prüfung. Wichtig für Operationen wie <code>common_prefix_len()</code> und Schlüssel-Vergleiche.

Bausteine

Variante	Beschreibung	Quelle
PathCompression None	Keine Pfad-Kompression; rohe Schlüsselpfade werden vollständig und unkomprimiert in Knoten gespeichert. Dient als Mess-Baseline mit <code>compression_ratio=1.0</code> . Verwendet als Kontrollfall zur Quantifizierung der Speicher- und Höhen-Einsparungen komprimierender Strategien.	Code/Baseline
ByteWisePath Compression	Byte-für-Byte-Kompression (ART-Stil); speichert bis zu 7 Präfix-Bytes pro Knoten in einem kompakten <code>u64</code> -Wort mit Längenetikett im High-Byte. Reduktion der Trie-Höhe um typischerweise Faktor 2 (<code>compression_ratio=0.5</code>). Ursprung: Leis et al., ART ICDE 2013 (P01). Verwendet Goldstandard-Operationen <code>common_prefix_len()</code> und <code>cut()</code> für Navigations-Optimierung.	P01
PatriciaPath Compression	Single-Bit-Split-Pfadkompression (Patricia-Trie); navigiert bitweise statt byteweise, reduziert Trie-Höhe um typischerweise Faktor 3 (<code>compression_ratio=0.3</code>). Ursprung: Morrison PATRICIA JACM 1968; modern implementiert in HOT (Binna SIGMOD 2018, P02) und Wormhole (Wu EuroSys 2019, P07). F15-operative Primitiven: <code>key_split_bit()</code> für MSB-first Bit-Extraktion, <code>path_descend_scan()</code> für goldstandard-konforme <code>_scan</code> -Signatur mit schlüssel-abhängiger variabler Tiefe.	P02

Beitragende Paper: P01, P02, P04, P05, P09.

D.5 T4 Knotentypen (Node-Type-Achse)

Die Achse `axis_04_node_type` definiert die vier Knotenformate des Adaptive Radix Tree (ART) als austauschbare Varianten: Node4, Node16, Node48 und Node256. Jeder Knotentyp bietet eine andere Balance zwischen Speicher, Cache-Effizienz und Suchgeschwindigkeit. Diese Achse bildet die Basis für adaptive Baum-Reorganisationen und ist zentral für das Design der Suchalgorithmen.

Statische Sub-Achsen

ID	Name	Beschreibung
NT1	Kapazitätsklasse (NT1)	Klassifizierung nach Fanout-Größe und Slotanzahl: klein (Node4 mit 4 Slots, linear search), mittel (Node16 mit 16 Slots, SIMD-binary-search) bis maximal (Node256 mit 256 Slots, direct-addressed).
NT2	Zugriffsmuster (NT2)	Art der Schlüsselsuche innerhalb eines Knotens: linearer Scan (Node4), binäre Suche mit SIMD (Node16), indirekte Indizierung (Node48) oder direkte Adressierung (Node256).
NT3	Kompaktheit (NT3)	Speicherlayout und Dichte: spärlich und cache-aligned (Node4), dicht mit Indirection (Node48) oder maximal direkt (Node256), mit Auswirkungen auf Speicherverbrauch und Cache-Line-Misses.

Bausteine

Variante	Beschreibung	Quelle
Node4NodeType	Kleiner ART-Knoten mit 4 Slots und linearer Suche. Ideal für spärlich besetzte obere Bauebenen, passt komplett in eine Cache-Line (64 Byte). Implementiert als CE-Reimplementierung der ART-Spezifikation mit <code>node_find_scan()</code> als linear scan über sortierte Schlüssel.	P01
Node16NodeType	Mittlerer ART-Knoten mit 16 Slots, optimiert für SIMD-basierte binäre Suche (SSE2 PCMPSTRM / AVX2 VPCMPSTRM). Balance zwischen Speicher und Suchgeschwindigkeit; größte Kapazität ohne Indirection. CE-Re-Impl der Leis-ICDE-2013-Spezifikation.	P01
Node48NodeType	Großer ART-Knoten mit 48 Slots, nutzt Indirection: 256-Byte ChildIndex (Byte→Slot-Mapping, 0=leer) plus 48er ChildArray. Trade-off zwischen Speicher (vs Node256 direct) und Cache-Misses (vs Node4 linear). Doppelter Indirektions-Zugriffsmuster (Index + Child).	P01
Node256NodeType	Maximaler ART-Knoten mit 256 Slots, direkt adressiert ohne Index. O(1) Lookup (kein Scan, kein Index-Touch). Hoher Speicherverbrauch (256 Pointer), aber optimale Suchgeschwindigkeit für dicht besetzte Knoten. Default für untere Bauebenen.	P01
NodeTypeSlotStore	Storage-Organ: <code>node_type</code> -getriebener boundierter Slot-Speicher. Erstreckt sich über <code>std::array</code> mit Kapazität <code>N: :max_capacity()</code> über einen gemeinsamen uint64-Key. Erfüllt das StorageOrgan-Concept und ist Drop-in-Substrat für ComposedSearch. Baseline-Variante ohne Layout/Allocator-Integration.	Code/Baseline
ComposedStore	3-Achsen-Storage-Organ: $\text{node_type}(N) \oplus \text{layout}(L) \oplus \text{allocator}(A)$. Unboundierter vector über Allocator A; layout fließt als statisches <code>cache_line_size</code> und via optionale <code>organ_scan_field_sum()</code> ein. Real messbar (F15-Brücke).	Code/Baseline
NodeChunkedStore	3-Achsen-Storage-Organ mit Laufzeitwirkung der <code>node_type</code> -Achse: Slots liegen in Chunks der Kapazität <code>N: :max_capacity()</code> . Chunk-Count = Zahl der node-großen Allokationen; damit ist die <code>node_type</code> -Achse REAL messbar (Node4 viele Chunks, Node256 wenige). Beweis für Delegation.	Code/Baseline
LayoutAware ChunkedStore	Layout-ehrende Variante von NodeChunkedStore: speichert Records am layout-getriebenen Stride (<code>eff_stride</code>). Jeder Record belegt <code>eff_stride</code> Bytes (Key 0..8, Value 8..16, Rest Padding). Macht <code>memory_layout</code> -Achse echt messbar, OOB-sicher für scan-Methoden.	Code/Baseline
ObservableNode Type	ObservableAxis-Hülle um eine <code>node_type</code> -Strategie. Reicht die statische API durch $(\text{max_capacity}/\text{name}/\text{topic_tag})$ + bietet <code>observe_node_find()</code> als Instanz-Driver. Trackt <code>find_count</code> , <code>keys_stored</code> , <code>queries_run</code> , <code>last_checksum</code> . Format-divergente Lookup-Latenz messbar (Pfad B).	Code/Baseline

Beitragende Paper: P01.

D.6 T5 Speicher-Layout (Memory Layout)

Diese Achse klassifiziert die räumliche Organisation von Datensätzen im Speicher und beeinflusst damit Cache-Effizienz, SIMD-Vectorisierbarkeit und False-Sharing-Charakteristiken. Sie erfasst orthogonal vier Design-Dimensionen: (1) Alignment-Strategien (Cache-Line-Padding vs. Strict-Packing), (2) Datenorganisation (zeilenorientiert AoS vs. spaltenorientiert SoA vs. Hybrid AoSoA), (3) Packing-Dichte (dense vs. succinct bit-packed), (4) Stride-Muster (sequenziell vs. interleaved). Die Layout-Wahl ist ein primärer Treiber für Cache-Lokalität und SIMD-Fähigkeit bei Index-Traversal.

Statische Sub-Achsen

ID	Name	Beschreibung
HM1	Alignment-Strategie	Entscheidung zwischen Cache-Line-gepaddetem Layout (False-Sharing-Vermeidung) und Strict-Packing (Speicherkompaktheit). Bestimmt Stride-Overhead bei sequenzieller Traversal.
HM2	Daten-Organisation	Grundlegende Speicher-Ordnung: Array-of-Structs (AoS, zeilenorientiert), Struct-of-Arrays (SoA, spaltenorientiert), oder Hybrid AoSoA (Block-basierte Mischung). Kritisch für Cache-Line-Ausnutzung bei Feld-selektiven Scans.
HM3	Packing-Dichte	Dichtegrad der Speichernutzung: von Standard-Records (8–64 Byte) bis zu kompaktem Bit-Packing (z.B. LOUDS-Bitmaps mit ~1 Bit pro Slot). Beeinflusst Decode-Latenz und L1/L2-Auslastung.
HM4	Stride-Muster	Muster der Speicheradressierung bei Traversal: Sequenzielle Strides (aufeinanderfolgende Records), Interleaved (z.B. um Hot-Cold-Daten zu trennen) oder Cache-Aware-Layouts (z.B. Eytzinger BFS). Optimiert Prefetch-Effizienz.

Dynamische Sub-Achsen

ID	Parameter	Beschreibung
CacheLineSize	Cache-Line-Größe (Unterachse KF-5)	Runtime-Parameter für die Zielplattform-Cache-Zeile: 64 Byte (Standard x86-64/ARM64), 128 Byte (Azure, manche Power ISAs) oder 32 Byte (embedded). Wird zur Compile-Time über <code>CacheLineConfig-NTTP</code> gewählt und beeinflusst Alignment/Padding-Overhead aller alignierten Layouts.

Bausteine

Variante	Beschreibung	Quelle
AoSStrictMemoryLayout	Array-of-Structs, kompakt gepackt ohne Cache-Line-Alignment. Vorteil: minimale Speichernutzung (record_size-Strides, kompakt). Nachteil: False-Sharing bei concurrent Writes, schlechte Cache-Hit-Quote bei schmalen Feldscans. Typisch für Wormhole-Index.	Code/Baseline
CacheLineAlignedMemoryLayout	AoS mit 64-Byte-Alignment per Record. Vermeidet False-Sharing durch Padding (Stride = round_up(record_size, 64)). Standard für ART/HOT/Masstree/START. Operativ: Bei record_size=48 (Mess-Build) entsteht 16-Byte-Overhead pro Record → messbar höhere Latenz als aos_strict, echter Cache-Effekt.	Code/Baseline
SoAMemoryLayout	Struct-of-Arrays: Alle Felder spaltenweise hintereinander. Vorteil: Kontiguierliche Feldscans (z.B. nur Keys ohne Values) nutzen maximal Cache-Lines (16× weniger Cache-Lines als AoS bei Einfeld-Scan). Ideal für SIMD-Vectorization und OLAP-Indizes. Vorteil beschrieben in Abadi et al. SIGMOD 2008.	P? (Abadi SIGMOD 2008 verwandt)
PackedBitmapMemoryLayout	Bit-packed succinct Layout (LOUDS / SuRF FST). Speichert n Einträge mit ~n*lg(sigma) Bits statt bytes. Extreme Kompaktheit (32× weniger Cache-Lines als AoS), aber höherer Decode-Overhead per Lookup. Fundament: Jacobson LOUDS FOCS 1989 (P09). Moderne Impl: SuRF Apache-2.0.	P09 (Jacobson LOUDS FOCS 1989)

Variante	Beschreibung	Quelle
AoSoAMemoryLayout	Array-of-Structures-of-Arrays: Hybrid-Layout mit Block-Breite W=8 (AVX2-Lanes). Innerhalb eines Blocks: SoA (Felder kontiguiertlich). Über Blocks: AoS (Blocks sind gestrided). Vereint SIMD-Vectorisierbarkeit (SoA innerhalb mit räumlicher Lokalität (AoS über). Standard in HPC/SIMD-Engines. Konvention; kein einzelnes Ursprungspaper, aber ISPC-Idiom.	Code/Baseline (HPC/SIMD-Konvention)

Beitragende Paper: P09.

D.7 T6 Allokator-Achse (T6)

Diese Achse stellt die Speicherverwaltungsstrategie für die Cache-Engine bereit. Sie definiert 25 konkrete Allocator-Wrapper für verschiedene Algorithmen und Techniken (von klassischen Baselines wie Buddy und Slab über moderne Research-Allokatoren wie mimalloc, jemalloc, sncmalloc bis zu spezialisierten Varianten für NUMA, PIM und formal verifizierte Systeme). Alle Wrapper erfüllen standardisierte Concepts und sind orthogonal nach 7 Compile-Time-Varianten-Familien (Freelist-Topologie, Size-Class-Schema, Thread-Lokalität, Synchronisation, Allokationspolicy, Reklamation, Fragmentierungsstrategie) strukturierbar.

Statische Sub-Achsen

ID	Name	Beschreibung
AA1	Free-List-Topologie	Wie sind freie Speicherblöcke organisiert? Beispiele: Per-Page-Sharding (mimalloc A04), Per-Thread-Caching (sncmalloc A07), Buddy-Tree (A19), Single-Global-Free-List (dlmalloc A20), Spans (scallop A08), hybride Strategien (hmalloc A15).
AA2	Size-Class-Schema	Wie werden Block-Größen klassifiziert? Beispiele: Slab pro Objektgröße (A02), dlmalloc Bins (A20), jemalloc Größenklassen (A05), scallop Spans (A08), Buddy Power-of-2 (A19), formal verifizierte Bins (StarMalloc A13), std::pmr pools (A22).
AA3	Thread-Lokalität	Wie lokal pro Thread/Core ist die Allokation? Beispiele: tcmalloc thread-local-caches (A06), rpmalloc thread-heaps (A10), sncmalloc message-passing (A07), tcmalloc-warehouse hyperscale (A14), Hoard per-heap-locks (A01).
AA4	Synchronisation	Welcher Concurrency-Mechanismus? Beispiele: Lock-Free CAS (Michael A03, LRMalloc A11), Per-Heap-Locks (Hoard A01), Single-Threaded (Exgen A18), Wait-Free (Crystalline A17).
AA5	Allokationspolicy	Wie werden Allokationsanfragen geroutet? Beispiele: NUMA-Origin-Awareness (NUMAMalloc A09), Cache-Set-Awareness (CAMA A12), NUCA-Awareness (TCMalloc-Warehouse A14), PIM-Awareness (PIM-malloc A16), polymorphe Memory-Resource (std::pmr A22).
AA6	Reklamation	Wie werden gelöschte Blöcke zurückgegeben? Beispiele: Magazine-Cache (Slab A02), Page-Heap-Release (TCMalloc A06), Lock-Free-Reclamation (Crystalline A17), Deferred-Free (mimalloc A04), Vmem-Magazine-Extension (A23).
AA7	Fragmentierungsstrategie	Wie wird Fragmentierung vermieden? Beispiele: dlmalloc Coalescing (A20), Object-Coloring (Slab A02), Buddy-Splitting (A19), Page-Local-Sharding (mimalloc A04), Span-Pooling (scallop A08).

Bausteine

Variante	Beschreibung	Quelle
HoardAllocator (A01)	Per-Heap Free-Lists mit SMP-Skalierbarkeit. Berger et al. (PPoPP 2000). Synchronisierungskonzept via Per-Heap-Locks + lokale Heap-Caches pro Thread.	P— (OSS, Apache-2.0)
SlabAllocator (A02)	Object-Caching-Kernel-Allokator mit Magazine-Cache-Strategie. Bonwick (USENIX 1994, 2001). Klassisches Lehrbuch-Paradigma ohne eigenständigen OSS-Code.	P— (Baseline/Textbook)
MichaelLockFree Allocator (A03)	Lock-Free CAS-Allokator mit Hazard-Pointers. Maged Michael (PLDI 2004). Re-Implementierung mit IBM-Patent-Hintergrund.	P— (MIT Re-Impl)
MimallocAllocator (A04)	Free-List-Sharding mit deferred-free und page-local management. Leijen/Zorn/de Moura (MSR-TR-2019-18, APLAS 2019). Original Code-Linking (is_original=2/2).	P— (OSS, MIT, is_original)
JemallocAllocator (A05)	Arena-basierte Größenklassen mit per-CPU Arenas und tcache Layer. Evans (BSDCan 2006). Standard-Allokator von FreeBSD und Facebook. Original Code-Linking (is_original=2/2).	P— (OSS, BSD-2-Clause, is_original)
TCMallocAllocator (A06)	Thread-Caching Malloc mit zentralem Page-Heap. Ghemawat/Menage (Google ca. 2005) + TEMERAIRE (OSDI 2021). Google-Produktions-Standard.	P— (OSS, Apache-2.0)
SnmallocAllocator (A07)	Message-Passing-Allokator mit asymmetrischen Heap-Operationen. Paul Liétar et al. (ISMM 2019). Original Code-Linking (is_original=2/2).	P— (OSS, MIT, is_original)
ScallocAllocator (A08)	Span-basierte Free-List mit Virtual Spans für Multi-Core-Skalierbarkeit. Aigner/Kirsch/Lippautz/Sokolova (OOPSLA 2015). Low-Fragmentierung via Span-Pooling.	P— (OSS, BSD-3-Clause)
NUMAAllocator (A09)	NUMA-Origin-Awareness mit schnellerer Speicherallokation. UTSASRG (ISMM 2023). Optimierte für Multi-Socket-Architekturen.	P— (OSS, vermutlich Apache-2.0)
RPMallocAllocator (A10)	Per-Thread Span-Caching ohne Dependency zu anderen Allokator-Bibliotheken. Mattias Jansson (2017). Original Code-Linking (is_original=2/2).	P— (OSS, Public-Domain/MIT, is_original)
LRMallocAllocator (A11)	Lock-Free Allokator mit Hazard-Pointers für Multi-Core-Skalierbarkeit. Leite/Rocha (VECPAR 2018, LNCS 11333). Original Code-Linking (is_original=2/2).	P— (OSS, MIT, is_original)
CAMAAAllocator (A12)	Cache-Aware Memory Allocator mit vorhersehbarem Verhalten. Herter/Backes/Haupenthal/Reineke (ECRTS 2011, Saarland). Keine eigenständige OSS-Implementierung.	P— (Baseline/Research)
StarMallocAllocator (A13)	Formal verifizierte härteste Memory-Allokator mit F*/Steel. Inria-Prosecco: Reitz/Fromherz/Protzenko et al. (OOPSLA 2024). Mathematisch verifizierter Allokator.	P— (OSS, Apache-2.0)
TCMallocWarehouse Allocator (A14)	TCMalloc-Hyperscale-Erweiterung mit Huge Page-Unterstützung für Warehouse-Scale-Computing. Hunter (Google ASPLOS 2024). NUCA-aware Variant von TCMalloc.	P— (OSS, Apache-2.0)
HMallocAllocator (A15)	Hybrid Lock-Free Allokator mit skalierbare Speicherverwaltung. Li/Yao/Tang/Lin (IEEE ICPADS 2019). Keine eigenständige OSS-Implementierung.	P— (Research Baseline)
PIMMallocAllocator (A16)	Processing-In-Memory Allokator für heterogene HW mit schnellem lokalen Speicher. VIA-Research (HPCA 2026, arXiv:2505.13002). Spezialisiert auf PIM-Hardware.	P— (OSS, MIT)
CrystallineAllocator (A17)	Wait-Free Memory Allocator mit formalen Guarantees. Solodkyy/Bunkov (PLDI 2021). Garantiert wait-free Reclamation ohne deferred free.	P— (Research)

Variante	Beschreibung	Quelle
ExgenAllocator (A18)	Single-Threaded Specialized Allocator mit Optimierungen für Single-Threaded Workloads. UT Austin (IEEE CAL 2025, arXiv:2510.10219). „Old is Gold“ Prinzip.	P— (Research, Konfidenz medium)
BuddyAllocator (A19)	Buddy-System mit Power-of-2-Splitting für einfache Fragmentierungskontrolle. Knuth (TAoCP Vol.1, 1968). Klassisches Lehrbuch-Paradigma.	P— (Baseline/Textbook)
DlmallocAllocator (A20)	Doug Lea Malloc mit Bins-basierter Klassifizierung. Lea (1987-2012). Original Code-Linking (is_original=2/2).	P— (OSS, Public-Domain/CC0, is_original)
PtMalloc2Allocator (A21)	ptmalloc2 (glibc malloc) mit expliziter Wrapper-Kontrolle. Wolfram Gloger / Ulrich Drepper (1990er-2000er). Lizenz-blockiert (LGPL-2.1+).	P— (OSS, LGPL-2.1+, blockiert Linking)
StdMalloc (A22a)	Standard libc malloc (Concept-Beweis) — portable Fallback-Allokator. Baseline für alle Standard-Bibliotheks-Implementierungen.	P— (Standard-Library)
PmrResourceAllocator (A22b)	std::pmr::memory_resource (polymorphe Memory-Ressource). Halpern (N3916, C++17). Abstrakte Schnittstelle ohne verhaltens-distinkte Default-Implementierung.	P— (Standard-Library, C++17)
PoolResourceAllocator (A22c)	std::pmr::unsynchronized_pool_resource mit eigenem Size-Class-Pool. Halpern (N3916, C++17). Verhaltens-distinkt ohne Vendor-Linking (F15-operativ).	P— (Standard-Library, C++17, F15-operativ)
VmemMagazinesAllocator (A23)	Vmem + Magazines als Erweiterung des Slab-Allocators für Multi-Core-Skalierung. Bonwick (USENIX ATC 2001). Keine eigenständige OSS-Implementierung.	P— (Baseline/Textbook)

Beitragende Paper: P29, P30.

D.8 T7 Prefetch-Strategie

Die Prefetch-Achse bestimmt die Strategie zum vorausschauenden Laden von Knoten und Adressen entlang des erwarteten Such-Pfads in einer Trie-Datenstruktur. Sie unterstützt vier konkrete Strategien mit unterschiedlichen Triggermechanismen und Heuristiken: keine Prefetch (Mess-Baseline), distanzgeschätzte Software-Prefetch auf Basis von Knoten-Dichte und Cache-Latenz (ART-Standard), explizite Hardware-Prefetch-Instruktionen (Wormhole-Standard) und pfadorientierte Prefetch mit Bundle-Granularität (PRT-ART Diplomarbeit).

Statische Sub-Achsen

ID	Name	Beschreibung
PF1	Triggermechanismus	Art der Kontrolle über den Prefetch-Moment: ob kein Mechanismus (Baseline), durch Compiler-Hinweise via Distance-Estimator, durch explizite Hardware-Prefetch-Instruktionen oder durch pfadorientierte Vorhersage
PF2	Distance-Heuristik	Berechnungsweise für die Prefetch-Distanz (wie weit voraus geladen wird): linear, adaptiv basierend auf Knoten-Dichte und Tier-Latenz, oder keine Heuristik
PF3	Granularität	Prefetch-Größe pro Trigger: Cache-Line (64 Byte, einzeln), Page (4K, Betriebssystem-Fenster) oder Bundle (mehrere Cache-Lines, Trie-pfadorientiert)

Bausteine

Variante	Beschreibung	Quelle
NonePrefetch	Keine Prefetch-Strategie (Baseline). Dient als Vergleichs-Referenz für Mess-Reihen. Trigger: keine; Heuristik: keine; Granularität: N/A. <code>is_active()</code> gibt false zurück.	Code/Baseline
DistanceEstimator Prefetch	Distanzgeschätzte Software-Prefetch (ART-Standard nach Leis ICDE 2013). Schätzt Cache-Distance zur nächsten Node basierend auf Knoten-Dichte [%] und Tier-Latenz [cycles] mit Heuristik: <code>estimate = clamp(1 + sparseness*8 + latency_factor, [1,16])</code> . Trigger: Compiler-Hinweise via <code>__builtin_prefetch</code> ; Heuristik: adaptiv (Dichte + Latenz); Granularität: Cache-Line. Setzt auf ART-Pattern (unodb/libart).	P01 (ART/unodb, Leis ICDE 2013)
HardwarePrefetch	Explizite Hardware-Prefetch-Instruktionen (Wormhole-Standard nach Wu EuroSys 2019). Nutzt PREFETCHT0/T1/T2/NTA auf Hash-Anchor-Chain ohne Software-Heuristik; CPU schätzt Distance autonom. Trigger: explizite Hardware-Instruktion; Heuristik: Hardware-managed; Granularität: Hardware-bestimmt. Lizenz GPL-3.0 blockiert <code>is_original-Linking</code> .	P07 (Wormhole, Wu EuroSys 2019)
PathOriented Prefetch	Pfadorientierte Prefetch-Heuristik (PRT-ART Diplomarbeit 2026). Pre-loadet alle Knoten entlang vorhergesagtem Trie-Pfad bevor Suche startet. Tracker verfolgt Suchpfad und empfiehlt nächste Adresse via linearer Schritt-Extrapolation. V11.1 Hot-Path-Hints: interpretiert erste 8 Byte eines Schlüssels als uint64-Adresse. Trigger: pfadorientierte Vorhersage; Heuristik: linear (Schritt-Extrapolation); Granularität: Bundle (mehrere Cache-Lines). Verwandte Literatur: Chen/Gibbons/Mowry SIGMOD 2001 (pB+-Trees Prefetching) + Mowry 1994.	P21 (verwandt: Chen/Gibbons/Mowry SIGMOD 2001); Diplomarbeit 2026

Beitragende Paper: P01, P07, P21.

D.9 T8 Nebenläufigkeits-Synchronisations-Achse (Concurrency Axis T8)

Diese Achse regelt Synchronisationsmuster und sichere Speicherreclamation für nebenläufige Zugriffe auf Baum-/Indexstrukturen. Die Achse stellt compile-time wählbare Synchronisations-Patterns (None/Blocking/Reader-Writer/Optimistic/Lock-Free/Wait-Free) bereit sowie orthogonale Reclamation-Schemata (RCU/Hazard-Pointer) für sichere Speicherfreigabe in lock-freien Strukturen.

Statische Sub-Achsen

ID	Name	Beschreibung
CC1	Synchronisations-Pattern (CC1)	Compile-time Varianten-Familie der Synchronisationsmuster: None (single-threaded Baseline), Blocking (pessimistic Mutex), ReaderWriter (shared_mutex), Optimistic (lock-free Versionsvalidierung), LockFree (CAS-Loops), WaitFree (gebundene Schritte). Jede Familie hat distinkte Laufzeit-Charakteristiken (0-Overhead bis Mutex-Overhead).
CC2	Reclamation-Schema (CC2)	Orthogonale Compile-time Varianten-Familie für sichere Speicherfreigabe in lock-freien Strukturen: RCU (deferred grace-period, via userspace-rcu), Hazard-Pointer (per-thread in-use-Marken). Kombinierbar mit lock-freien Patterns, optional bei Blocking/Optimistic.

Bausteine

Variante	Beschreibung	Quelle
NoneConcurrency	Keine Synchronisation, single-threaded Baseline. Beide acquire() und release() sind no-ops (volatile Dummy). Vergleichs-Nullpunkt für Concurrency-Overhead-Messung (F15).	Code/Baseline
Blocking Concurrency	Pessimistisches Mutex-Locking mit std::mutex. Globaler coarse-grained Lock. Serialisiert alle Zugriffe. Obergrenze des Locking-Overheads. Paper: Dijkstra CACM 1965.	Code/Baseline
ReaderWriter Concurrency	Single-Writer/Multi-Reader via std::shared_mutex. Lock-Sharing bei read-lastigen Workloads. Mittelgewicht zwischen pessimistischen und optimistischen Mustern. Paper: Courtois et al. CACM 1971.	Code/Baseline
OlcOptimistic Concurrency	Optimistic Lock-Coupling (ART-Sync): Versions-Counter pro Knoten, lock-freie Reader mit Retry-on-Conflict. Billig bei niedrigen Konflikten. Paper: Leis DaMoN 2016 / DEBULL 2019 (P08, is_original=true, Apache-2.0, unodb).	P08
LockFree Concurrency	Lock-free via Compare-And-Swap (std::atomic CAS-Loops). System-weiter Fortschritt. Generisches CAS-Pattern ohne Blockierung. Paper: Michael/Scott PODC 1996 (Anker-Paper).	Code/Baseline
WaitFree Concurrency	Wait-free: Jeder Thread schließt in beschränkter Schrittzahl ab (stärkste Garantie). OHNE Retry-Loop wie LockFree. Realisiert via Sequence-Lock / Fetch-Add. Paper: Herlihy TOPLAS 1991 (Theorie-Anker).	Code/Baseline
RcuConcurrency	Read-Copy-Update (McKenney OLS 2001): Lock-freie Reader via per-Thread Nesting-Zähler (RELAXED-Order, KEIN Memory-Barrier auf Read-Side). Reclamation-Schema orthogonal zu Pattern. Lokale Re-Impl (P29 userspace-rcu LGPL-2.1, is_original=false).	P29
HazardPointer Concurrency	Hazard Pointers (Michael TPDS 2004, P30): Pro-Thread markierte in-use-Zeiger verhindern vorzeitige Speicherfreigabe (ABA-safe). Reclamation-Schema. Lokale Re-Impl (P30 NO LICENSE, is_original=false). SEQ_CST-Store auf acquire().	P30
OlcReservedBlocks Concurrency	OLC + Cache-Line-Reservierte Value-Blocks (Multi-Writer Cache-Coherence-Storm-Vermeidung). Spezialisierung des Optimistic-Patterns mit zusätzlicher atomarer Block-ID-Reservierung (fetch_add). F.6 Doku 19 (prt-art-Optional, nicht ist-original).	Code/Baseline

Beitragende Paper: P08, P29, P30.

D.10 T9 Serialisierung (Kodierung und Kompression)

Achse 10 bestimmt, wie Daten innerhalb des Cache-Engine-Systems kodiert und komprimiert werden: vom Raw-Binary-Speicherlayout über variable Längenkodierung bis zur Succinct-Bit-Packing. Die Wahl der Serialisierungsstrategie beeinflusst direkt CPU-Kosten (Delta-Encoding, Bit-Manipulation, LEB128-Varint), Speicher-Overhead und IO-Bandbreitennutzung — kritisch für LSM-Trees und Read-Heavy-Workloads mit Approx-Filter (SuRF/LOUDS).

Statische Sub-Achsen

ID	Name	Beschreibung
SR1	Byte-Order und Wortformat	Wie werden Bytes innerhalb eines Datensatzes angeordnet? Raw (memcpy Roh-Layout), Varlen (LEB128-Varint mit Signalisierungsbits), oder Packed (Bit-für-Bit-Packing)?

ID	Name	Beschreibung
SR2	Dichte (Speicherauslastung)	Speicher-Overhead pro Datensatz: Sparse (vollständige Records), Dense (mit Padding), oder Succinct (Information-theoretisch dicht, $n \cdot H + o(n)$ bits)?
SR3	Kompression und Vorverarbeitung	Block-Kompression (LZ4/Snappy, LZ77-basiert) oder Delta-Encoding (Zigzag-Transform für negative Deltas)? Oder keine Kompression?

Bausteine

Variante	Beschreibung	Quelle
RawBinary Serialization	Roh-Byte-Speicherlayout (memcpy). Baseline ohne Transformation. Minimaler CPU-Aufwand. Typ. für In-Memory-Workloads ohne Kompression. Variante: <code>serieialisierte_scan()</code> liest 32-Bit Felder + Order-sensitive Checksum.	Code/Baseline
VarLen Serialization	Variable-Längen-Kodierung mit Signalisierungsbits (LEB128-VarInt). Standard für ART (Leis ICDE 2013 P01). Kleine Werte = wenige Bytes. Signalisierungsbits markieren Byte-Länge. Datenabhängige Schleife (1-5 Bytes). FNV-1a-Mix der Varint-Bytes.	P01
Succinct Serialization	Bit-für-Bit-Packing mit minimaler Bitbreite (LOUDS/SuRF). Erreicht Information-theoretisches Minimum $n \cdot H + o(n)$ bits. Read-Only nach Build. Optimal für Approx-Filter + Read-Heavy. Höchster Per-Element-CPU-Aufwand (echte Bit-Manipulation). Variante: <code>serialize_scan()</code> mit 64-Bit Akkumulator.	P10,P09
Compressed Serialization	Block-Kompression (LZ4/Snappy) auf Roh-Binary aufgesetzt. Trade-off: CPU-Kosten vs IO/Memory-Bandbreite. Variante: Delta-Encoding gegen Vorgänger + Zigzag-Transform (negatives Delta → kleine unsigned). Typisch für LSM-Trees. Mittlerer CPU-Aufwand.	Code/ LZ77-Based

Beitragende Paper: P01, P09, P10.

D.11 T10 Telemetrie und Messinstrumentierung (axis_11)

Definiert Strategien zur Messung von Index-Operationen (Insert, Lookup, Erase) mit verschiedenen Overhead-Profilen. Unterstützt Density-Tracking für adaptive Node-Typ-Wahl, globale Insert-Counter für Durchsatz-Messungen, HDR-Histogramme für Latenz-Quantile und leaf-only-Counter zur Vermeidung von Cache-Line-False-Sharing in den Messzyklen.

Statische Sub-Achsen

ID	Name	Beschreibung
TM1	Scope (Geltungsbereich)	Gültigkeitsbereich der Zählerverwaltung: leaf-only (nur Blatt-Knoten), per-node (alle Knoten), oder global (einzelner Akkumulator für die ganze Struktur). Compile-Time-Wahl, beeinflusst Overhead durch Konkurrenz-Druck auf gemeinsame Cache-Linien.
TM2	Metrik-Typ (Messgröße)	Art der gemessenen Metrik: counter (Zähler für Ereignisse), histogram (Verteilung mit Quantilen), oder density (Slot-Belegung pro Knoten). Beeinflusst den Speicher-Footprint und die Mess-Granularität.
TM3	Overhead-Level (Messlast)	Größenordnung des Mess-Overheads: low (atomare Operationen), medium (per-node-Datenstrukturen), high (Histogramm-Logging). Bestimmt die Eignung für verschiedene Workload-Szenarien (Benchmarking vs. Produktion).

Bausteine

Variante	Beschreibung	Quelle
LeafOnly Counter	Zähler NUR in Blatt-Knoten (vermeidet Cache-Line-Ping-Pong durch Verzicht auf Inner-Node-Updates). Niedriger Overhead bei leaf-only Scope. Entspricht dem kohärenz-schonenden Zähler-Konzept aus Kuehn DaMoN 2023, obwohl das Original-Paper kein Leaf-Counter-Algorithmus ist — CE-Baseline für False-Sharing-Vermeidung.	P28 (Kuehn DaMoN 2023, Kontext; Code/Baseline)
DensityTracker	Per-Node Density/Slot-Occupancy-Tracking für adaptive Node-Type-Wahl (4→16→48→256). Kontextuelle Telemetrie zur ART-Struktur (Leis ICDE 2013), misst Slot-Belegung aller Knoten. Höherer Overhead als LeafOnly, liefert aber Adaption-Signal für dynamische Rebalancing-Entscheidungen.	P01 (ART/Leis ICDE 2013, Kontext; Code/Baseline)
InsertCounter	Globaler atomarer Insert-Zähler mit minimalem Overhead (1 atomic increment pro Insert). Kontextuelle Telemetrie zur HOT-Struktur (Binna SIGMOD 2018), global Scope. Ausreichend für Durchsatz-Mess-Reihen, keine Laten-Granularität.	P02 (HOT/Binna SIGMOD 2018, Kontext; Code/Baseline)
Latency Histogram	HDR-Histogramm zur Lookup-Latenz-Verteilung (p50/p95/p99 Quantile). Basiert auf HdrHistogram (Gil Tene), CC0-1.0 Lizenz. Höchster Overhead (pro Lookup), liefert aber vollständige Latenz-Statistik für SLA-orientierte und research-grade Messreihen. Einziger is_original-Kandidat der Achse.	HdrHistogram (github.com/HdrHistogram/HdrHistogram_cc0-1.0, is_original)

Beitragende Paper: P01, P02, P28

D.12 T11 Wert-Handle (Value-Handle)

Die Wert-Handle-Achse definiert, wie Datenwerte (Values) im Cache-Index gespeichert und zugegriffen werden: inline im Node-Slot, extern in einem Pool, mittels immutable-shared References (RCU-Style), versioned Pointer (MVCC), oder verkettete externe Referenzen (Multi-Value). Diese Achse charakterisiert die Speicherlayout-Strategie für Values und bestimmt damit die Zugriffslatenz (direkt vs. indirekt) und Concurrency-Semantik (Snapshot vs. Versioning).

Statische Sub-Achsen

ID	Name	Beschreibung
VH1	Speicherort (Storage-Location)	Compile-Time Varianten für die physische Platzierung des Value: inline im Node-Slot (kombiniert Key/Value in einem Slot ohne Indirektion) oder extern in einem separaten Pool (Node hält nur Offset/Pointer).
VH2	Eigentümer-Semantik (Ownership)	Compile-Time Varianten für Ownership und Sharing-Strategie: owned (exclusive per Key), shared (immutable reference-counted via RCU-style), weak (borrowing ohne Increment).
VH3	Versionierung (Versioning)	Compile-Time Varianten für Concurrency-Kontrolle: keine Versionierung (simple Ownership), Version-Tags in Pointer (MVCC für Reader-Snapshot-Isolierung).

Bausteine

Variante	Beschreibung	Quelle
<code>InlineValueHandle</code>	Value wird direkt im Node-Slot eingebettet (keine Indirektion). Standard für kleine Values wie <code>uint64_t</code> . Minimale Zugriffslatenz (1 direkter Read), aber Node-Speicher wird durch größere Values beansprucht. Quelle: ART (Leis ICDE 2013, P01).	P01
<code>ExternalPoolValueHandle</code>	Value extern in Pool gespeichert; Node hält nur Pool-Offset. Ermöglicht variable Value-Größen und kompaktere Nodes, kostet aber 1 zusätzlichen Cache-Miss pro Lookup (Pointer-Chasing). Quelle: Wormhole (Wu EuroSys 2019, P07).	P07
<code>ImmutableSharedRefValueHandle</code>	Value via reference-counted Pointer (RCU-Style Snapshot-Sharing). Optimal für Reader-Concurrency: Leser sehen stabile Snapshots, Writer erstellen neue immutable Snapshots. Requires <code>RefCountAcquire/Release</code> pro Zugriff. Quelle: RCU (McKenney OLS 2001, P29).	P29
<code>VersionedPointerValueHandle</code>	Pointer mit Version-Tag im oberen Byte (MVCC). Ermöglicht Multi-Version-Concurrency-Control: Reader bekommt Snapshot-Version zur Sichtbarkeitskontrolle, Writer erstellt neue Version. Tombstone-Erkennung via Versions-Bit. Quelle: Masstree (Mao EuroSys 2012, P03) und SMART (OSDI 2023).	P03
<code>ChainRefValueHandle</code>	Multi-Value verkettete externe Referenz (Linked-List). Node hält Offset zum Chain-Head im Pool; jeder Chain-Knoten hält (<code>value_offset</code> , <code>next_offset</code>). Teuerste Variante mit 2 abhängigen Derefs pro Value-Zugriff. Quelle: prt-art (lokale Re-Impl, F.6 Migration).	Code/Baseline

Beitragende Paper: P01, P03, P07, P29.

D.13 T12 Instruktionssatz-Architektur (ISA)

Achse T12 spezifiziert die Ziel-CPU-Instruktionssatz-Architektur als Compile-Time-Konstante und ermöglicht damit Cross-Platform-Optimierungen für vier dominante Serverarchitekturen: x86-64 (Intel/AMD), AArch64 (ARM), Power-ISA (IBM) und RISC-V (Open-Source). Jede Variante definiert eine hardware-spezifische SIMD-Baseline (SSE2, NEON, VSX, skalar) und charakterisiert damit die Vektorbeschleunigungspotenziale für Cache-Engine-Operationen wie `simd_field_sum` (Akkumulation von 32-bit-Wort-Feldern).

Statische Sub-Achsen

ID	Name	Beschreibung
IS1	Wortbreite (Word-Size)	CPU-Datenpfadbreite: 32-bit, 64-bit oder 128-bit. In der Cache-Engine auf 64-bit standardisiert für alle vier ISA-Profile.
IS2	CPU-Hersteller/Familie (Vendor)	Instruktionssatz-Familie: <code>intel_amd</code> (x86-64), <code>arm</code> (AArch64), <code>ibm</code> (Power-ISA), <code>open</code> (RISC-V). Bestimmt SIMD-Extension-Kompatibilität: SSE2/AVX2/AVX-512 (x86-64), NEON/SVE/SVE2 (AArch64), VSX (Power), RVV (RISC-V).
IS3	ZIH-Cluster-Zugehörigkeit	Produktionsmaschinen an TU Dresden: Barnard (AMD EPYC 7763 Zen 3, x86-64), Capella (Intel Xeon, x86-64), GraceHopper (NVIDIA Grace Hopper ARMv9 Neoverse V2, AArch64), N/A (Forschungsboards, RISC-V/Power). Ermöglicht Hardware-spezifische Experimente mit echten Produktionscluster-Profilen.

Bausteine

Variante	Beschreibung	Quelle
Amd64Isa	x86-64 / Intel 64 ISA (AMD/Intel Server+Desktop). Dominante Serverplattform mit SSE2-SIMD-Baseline (4 uint32-Lanes pro 128-bit-Register). Implementiert echte SSE2-Vektorisierung via <code>_mm_loadu_si128 + _mm_add_epi32</code> auf nativen x86-64-Hosts; skalarer Fallback auf Nicht-x86-Build-Hosts. ZIH-Präsenz: Barnard (AMD EPYC 7763 Zen 3), Capella (Intel Xeon).	Vendor-Spec (AMD 2000 / Intel fortlaufend)
Aarch64Isa	ARMv8-A 64-bit ISA (ARM Holdings). Wachsende Serverrelevanz (Apple M-Series, AWS Graviton, Ampere Altra). SIMD-Baseline: NEON (128-bit, 4 uint32-Lanes). Cache-Engine-Implementierung: 4-fach entrollter Skalar-Fallback (modelliert NEON-Breite ohne ARM-Intrinsics auf x86-Build-Hosts); echte NEON-Vektorisierung auf nativen ARMv8-Hosts. ZIH-Präsenz: GraceHopper (NVIDIA Grace Hopper, Neoverse V2 + Hopper GPU).	Vendor-Spec (ARM Ltd., ab 2011/2013)
PowerPcIsa	Power ISA (POWER9/10, ppc64le Little-Endian). IBM Power Systems (AC922, IC922, S1022). SIMD-Baseline: VSX (Vector-Scalar Extension, 128-bit, 4 uint32-Lanes). Cache-Engine-Implementierung: 4-fach entrollter Skalar-Fallback (modelliert VSX-Breite ohne Power-Intrinsics auf x86-Build-Hosts). Relevant für HPC-Workloads mit NVLink-GPU-Anbindung (AC922). ZIH-Präsenz: nicht primär, aber für Forschungs-Vergleiche verfügbar.	Vendor/Standards-Spec (IBM/ OpenPOWER, v3.0 2017, v3.1 2020)
RiscVIsa	RISC-V RV64GC Open-Source ISA (RV64 General + Compressed). Wachsende Bedeutung (SiFive, Alibaba Xuantie, Andes, ETH Zürich Cores). SIMD-Baseline: KEINE (RVV ist optional, nicht Teil von RV64GC baseline). Cache-Engine-Implementierung: bewusst plain skalarer Pfad (nicht entrollt) – modelliert fehlende Vektoreinheit der Basis-ISA und erzeugt damit reale, langsamere Laufzeit gegenüber x86-64 und ARM. ZIH-Präsenz: embedded + Forschungs-Boards.	Standards-Spec (RISC-V Intl.; UCB TR ab 2011)

Beitragende Paper: —

D.14 T13 Index-Organisation

Klassifizierung der Datenspeicherung und Indexstrukturierung nach drei orthogonalen Dimensionen: Storage-Order (Heap/Clustered/Ungeordnet), Index-Count (keine/einzeln/mehrfach) und Data-Embedding (separate Speicherung vs. in Leaf-Pages eingebettet). Diese Achse modelliert fundamentale DBMS-Entwurfsentscheidungen, die Cache-Verhalten, Zugriffsmuster und I/O-Charakteristiken auf Basis von Index-Order vs. Storage-Order-Alignment entscheidend beeinflussen.

Statische Sub-Achsen

ID	Name	Beschreibung
io1_storage_order	Storage-Order	Anordnung der Datensätze im Speicher: ungeordnet (Insert-Order im Heap), sortiert nach Primary-Key (Clustered), oder mit Pointer-Indirektion (Non Clustered). Definiert die sequenzielle oder zufällige Zugriffsmuster bei Range-Scans.
io2_index_count	Index-Multiplizität	Anzahl unterstützter Indizes pro Tabelle: keine (Heap-Baseline), Single-Index (Clustered, MySQL InnoDB Primärschlüssel), oder Multiple Secondary-Indizes (NonClustered, SQL Server NONCLUSTERED INDEX).

ID	Name	Beschreibung
io3_data_embedding	Daten-Einbettung	Speicherplatzierung der Daten relativ zum Index: getrennt im Row-Store (Clustered/NonClustered mit Pointer-Hop), oder direkt in Index-Leaf-Pages eingebettet (IOT/Trie-Style wie ART/HOT).

Bausteine

Variante	Beschreibung	Quelle
ClusteredIndex Organization	Speicher-Reihenfolge entspricht Index-Reihenfolge (1 Primary-Key). Optimiert für Range-Scans entlang des Primary-Key mit sequenziellen Disk-Reads (cache-freundlich). Standard: MySQL InnoDB, PostgreSQL CLUSTER, SQL Server CLUSTERED INDEX. Zugriffsmuster: sequenzielle Forward-Iteration ohne Predicate-Evaluation, O(n) Summen-Scan.	Bayer & McCreight, Acta Informatica 1972 (10.1007/BF00288683)
HeapIndex Organization	Storage-Reihenfolge = Insert-Order, KEIN Index (Baseline). Lookup mittels O(n) Full-Scan mit Predicate-Evaluation pro Record (datenabhängiger Branch, kein früher Abbruch). Standard: PostgreSQL Heap-Tabellen, HBase. Zugriffsmuster: ungeordneter sequenzieller Scan mit Vergleichslast und Branch-Misprediction.	Garcia-Molina, Ullman & Widom, Database Systems (Textbook Baseline)
IotIndexOrganization	Daten direkt in Index-Leaf-Pages eingebettet (Oracle IOT, SQL Server CLUSTERED INDEX style). Spart Pointer-Indirektion vs. NonClustered. Äquivalent zu B+Tree-Leaves mit eingebetteten Daten oder Trie-Knoten (ART/HOT/Wormhole). Zugriffsmuster: sequenziell wie Clustered, aber mit Zusatzlast durch Daten im Leaf (zwei Felder pro Record).	Srinivasan, Carey & Livny, VLDB 2000 (Oracle8i IOT)
NonClusteredIndex Organization	Index-Reihenfolge != Storage-Reihenfolge, mehrere Secondary-Indizes pro Tabelle. Lookup über Index + Pointer-Hop zur Daten-Row (Random-Stride, Cache-Misses). Standard: SQL Server NONCLUSTERED INDEX, PostgreSQL CREATE INDEX. Zugriffsmuster: Index-navigation gefolgt von zufälligen Storage-Hops (LCG-determiniert, REAL cache-Miss-Latenz durch TLB/L2-Invalidation).	Comer, ACM Computing Surveys 1979 (The Ubiquitous B-Tree, 10.1145/356770.356776)

Beitragende Paper: Bayer & McCreight 1972, Comer 1979, Srinivasan, Carey & Livny 2000, Garcia-Molina, Ullman & Widom (Textbook).

D.15 T14 I/O-Dispatch-Achse

Modellierung und Auswahl von I/O-Strategien für die Persistierung und den Speicherzugriff im Index: Speicherung im RAM-only (Baseline), gepuffert via OS-Page-Cache (Standard), direkter I/O mit `O_DIRECT`-Flag (NVMe-Optimierung), oder Memory-Mapped File I/O (Persistent Memory). Diese Achse ermöglicht die Entkopplung der Such- und Baum-Strukturen von der Persistierungs-Implementierung und gestattet A/B-Messungen der Dispatch-Overhead-Profile unter identischen Workloads.

Statische Sub-Achsen

ID	Name	Beschreibung
IO1	Persistierungsmodell	Klassifiziert die Speicherart: RAM-only (keine Persistierung, reine Mess-Baseline), oder Disk/Persistent-Memory (Datei-gesicherte Indizes mit verschiedenen Caching-Strategien).
IO2	Caching-Strategie	Charakterisiert die OS-Puffer-Nutzung: Buffered (OS Page-Cache, read-ahead + write-back, Standard), Direct I/O (<code>O_DIRECT</code> , Page-Cache-Bypass, 512B/4KB-Alignment), oder Memory-Mapped (file-backed <code>mmap</code> mit automatischer OS-Pagination und page-fault-getriebenem Zugriff).
IO3	Schreib-Dauerhaftigkeit	Legt die Konsistenz-Garantien fest: keine <code>fsync</code> (höchste Performance, keine Absicherung), <code>fsync</code> nach Batch (Kompromiss, Batch-Commits), oder atomare Schreibvorgänge (höchste Sicherheit, niedrigste Performance). In dieser Implementierung hauptsächlich als konzeptionelle Sub-Achse vorhanden; operative Umsetzung erfolgt über die Caching-Strategie.

Bausteine

Variante	Beschreibung	Quelle
InMemoryOnly	Rein RAM-basierter Index ohne Persistierung. Direkter, sequentieller Record-Zugriff ohne Dispatch-Overhead. Schnellster Pfad und Vergleichs-Nullpunkt der Achse. Verwendet für Mess-Baseline und Performance-Referenzierung unter Bedingungen ohne I/O-Latenz.	Code/Baseline
BufferedIo	Standard <code>read()/write()</code> via OS-Page-Cache mit automatischem read-ahead und write-back. Default für allzweck-DBMS. Trade-off: bequem, aber Double-Buffering (App-Cache + OS-Cache führt zu erhöhter Speichernutzung). Papier-Anker: Stonebraker CACM 1981 (Konzept-/Position-Paper Operating System Support for Database Management, DOI 10.1145/358699.358703). Confidence: medium (Engineering-Pattern, kein Algo-Code).	P-Stonebraker-1981
DirectIo	Bypass des OS-Page-Cache mittels <code>O_DIRECT</code> -Flag (<code>open(2)</code>), NVMe-SSD-optimal. 512B/4KB-Alignment erforderlich. Ermöglicht DBMS-eigene Cache-Strategien (RocksDB, MySQL). Höhere CPU-Kosten, aber keine Cache-Pollution durch Background-Activity. Keine dedizierte Papier-Quelle (Linux-API seit 2.4.10); wird als bewusste Engineering-Baseline behandelt.	Code/Baseline
MmapIo	Memory-Mapped File I/O (<code>mmap</code>) mit Persistent Memory oder Read-Heavy Workloads. OS pagiert automatisch, kein Copy zwischen User- und Kernel-Space. Trade-off: page-fault-Latenz-Spitzen vs. Reuse-Freundlichkeit. Papier-Anker: Crotty et al. CIDR 2022 (Anti-Pattern-Baseline Are You Sure You Want to Use MMAP in Your DBMS?, db.cs.cmu.edu/.../cidr2022-p13-crotty.pdf). Bewusst als Anti-Pattern-Referenz zitiert, um Grenzen und Overhead zu dokumentieren. OSS-Referenz MIT-lizenziert.	P-Crotty-2022

Beitragende Paper: Stonebraker-1981, Crotty-2022

D.16 T15 Migrations-Strategie-Achse

Die Migrations-Strategie-Achse (T15, `axis_migration`) kapselt Entscheidungslogiken zur Daten-Platzierung across mehrschichtige Speicher-Hierarchien (RAM/SSD/HDD, Hot/Cold-Tiers). Sie modelliert, wann und wie häufig zugegriffene Daten zu schnellerem Storage promoviert bzw. selten zugegriffene demoviert werden, ohne echte Tier-Migration durchzuführen (Entscheidungslogik-Messung nur). Unter-

stützt vier konkrete Strategien: static (Baseline), Heat/Cold-Separation, Multi-Tier-Hierarchie, ML-driven Adaptive.

Statische Sub-Achsen

ID	Name	Beschreibung
MG1	Trigger-Mechanismus (MG1)	Bestimmt, wann eine Migrations-Entscheidung auslöst wird: keine (static), Schwellwert-basiert (Threshold), beobachtungsgesteuert (Observation). Compile-Time Variante pro Strategy.
MG2	Migrations-Richtung (MG2)	Bestimmt die Richtung von Daten-Bewegung: keine (static), aufwärts (RAM→SSD), abwärts (SSD→RAM), bidirektional. Compile-Time Variante.
MG3	Migrations-Granularität (MG3)	Spezifiziert die Skalierungseinheit für Migrations-Entscheidungen: keine (static), Page (4KB), Block (64KB), Object (einzelnes Item). Compile-Time Kategorie.

Dynamische Sub-Achsen

ID	Parameter	Beschreibung
D1	Entscheidungs-Schwellwert (Decision Threshold)	Laufzeit-Parameter: Hot/Cold-Klassifikations-Grenzwert (z.B. Recency-Counter-Limit für LRU-basierte Promotions/Demotions). Benutzer-konfigurierbar.
D2	Gewichtungen für Feature-Scoring	Laufzeit-Parameter für AdaptiveMigration: Gewichte (w_{freq} , w_{rec}) für Häufigkeits- und Recency-Features in der ML-Linearkombination. Self-tuning möglich.
D3	Tier-Konfiguration	Laufzeit: Anzahl Speicher-Tiers (z.B. $kTiers=3$ für RAM/SSD/HDD in Tier BasedMigration), Kapazitäten pro Tier, Promotions/Demotion-Schwellen.

Bausteine

Variante	Beschreibung	Quelle
NoMigration	Statische Platzierung ohne Migration (Baseline). Daten werden einmalig positioniert, keine Bewegung zwischen Tiers. Benchmark-Nullpunkt für Vergleich mit aktiven Strategien.	Code/Baseline
HotColdMigration	Heat/Cold-Separation via LRU + probabilistisches Recency-Counting. Häufig zugegriffene Daten (hot) werden zu schnellerem Cache promoviert, selten zugegriffene (cold) zu langsamem Storage demoviert. Standard für LRU-K und LFU-Cache-Systeme.	P-13 (Anti-Caching, PVLDB 2013)
TierBased Migration	Multi-Ebenen-Migration RAM→NVM→SSD→HDD mit Modulo-basiertem Tier-Mapping. Jedes Datenitem wird anhand eines Feldwertes modulo-auf Ziel-Tier gemappt. Standard für RocksDB, LSM-Trees und Tiered-Cache-Systeme mit konfigurierbaren Schwellwerten.	P-11 (Managing NVM in DBMSs, SIGMOD 2018)
AdaptiveMigration	ML-driven adaptive Migration mit Online-Learning (Cachelib/LeCaR-Stil). Gewichtete Linearkombination von Häufigkeits- und Recency-Features berechnet adaptiven Score; gleitender Lern-Akkumulator modelliert Online-Learning. Höherer Overhead vs. optimale Hit-Rate.	P-13b (Driving Cache Replacement with ML-based LeCaR, USENIX HotStorage 2018)

Beitragende Paper: P-13, P-13b, P-11

D.17 T16 Filter-Achse

Die Filter-Achse T16 bietet wählbare probabilistische und deterministische Membership-Query-Filter (Punkt- und Bereichsabfragen). Sie deckt klassische Bloom-Filter mit k Hash-Funktionen, den deletable-Cuckoo-Filter mit Bucket-Hashing und Fingerprints, den succinct Range-Filter SuRF mit LOUDS-Trie-Navigation sowie den platzsparenden Xor-Filter mit konstanter 3er-XOR-Latenz ab. Zweck: Trade-offs zwischen Raum, Abfragelatenz, Mutability und Falsch-Positiv-Rate direkt im Cache-Engine-Experiment messbar machen.

Statische Sub-Achsen

ID	Name	Beschreibung
FT1	Abfrage-Typ	Unterscheidung zwischen Punkt-Abfragen (membership: key enthalten?) und Bereichs-Abfragen (range: keys in $[a,b]$?). Punkt-Filter sind Bloom/Cuckoo/Xor; nur RangeSurfFilter unterstützt Range-Semantik.
FT2	Mutierbarkeit	Unterteilung in unveränderbare Filter (Bloom, RangeSurf, Xor nach Build immutable) vs. Mutable (Cuckoo mit delete-Operation). Bestimmt Design des Wrapper und Cachingmöglichkeiten während Laufzeit.
FT3	Fehler-Profil	Charakterisierung der fehlertoleranten Semantik: False-Positive-Only (Bloom, Cuckoo, Xor — Filter sagt 'ja' falsch), vs. lossless (theoretisch deterministische Antwort). Bloom: approx. $k \cdot n$ Hash-Tests; Xor: exakt 3 XOR-Operationen (branch-frei).

Bausteine

Variante	Beschreibung	Quelle
BloomFilter	Klassischer Bloom-Filter nach Bloom 1970: m -Bit-Bitmap, k unabhängige Hash-Funktionen (double-hashing nach Kirsch/Mitzenmacher 2006). Punkt-Abfragen nur; False-Positives möglich, keine Deletes. Implementierung als ehrlich deklarierte Pseudo-Bitmap-Probe: $k=4$ Hash-Tests pro Query, charakteristische early-exit-Latenz bei Mismatch.	Bloom 1970 (CACM 13:422-426)
CuckooFilter	Cuckoo Filter nach Fan et al. CoNEXT 2014: Bucketed Cuckoo-Hashing mit 1-Byte-Fingerprints. Unterstützt Deletes und Lookup-Counting (Vorteil vs Bloom). Probe-Charakteristik: 2 Bucket-Lookups + Fingerprint-Vergleich (partielle-Schlüssel-Cuckoo nach CoNEXT 2014 §3: $i2 = i1 \text{ XOR } \text{hash}(fp)$). Höhere Insert-Kosten, aber flexible Mutierbarkeit.	Fan et al. CoNEXT 2014 (DOI:10.1145/2674005.2674994)
RangeSurfFilter	SuRF (Succinct Range Filter) nach Zhang et al. SIGMOD 2018: LOUDS-Bitmap-Trie für Range-Abfragen. Einziger Filter mit Range-Semantik (lower_bound/upper_bound auf Byte-Prefix-Ebene). Probe-Charakteristik: mehrstufiger Trie-Abstieg mit succinct-LOUDS-Navigation (bis $k_{\text{MaxDepth}}=6$), nicht exakt 1 Hash-Test, daher latenz-variabel je Prefix-Länge. Praktisch in RocksDB SST-Optimierungen.	P10 (Zhang et al. SIGMOD 2018)
XorFilter	Xor Filter nach Graf+Lemire 2020 (JEA 25): XOR-basierter Filter mit ca. 9 bits/key (vs. 10 bits/key Bloom bei gleicher FP-Rate). Immutable nach Build. Probe-Charakteristik: exakt 3 unabhängige Tabellen-Zugriffe (3 disjunkte Tabellen-Drittel $h0, h1, h2$) + 3er-XOR, branch-frei, konstante Latenz ohne early-exit. Sicherheit gegen Wegoptimierung, kleinster Speicherfootprint aller Filter.	Graf+Lemire 2020 (JEA 25:Article 1.5, DOI:10.1145/3376122)

Beitragende Paper: Bloom 1970, Fan et al. CoNEXT 2014, Zhang et al. SIGMOD 2018, Graf+Lemire JEA 2020.

D.18 T17 Pufferung Q1 (Operations-Puffer)

Die Achse `axis_q1_queuing` realisiert verschiedene Pufferungsstrategien für Operationen im Cache-Engine-Kontext. Sie umfasst Lehrbuch-ADTs (FIFO, LIFO, Append-Only, Priority-Heap), spezialisierte Versioning-Techniken (Delta-Chain, Copy-on-Write, Epoch, Tombstone) sowie moderne Lock-Free-Queues (SPSC/MPMC nach Lamport und Vyukov). Die Strategien werden zur Laufzeit durch eine W2-Registry selektiert und sind klassifizierbar nach Zugriffsmuster (sequenziell, sortiert, zyklisch, versioniert, batched, lock-free).

Statische Sub-Achsen

ID	Name	Beschreibung
QS1	Sequenzieller Zugriff	FIFO/LIFO/Append-Only: Zugriff am Anfang oder Ende, typisch Write-Coalescing + MemTable-Pufferung
QS2	Sortierter Zugriff	Skip-List/Priority-Heap: Sortierter Drain, typisch LSM-Compact und Hot-Key-Eviction
QS3	Zyklischer Zugriff	Bounded Ring-Buffer: Feste Kapazität, Disruptor-Pattern, wrap-around mit Overflow-Handling
QS4	Versionierter Zugriff	Delta-Chain/Tombstone/CoW/Epoch: Multi-Version Concurrency Control, Snapshot-Isolation, Reclamation-Windows
QS5	Batch-Operationen	Batched-Insert-Buffer: Sammlung von Operationen zu Sub-Batches, SIMD-Vectorization, Cache-Locality-Optimierung
QS6	Lock-Free Concurrent	SPSC/MPMC: CAS-Loop oder wait-free Lamport-Modell, echte Lock-Freedom ohne Mutex

Bausteine

Variante	Beschreibung	Quelle
NoBuffer	Passthrough/Identität (Baseline): <code>put()</code> ist no-op, <code>get()</code> liefert immer <code>std::nullopt</code> . Sinnvoll als Vergleichspunkt und in klassischen B+/ART-Szenarien mit direktem Leaf-Zugriff. Family Q01, QS1 <code>sequential_access</code> .	Code/Baseline
AppendOnlyBuffer	Append-Only Linear Buffer (LSM-MemTable, Bw-Tree-Delta): monoton wachsendes <code>std::vector</code> ohne mittiges Löschen. Optimiert für reine Append + Bulk-Drain. Family Q02, QS1 <code>sequential_access</code> . Literatur: Levandoski/Lomet/Sengupta Bw-Tree ICDE 2013, O'Neil LSM 1996.	Code/Baseline
FIFOQueueBuffer	FIFO-Queue (<code>std::deque</code>): First-In-First-Out via <code>deque::push_back / deque::pop_front</code> . Unbounded, kann mittig effizient wachsen. Lehrbuch-ADT, typisch für Write-Coalescing. Family Q03, QS1 <code>sequential_access</code> .	Code/Baseline
LIFOStackBuffer	LIFO-Stack (<code>std::vector</code>): Last-In-First-Out via <code>vector::push_back / vector::pop_back</code> . Unbounded, optimiert für Hot-Path-Wiederverwendung und Versions-Tombstones. Family Q04, QS1 <code>sequential_access</code> .	Code/Baseline

Variante	Beschreibung	Quelle
BoundedRingBuffer	Bounded Ring-Buffer (Disruptor-Pattern): Feste Kapazität, zyklischer Zugriff mit wrap-around. Overflow: drop_oldest (default). iterable_aspect_t: Capacity als Laufzeit-Loop. Family Q05, QS3 cyclic_access. Literatur: LMAX Disruptor 2011.	LMAX 2011
PriorityHeapBuffer	Priority-Heap (std::priority_queue): Max-Heap, get() liefert höchste Priorität zuerst. Erste Strategie mit supports_priority_ordering()=true. Unbounded, Lehrbuch-ADT (Williams 1964). Anwendung: LRU-Approximation, Hot-Key-Promotion, Eviction-Queues. Family Q06, QS2 ordered_access.	Code/Baseline
DeltaChainBuffer	Delta-Chain Buffer (Bw-Tree, Levandoski 2013): Erste versioned-Strategie (is_versioned()=true). Jeder put() hängt Delta-Record mit aufsteigender Version-ID an Spitze der Chain an. get() LIFO-Drain mit Versions-ID. Append-Versioning-Paradigma. Family Q07, QS4 versioned_access.	P04 (Bw-Tree ICDE 2013)
SkiplistBuffer	Skip-List Buffer (sortierter Buffer, RocksDB-Stil): put() fügt in sortierter Reihenfolge ein via std::set.get() ASCENDING-drain (minimum first), typisch für LSM-Compact-Output. Unterschied zu PriorityHeapBuffer: Skiplist=ascending, Heap=descending. Literatur: Pugh CACM 1990. Family Q08, QS2 ordered_access.	Pugh 1990, RocksDB
TombstoneBuffer	Tombstone-Marker Buffer (LSM + MVCC): put() erzeugt Live-Eintrag, deleted-Einträge bleiben als Tombstones (std::nullopt) bis Compact-Lauf. get() überspringt Tombstones FIFO-artig, is_versioned()=true (Erase-Versioning, anders als Delta-Chain Append-Versioning). size() zählt nur Live-Slots. Family Q09, QS4 versioned_access. Literatur: O'Neil LSM-tree 1996.	O'Neil 1996
CopyOnWriteBuffer	Copy-on-Write Snapshot-Buffer (Persistent Data Structures): put() / get() inkrementiert Snapshot-Version, frühere Snapshots bleiben logisch erhalten via shared_ptr. Snapshot-Versioning-Paradigma (anders als Delta-Chain Append, Tombstone Erase). is_versioned()=true. Family Q10, QS4 versioned_access. Literatur: Driscoll/Sarnak/Sleator/Tarjan JCSS 1989.	JCSS 1989, STOC 1986
EpochBuffer	Epoch-based Buffer / QSBR (Quiescent-State-Based-Reclamation): Einträge in aktueller Epoche akkumuliert. Nach epoch_threshold Übergang => frühere Epoche kann reclaimed werden. iterable_aspect_t: epoch_threshold als Laufzeit-Parameter. is_versioned()=true (Reclamation-Window-Versioning). Family Q11, QS4 versioned_access. Literatur: McKenney RCU OLS 2001, Masstree EuroSys 2012, SMART OSDI 2023.	P29 (RCU OLS 2001), Masstree 2012
BatchedInsertBuffer	Batched-Insert Buffer (OLAP-Indexing, ART-Bulk-Insert): put() puffert in Sub-Batches der Größe batch_size_, vollständige Batches werden übergeben (bulk-Drain). Ziel: amortisierter Insert-Cost durch Bulk-Operations, SIMD-Vectorization, Cache-Locality. Erste QS5-Strategie. Family Q12, QS5 batched_access. Literatur: Leis ART ICDE 2013 §6.	P01 (Leis ART ICDE 2013)
LockFreeSPSCBuffer	Lock-Free SPSC Ring-Queue (Lamport 1983): Single-Producer/Single-Consumer. put() wait-free (1 atomic.store + 1 atomic.load), get() wait-free. Kein CAS nötig dank disjoint Modified-Sets. ProgressGuarantee: LockFree (sogar wait-free). iterable_aspect_t: Capacity. Overflow: dropping. Pflicht-Vertrag: genau 1 Producer + 1 Consumer. Family Q13a, QS6 lock_free_access.	Lamport 1983
LockFreeMPMCBuffer	Lock-Free Bounded MPMC Queue (Vyukov 2010-11): Multi-Producer/Multi-Consumer via per-cell atomic Sequence-Number und CAS-Loop. Cell.sequence == pos (Producer schreibt), dann pos+1 (Consumer liest). Retries bei Contention (wait-free pro Op NICHT garantiert). Capacity MUST Power-of-2. iterable_aspect_t. Family Q13b, QS6 lock_free_access. is_original_eligible=true (Vyukov BSD-2).	Vyukov 1024cores 2010-11, Michael/Scott PODC 1996

Variante	Beschreibung	Quelle
OriginalLockFree MpmcConcurrent Queue	moodycamel::ConcurrentQueue (Cameron Desrochers, Block-Based MPMC): Habich-Compliance-Wrapper mit Original-Code-Linking (2/6 Methoden original: put, get; 4/6 Lücken: emplace, peek_front, peek_back, clear sind Re-Impl). is_original_module()=false wegen Lückenfüllungen. Header-only, BSD-2/BSL-1.0 dual. Analog LockFreeMPMCBuffer, aber mit echter Submodule-Bindung. Family Q15, QS6 lock_free_access.is_original_eligible=true (moodycamel BSD-2).	moodycamel 2014, BSD-2/BSL-1.0

Beitragende Paper: LMAX 2011, P01 (Leis ART ICDE 2013), P04 (Bw-Tree ICDE 2013), P29 (RCU OLS 2001), Lamport 1983, Vyukov 1024cores 2010-11, Michael/Scott PODC 1996, Pugh CACM 1990, O’Neil LSM-tree 1996, JCSS 1989, STOC 1986, Masstree EuroSys 2012, SMART OSDI 2023, Williams 1964, moodycamel 2014.

D.19 T18 Pufferung Q2 (Flush-Policy)

Die Achse Q2 definiert automatische Spülstrategien für Schreibpuffer. Sie entscheidet, WANN ein gefüllter Buffer sein Ausgabeergebnis an die nächste Stufe propagiert — nach je einer Operation (Eager), gar nicht (Lazy), bei Schwellwertüberschuss (Watermark), nach Zeitfenster (Timed) oder adaptiv nach Workload-Häufigkeit (EWMA-gesteuert LSM-Stil). Dies ist zentral für den Speicher-/Durchsatz-Tradeoff und die Batching-Effizienz in Cache-Engines.

Statische Sub-Achsen

ID	Name	Beschreibung
FS1	Ereignisgetriggert (Event-Triggered)	Flush wird durch explizite Ereignisse (put/get-Operation oder Eviction) ausgelöst. Umfasst EagerFlush (nach jeder Operation) und LazyFlush (nur bei Eviction).
FS2	Schwellwertgetriggert (Threshold-Triggered)	Flush wird ausgelöst, wenn die Pufferfüllung einen Schwellwert (Default 75%) überschreitet. Reduziert unnötige Spülungen bei sparsamem Traffic.
FS3	Zeitgetriggert (Time-Triggered)	Flush wird periodisch nach einem Zeitfenster (Micro-Batching) ausgelöst, unabhängig von Pufferfüllung. Klassisches Hadoop/Spark/Kafka-Pattern für bounded-latency Guarantees.
FS4	Adaptiv (Adaptive-Triggered)	Flush-Schwellwert wird adaptiv aus Workload-Häufigkeit via EWMA gelernt. Hohe Write-Rate → früher spülen (60% Schwelle); niedrige Rate → später spülen (95% Schwelle). Erste Q2-Strategie mit self-tuning.

Bausteine

Variante	Beschreibung	Quelle
EagerFlush	Pessimistische Write-Through-Policy: spült nach JEDER put()-Operation vollständig (FullFlush). Niedrigste Latenz, aber maximale Spülfrequenz und höchster Durchsatzaufwand — klassisches Synchronous-Write-Pattern.	Lehrbuch Write-Through Policy / Code Baseline

Variante	Beschreibung	Quelle
LazyFlush	Optimistische Write-Back-Policy: spült NIE automatisch, nur bei expliziter Eviction aus dem Buffer. Maximales Batching, minimale Spülfrequenz — klassisches Deferred-Write-Pattern. <code>should_flush()</code> gibt IMMER NoFlush zurück.	Lehrbuch Write-Back Policy / Code Baseline
WatermarkFlush	Schwellwert-gesteuerte Flush-Policy (Default 75% Kapazität). Iterable Aspekt mit 5 Varianten (50%, 65%, 75%, 85%, 95%) für Laufzeit-Permutation. Verwandt mit LSM-Tree Memtable-Trigger (O'Neill 1996).	LSM-Tree O'Neill 1996 (verwandt) / Code Implementation
TimedFlush	Zeit-Fenster-basierte Micro-Batching-Policy (iterable Aspekt: 10/100/1000/10000 ms). Inspiriert von Kafka (Net DB 2011) und Spark D-Streams (SOSP 2013) für Bounded-Latency-Guarantees. Unabhängig von Pufferfüllung.	Kafka NetDB 2011 / Spark SOSP 2013 / Code Implementation
AdaptiveLsm Flush	Selbst-tunende adaptive Watermark-Policy mit EWMA über Burst-Rate. Threshold interpoliert zwischen 60% (hohe Write-Rate) und 95% (niedrige Rate). Konzeptuell RocksDB Dynamic Leveled Compaction + O'Neill LSM 1996; EWMA-Logik ist CE-eigen. Erste Q2-Strategie mit <code>is_adaptive=true</code> .	RocksDB Dynamic Leveled Compaction / O'Neill LSM-Tree 1996 / Code Implementation

Beitragende Paper: 10.1007/s002360050048, NetDB 2011, SOSP 2013.

D.20 Build-PG Seitentyp-Achse (PAGE-TYPE)

Die Seitentyp-Achse definiert die Struktur- und Indexierungsstrategien von Branch-Seiten im Trie-Index: von dicht-indiziertem Direktadressierungsindex (DenseByte), über mehrstufige Dichte-Varianten (ExtendedDense, SparsePatricia) bis zu Pfadkollaps-Optimierungen (Redirect, CustomCache) und B+-artige sortierte Seiten (BPlus). Alle Seitentypen sind Cache-Line-bewusst und implementieren das Concept-Interface `PageTypeStrategy` mit garantierter Austauschbarkeit (F15-Permutation).

Statische Sub-Achsen

ID	Name	Beschreibung
PG1	Struktur-Rolle (Branching)	Klassifizierung der Verzweigungsseiten-Architektur: Branch-Seite mit voller Verzweigung, kollabierter Single-Path-Knoten (Redirect) oder spezielle Cache-optimierte Variante.
PG2	Dichte-Klasse (Density)	Grad der Slot-Auslastung und daraus resultierende Indexierungsstrategie: dicht besiedelt (0-256 Slots), erweitert (>256 Slots) oder sparse mit Patricia-Kompression.
PG3	Pfad-Kollaps-Strategie	Handhabung eindeutiger Restpfade: keine Kompression (Standard), oder Kollaps durch Redirect/CoCo-Technik zur Speicherminimierung.

Bausteine

Variante	Beschreibung	Quelle
DenseBytePageType	Dichte byte-indizierte Verzweigungsseite mit direkter 256-Slot-Adressierung (direct-addressed). Seitentyp 1: direkte Abbildung von Schlüssel-Byte auf Slot-Index ohne Umleitung. Quelle: Adaptive Radix Tree (ART), Leis et al. 2013.	P01
ExtendedDensePageType	Erweiterte dichte Seite für >256 Einträge mit mehrstufigem Indexierungsschema (z.B. 2-Level-Lookup oder variable k-Byte-Span). Seitentyp 2: Dichte-Klasse für komplexere Slot-Anordnungen. Quelle: Adaptive Radix Tree Erweiterungen, PRT-ART.	P01 (extended)
SparsePatriciaPageType	Spärlich besetzte Seite mit Patricia-Pfadkompression (Single-Bit-Split). Seitentyp 3: Dichte-Klasse mit Bit-Vektor-Traversal für niedrige Branchingness. Quelle: HOT (Height Optimized Trie, Binna et al. 2018, basierend auf Patricia-Trie, Morrison 1968).	P02
RedirectPageType	Kollabierter eindeutiger Restpfad (Single-Path-Knoten). Seitentyp 4: Pfad-Kollaps-Strategie zur Speicheroptimierung bei monopol eindeutigen Nachfolgersequenzen. Weder klassischer Branch noch Leaf, speichert nur das Suffix bis zum nächsten echten Verzweigungs- oder Wert-Knoten. Quelle: CoCo-Trie (Boffa et al. 2024).	P04
CustomCachePageType	Cache-Line-orientierte Custom-Seite mit explizitem Layout-Design für 64-Byte-Alignment. Seitentyp 5: Struktur-Rolle Branch mit Hardware-bewusster Geometrie. Quelle: PRT-ART Eigenentwicklung mit TLB-Offset + Cache-Line-Aligned Memory Layout (Achse 5).	PRT-ART
BPlusPageType	B+-artige sortierte Verzweigungsseite mit Range-Walk-Traversal. Seitentyp 6: Struktur-Rolle Branch, aber mit sortierter Key-Array-Organisation statt Byte-Indexierung. Quelle: Masstree (Mao et al. 2012), B+-Tree-Familie.	P03

Beitragende Paper: P01, P02, P03, P04, P05, P06, P10, P11, P12, P13, P20, PRT-ART.

D.21 Build-SIMD: SIMD-Erweiterungs-Achse (Hardware-Beschleuniger)

Die Achse `axis_09b_simd_extension` modelliert die verfügbaren SIMD-/Accelerator-Erweiterungen der CPU-Instruktionssatz-Architektur als Sub-Achse zu `axis_09` (Haupt-ISA). Sie ermöglicht Compile-Time-Auswahl zwischen verschiedenen Vektor-Erweiterungen (SSE2/AVX2/AVX-512 für x86; NEON/SVE2 für ARM; RVV für RISC-V) sowie Baseline (Skalar) und GPU-Beschleunigung (CUDA), mit rückwärts-kumulativer Schichthierarchie und hardwarespezifischer Topologie (SIMD-Units pro Sockel, P/E-Core-Zugriff).

Statische Sub-Achsen

ID	Name	Beschreibung
SE1	Vector-Width (Vektorbreite)	Compile-Time-bekannte maximale Vektorbreite der SIMD-Extension in Bits: 0 (keine), 128, 256, 512, oder -1 (skalierbar). Bestimmt Lane-Breite und Parallelisierungsgrad.
SE2	Accelerator-Type (Beschleuniger-Typ)	Kategorisiert die Beschleuniger-Klasse: none (Baseline), simd (Vektor-SIMD), matrix (Matrix-Engine), gpu (GPU), npu (Neural Processing Unit). Influenziert Komposition mit hardware-Topologie (<code>axis_12</code>).
SE3	Compat-Family (Kompatibilitäts-Familie)	Constraint zur Haupt-ISA (<code>axis_09</code>): none (universell), x86, arm, riscv, cuda. Sichert Compile-Time-Validierung: z.B. <code>Sse2SimdExtension</code> kompatibel nur mit <code>Amd64Isa</code> .

Bausteine

Variante	Beschreibung	Quelle
NoSimdExtension	Baseline-Variante ohne SIMD-Beschleunigung. Skalare Fallback-Strategie für alle Permutationen ohne Accelerator. Universal kompatibel zu allen Haupt-ISAs (x86/ARM/RISC-V/PowerPC). Pflicht-Vergleichspunkt.	Code/Baseline
Sse2SimdExtension	Intel SSE2 128-bit Vektor-Extension (x86_64 ABI-Pflicht seit 2003). Nur kompatibel mit Amd64Isa. Baseline auf x86_64; bietet 4x uint32-Lanes/Iteration. Standard auf Cluster-Knoten Barnard/Capella.	Intel ISA Ext Ref 319433-060
Avx2SimdExtension	Intel AVX2 256-bit Vektor-Extension (Haswell+ 2013). Nur x86_64. Rückwärts-kumulativ; umfasst alle SSE+AVX. Default-Optimierungs-Stufe für Server-DBMS. 2 AVX2-Units/Sockel. ZIH-Standard auf Barnard (AMD EPYC Zen 3) und Capella (Intel Xeon).	Intel ISA Ext Ref 319433
Avx512SimdExtension	Intel AVX-512 512-bit Vektor-Extension (Skylake-X+, Ice Lake Server, Zen 4+). Rückwärts-kumulativ; alle SSE/AVX/AVX2 + 15 separate Flags (VNNI für Vector-Indexes, VPOPCNTDQ für Bitmap-Indexes). Nicht auf allen x86_64-CPU's (z.B. Haswell/Raptor Lake disabled). 1 AVX-512-Unit/Sockel; E-Cores kein Zugriff (Intel Hybrid).	Intel ISA Ext Ref 319433-060
NeonSimdExtension	ARM NEON / Advanced SIMD 128-bit (AArch64 ABI-Pflicht ARMv8+). Kompatibel nur mit AArch64Isa. Baseline auf allen AArch64-CPU's (Apple M-Series, AWS Graviton 2/3/4, NVIDIA Grace). 1 NEON-Unit/Sockel; alle Cores zugänglich (big.LITTLE-Architektur).	ARM ARM DDI 0487 / Cortex-A8 TRM
Sve2SimdExtension	ARM SVE2 (Scalable Vector Extension 2) ARMv9-Pflicht (skalierbar 128–2048 bit). Kompatibel nur mit AArch64Isa. ZIH-Cluster Grace Hopper (NVIDIA Grace = ARM Neoverse V2 mit 128-bit SVE2). Skalierbare Lane-Breite; 1 Unit/Sockel; alle Cores.	IEEE Micro 37(2) 2017, DOI:10.1109/MM.2017.35
RvvSimdExtension	RISC-V Vector Extension (RVV v1.0, skalierbar VLEN 128–65536 bit). Kompatibel nur mit RiscVIsa. Optional (nicht Baseline wie SSE2/NEON). Wachsende Relevanz: SiFive Performance P870 (128-bit), T-Head Xuantie C908 (128-bit). Forschungs-Boards an TU Dresden/ZIH.	ACM TACO 2023, DOI:10.1145/3575861 + RVV v1.0 Spec
CudaGh200SimdExtension	NVIDIA CUDA auf Hopper GH200 (Grace Hopper Superchip GPU-Beschleuniger). ZIH-Cluster Alpha Centauri mit NVIDIA HGX. Kompatibel mit AArch64Isa (Grace CPU) UND Amd64Isa (Hopper-Host). NVLink-C2C 900 GB/s zwischen Grace-CPU und Hopper-GPU. Massive parallel (96 GB HBM3 GPU; ~16.896 CUDA Cores). <code>units_per_socket()</code> = -1 (GPU ist separate Device).	IEEE Micro 28(2) 2008 Tesla Paper + NVIDIA GH200 Whitepaper

Beitragende Paper: —

D.22 Build-HW — Plattform-Hardware-Strategie

Definiert die Plattform-Hardware-Familie (CPU-Architektur, SIMD-Capabilities, Cache-Topologie, Memory-Paging) als Compile-Time-Konstanten für alle Cache-Engine-Algorithmen. Die Achse erfasst, welche Plattformen zur Build-Zeit konfigurierbar sind, und liefert statische Properties (Cache-Line-Größe, Page-Größe, SIMD-Bit-Breite, NUMA-Fähigkeit, HugePage-Fähigkeit) für algorithmengerechte Datenstrukturen.

Statische Sub-Achsen

ID	Name	Beschreibung
hw1_simd_family	12.1 SIMD-Familie	Welche Vector-Instruction-Set wird aktiv genutzt? (AVX2, AVX-512, NEON, SVE2, scalar/none). Bestimmt die maximale Vektorbreite für parallele Operationen innerhalb eines Algorithmus.
hw2_cache_level_targeting	12.2 Cache-Level-Targeting	Welcher Cache-Level wird optimiert (L1-aware, L2-aware, L3-aware, HBM-aware)? Entscheidend für Prefetch-Strategien und Speicher-Layout-Entscheidungen.
hw3_numa_strategy	12.3 NUMA-Strategie	Wie wird NUMA-Topologie berücksichtigt (single-node, multi-node, NUMA-aware-pinning, NUMA-balanced)? Beeinflusst die Allocator-Lokalität und Thread-Pinning-Strategie.
hw4_prefetch_hardware	12.4 Prefetch-Hardware-Instruktionen	Welche expliziten Hardware-Prefetch-Instruktionen sind verfügbar (PREFETCH, PREFETCHNTA, PREFETCHW, none)? Kernbestandteil von Distance-Estimator und path-oriented Prefetch.
hw5_atomic_instruction_family	12.5 Atomic-Befehl-Familie	Welche atomaren Read-Modify-Write-Befehle sind verfügbar (CAS, LL-SC, RMW-extended via HTM/XBEGIN/XEND, none)? Fundamentale Basis für Lock-Free-Concurrency und Optimistic-Lock-Coupling.

Bausteine

Variante	Beschreibung	Quelle
X86_64HardwareProfile	Moderne x86-64 Desktop/Server-Plattform (Intel/AMD). Build-Time-Konstanten: cache_line_size=64, memory_page_size=4096, simd_width_bits=256 (AVX2 konservativ), numa_capable=true, huge_page_capable=true. Basis für desktop- und server-seitige Experimente.	Code/Baseline
Aarch64HardwareProfile	ARM64/AArch64 Server und Mobile Plattformen (Ampere Altra, Apple Silicon M-Serie, Linux ARM). Build-Time-Konstanten: cache_line_size=64, memory_page_size=4096, simd_width_bits=128 (NEON Standard), numa_capable=true, huge_page_capable=true. Ermöglicht Cross-Plattform-Vergleiche.	Code/Baseline
GenericHardwareProfile	Konservatives Fallback-Profil für plattform-agnostische Konfigurationen. Build-Time-Konstanten: cache_line_size=64, memory_page_size=4096, simd_width_bits=0 (scalar-only), numa_capable=false, huge_page_capable=false. Lowest-Common-Denominator wenn kein spezialisierter Adapter passt.	Code/Baseline

Beitragende Paper: —

D.23 A-Korpus Allokator-Korpus (A01-A23)

Die Allokator-Achse dokumentiert 25 speicherverwaltungs-Strategien, die das dynamische Speicher-management in Such-Workloads beeinflussen. Sie umfasst klassische Lehrbuch-Allokatoren (Buddy, Slab), industrielle Standards (dlmalloc, jemalloc, TCMalloc, mimalloc, smalloc), spezialisierte Varianten (NUMA-aware, PIM-Malloc, Cache-Aware) und moderne gehärtete Implementierungen (StarMalloc). Alle 25 Wrapper erfüllen das `AllocatorStrategy`-Konzept und ermöglichen systematische Messung des Speichermanagement-Einflusses auf Suchbaum-Performance (F15-Operativität).

Statische Sub-Achsen

ID	Name	Beschreibung
AA1	Free-List-Topologie	Struktur der freien Speicher-Blöcke: Per-Page-Sharding (mimalloc), Per-Thread-Caching (snmalloc, rpmalloc, TCMalloc), Buddy-Baum (Buddy-Allocator), Single-Global-Free-List (dlmalloc), Per-Heap-Locks (Hoard).
AA2	Size-Class-Schema	Klassifizierung von Block-Größen: Slab-pro-Objektgröße (Slab), dlmalloc Bins, jemalloc Size-Classes, scallo Spans, Buddy-Power-of-2.
AA3	Thread-Lokalität	Grad der Thread-Isolation: tcmalloc thread-local-caches, rpmalloc per-thread-spanning, snmalloc message-passing, Hoard per-heap-locks.
AA4	Synchronisation	Concurrency-Mechanismus: Lock-Free CAS (Michael, LRMalloc), Per-Heap-Locks (Hoard), Wait-Free (Crystalline), Mutex-basiert.
AA5	Allokations-Richtlinie	Routing von Allokationsanfragen: NUMA-origin-aware (NUMALloc), Cache-Set-aware (CAMA), NUCA-aware (TCMalloc-Warehouse), PIM-aware (PIM-Malloc).
AA6	Rückgewinnung	Freigabe gelöschter Blöcke: Magazine-Cache (Slab), Page-Heap-Release (TCMalloc), Lock-Free-Reclamation (LRMalloc), Deferred-Free (mimalloc).
AA7	Fragmentierungsstrategie	Vermeidung von Fragmentierung: dlmalloc Coalescing, Object-Coloring (Slab), Buddy-Splitting, Page-Local-Sharding (mimalloc), Span-Pooling (scallo).

Bausteine

Variante	Beschreibung	Quelle
HoardAllocator (A01)	Per-Heap Free-Lists mit SMP-Skalierung. Drei-Ebenen-Struktur: Thread-lokale Heaps, globale Heap für Cross-Thread-Migration, False-Sharing-Reduktion durch segregierte Allocations-Anfragen.	Paper-P00 (Berger/McKinley/Blumofe/ Wilson, ASPLOS 2000, 10.1145/ 378993.379232) + Code/Baseline
SlabAllocator (A02)	Object-Caching Kernel-Allocator. Slab-pro-Objektgröße mit Magazine-Cache pro CPU (Bonwick 1994/2001). Klassische Solaris-Kernel-Implementierung für vorallokierte Objekt-Pools.	Paper (Bonwick USENIX 1994, usenix.org/.../bonwick.html) + Code/ Baseline
MichaelLockFree Allocator (A03)	Lock-Free CAS-Allocator nach Michael 2004. Hazard-Pointer-geschützte Lock-Free-Datenstrukturen, MIT-Re-Implementierung mit IBM-Patent-Hintergrund.	Paper-P30 (Michael, PLDI 2004, 10.1145/996841.996848) + Code/ Baseline
MimallocAllocator (A04)	Free-List-Sharding (3 Free-Lists pro Page). Microsoft Research 2019: deferred_free Hooks, temporal Cadence, Page-lokale Sharding. is_original: MIT-Direktbindung.	Paper-P04 (Leijen/Zorn/de Moura, APLAS 2019, 10.1007/ 978-3-030-34175-6_13) + is_original (github.com/microsoft/mimalloc, MIT)
JemallocAllocator (A05)	Arena-based Size-Class-Allocator. Evans 2006 / Free BSD. Pro-CPU Arenas mit Tcache-Layer für Thread-lokale Caches. Facebook-Standard. is_ original: BSD-2-Clause-Direktbindung.	Paper-P05 (Evans, BSDCan 2006, papers.freebsd.org/.../jemalloc) + is_ original (github.com/jemalloc/jemalloc, BSD-2-Clause)
TCMallocAllocator (A06)	Thread-Cache + Central-Cache + Page-Heap. Google gperftools 2005. Später TEMERAIRE (OSDI 2021) mit Warehouse-Scale-Optimierungen.	Paper (Google Design-Doku ca. 2005; TEMERAIRE OSDI 2021) + Code/ Baseline

Variante	Beschreibung	Quelle
SnmallocAllocator (A07)	Message-Passing-Allocator. Paul Liétar et al. ISMM 2019: Free-Operationen via Lock-Free-Queue an Allocating-Thread, Cross-Thread-Lock-Contention-Vermeidung. is_original: MIT-Direktbindung.	Paper-P07 (Liétar, ISMM 2019, 10.1145/3315573.3329980) + is_original (github.com/microsoft/snmalloc, MIT)
ScallocAllocator (A08)	Span-basiert, Virtual-Spans-Technik. Aigner/Kirsch/Lippautz/Sokolova OOPSLA 2015. Multicore-skalierbar mit niedriger Fragmentierung.	Paper (Aigner et al., OOPSLA 2015, 10.1145/2814270.2814294) + Code/Baseline
NUMallocAllocator (A09)	NUMA-Origin-Aware Allocator. UTSASRG ISMM 2023. Knoten-Hinweis für lokale NUMA-Allokation, Multi-Knoten-Systeme optimiert.	Paper (Yang/Zhao/Liu et al., ISMM 2023, 10.1145/3591195.3595276) + Code/Baseline
RPmallocAllocator (A10)	Per-Thread Span-Caching. Mattias Jansson 2017, OSS ohne Paper. Sonderfall: Pflicht-Init (rpmalloc_initialize, rpmalloc_thread_initialize). is_original: Public-Domain/MIT-Direktbindung.	Code/Baseline (github.com/mjansson/rpmalloc, Public-Domain/MIT) + is_original
LRmallocAllocator (A11)	Lock-Free + Hazard-Pointers. Leite/Rocha VECPAR 2018 (LNCS 11333). Moderner Lock-Free-Allocator mit ABA-Protection. is_original: MIT-Direktbindung.	Paper-P11 (Leite/Rocha, VECPAR 2018, 10.1007/978-3-030-15996-2_17) + is_original (github.com/ricleite/lrmalloc, MIT)
CAMAAAllocator (A12)	Cache-Aware Memory Allocator. Herter/Backes/Hauptenthal/Reineke ECRTS 2011 (Saarland). Vorhersagbare Cache-Behavior, keine öffentliche Code-Referenz.	Paper (Herter et al., ECRTS 2011, 10.1109/ECRTS.2011.10) + Code/Baseline
StarMalloc Allocator (A13)	Formal Verified Hardened Allocator. Inria-Prosecco (Reitz/Fromherz/Protzenko et al.) OOPSLA 2024. F*/Steel-formale Verifikation, Apache-2.0-Lizenz.	Paper (Reitz et al., OOPSLA 2024, 10.1145/3689773) + Code/Baseline
TCMallocWarehouse Allocator (A14)	TCMalloc-Warehouse / Hyperscale. Google Cloud OSDI 2021 / ASPLOS 2024. Hupage-aware Variante, Marketing-Variante ohne eigenständiges Artefakt.	Paper (Hunter et al., OSDI 2021 / ASPLOS 2024) + Code/Baseline
HMallocAllocator (A15)	Hybrid, Scalable, Lock-Free Memory Allocator. Li/Yao/Tang/Lin ICPADS 2019. Hybrid-Ansatz für Skalierbarkeit, keine öffentliche Code-Referenz.	Paper (Li/Yao/Tang/Lin, ICPADS 2019, 10.1109/ICPADS47876.2019.00114) + Code/Baseline
PIMMalloc Allocator (A16)	Processing-In-Memory Allocator. VIA-Research HPCA 2026 (arXiv:2505.13002). Speicherverteilung über Rechenknoten, PIM-Hardware-optimiert.	Paper-P16 (VIA-Research, HPCA 2026, arxiv.org/abs/2505.13002) + Code/Baseline
Crystalline Allocator (A17)	Formally Verified Wait-Free Allocator. Solodkyy/Bunkov PLDI 2021. Garantierte Wait-Free-Progress, formale Verifikation mit Dafny.	Paper (Solodkyy/Bunkov, PLDI 2021) + Code/Baseline
ExgenAllocator (A18)	Single-Threaded Specialized Allocator. UT Austin IEEE CAL 2025 (arXiv:2510.10219). 'Old is Gold'-Optimierungen für Einthread-Workloads, keine öffentliche Code-Referenz.	Paper-P18 (UT Austin, IEEE CAL 2025, arxiv.org/abs/2510.10219) + Code/Baseline
BuddyAllocator (A19)	Buddy System (Power-of-2-Splitting). Knuth TAOCP Vol.1 (1968), CACM 1965. Klassischer Lehrbuch-Allocator, Binärbaum-Splitting ohne OSS-Code-Referenz.	Paper-P19 (Knuth/Carlson, CACM 1965, 10.1145/365628.365655) + Code/Baseline
DlmallocAllocator (A20)	Doug Lea Malloc (Bins-based). Lea 1987-2012, Public-Domain (CC0). Klassiker seit 1987, Bins-Size-Class-Schema, älteste produktive Implementierung. is_original: Public-Domain-Direktbindung.	Paper-P20 (Lea, gee.cs.oswego.edu/.../malloc.html, 1996) + is_original (github.com/ennorehling/dlmalloc, Public-Domain/CC0)
PtMalloc2 Allocator (A21)	ptmalloc2 (glibc malloc). Wolfram Gloger 1990er-2000er. LGPL-2.1+ Copyleft, Linking blockiert (no is_original). Standard-Basis für glibc-basierte Systeme.	Code/Baseline (malloc.de/en/) + License restriction

Variante	Beschreibung	Quelle
PmrResource Allocator (A22a)	std::pmr::memory_resource. WG21 N3916 (2014 → C++17). Polymorphic Memory Resources Standard-Bibliothek, Fallback auf new/delete-Allokator.	Code/Baseline (C++17 Standard Library, open-std.org/.../n3916.pdf) + Baseline
PoolResource Allocator (A22b)	std::pmr::unsynchronized_pool_resource. WG21 N3916 (C++17). Eigener Size-Class-Pool, F15-operativ (erste Wrapper mit eigenem Speicherhaltungsverhalten ohne Vendor-Linking).	Code/Baseline (C++17 Standard Library, open-std.org/.../n3916.pdf) + is_operative
VmemMagazines Allocator (A23)	Vmem + Magazines (Slab-Erweiterung). Bonwick 2001 USENIX ATC. Per-CPU-Magazine für Skalierung, Kernel-Klassiker ohne öffentliche Code-Referenz.	Paper-P23 (Bonwick, USENIX ATC 2001, usenix.org/.../bonwick.html) + Code/Baseline

Beitragende Paper: 10.1145/378993.379232, 10.1007/978-3-030-34175-6_13, 10.1145/996841.996848, papers.freebsd.org/2006/bsdcan/evans-jemalloc/, 10.1145/3315573.3329980, 10.1145/2814270.2814294, 10.1145/3591195.3595276, 10.1007/978-3-030-15996-2_17, 10.1109/ECRTS.2011.10, 10.1145/3689773, 10.1145/365628.365655, 10.1109/ICPADS47876.2019.00114

E Architekturentscheidungen

F Vergleichsinterfaces: `std::map` und `std::vector`

Dieser Anhang führt — ergänzend zu Abschnitt 2.3 — die vollständige Auflistung aller Standard-Interface-Funktionen der beiden Vergleichshüllen `std::map` (geordnet-assoziativ) und `std::vector` (Sequenz) nach dem Stand von C++23 mit ihrer offiziell erwarteten Semantik und ihrer Komplexitäts-Garantie gemäß dem ISO-C++-Standard [ISO24]. Er dient zugleich als zentrale Referenz für die Weiterentwicklung der Cache-Engine: Da die Suchalgorithmus-Hülle ihr Innenleben durch dynamisch austauschbare Achsen-Algorithmen ersetzt (Abschnitt 2.3.3), wird je Standard-Funktion erklärungsbedürftig, welches Verhalten der äußere Vertrag garantiert.

Variadische Abbildung. Ein Template-Parameter stellt die `std::vector`-API bereit, zwei Parameter die `std::map`-API und $N > 2$ eine `std::map<K, std::tuple<...>>` (Abschnitt 2.3.1); die geteilten Funktionen (Iteration, `size/empty/clear`, `swap`, `get_allocator`) tragen in beiden Hüllen dieselbe Semantik.

Zusammengesetzte Funktionen. Einige Funktionen ergeben sich „unter der Haube“ als Komposition primitiverer Operationen — etwa `operator[]` als `lower_bound` mit anschließendem Default-Einfügen bei fehlendem Schlüssel oder `at` als `find` mit Wurf einer `std::out_of_range`-Ausnahme bei Fehlen —; solche Fälle sind in der Spalte „Anmerkung“ gekennzeichnet. Die Spalte „seit“ nennt die einführende Standard-Fassung. Die exakten cache-engine-internen Funktions-Sprünge (*function-handle-hops*) dokumentiert ein eigenes Entwickler-Dokument der Cache-Engine.

Konformitäts-Teilmenge. Das Konformitäts-Gatter (Abschnitt 2.5.2) prüft vor jeder Messung eine Kern-Teilmenge der `std::map`-Operationen, damit nur Vergleichbares verglichen wird; sie ist in der `std::map`-Tabelle als solche gekennzeichnet.

F.1 `std::map` (geordnet-assoziativ)

Tabelle F.1 listet die vollständige Schnittstelle von `std::map<Key, T, Compare, Allocator>` nach C++23. Ein † markiert die Kern-Operationen, die das Konformitäts-Gatter (Abschnitt 2.5.2) vor jeder Messung prüft, damit nur Vergleichbares verglichen wird. Komplexitäten beziehen sich auf $n = \text{size}()$; bei Bereichs-Operationen ist N die Zahl der Eingabe-Elemente.

Tabelle F.1: Vollständige `std::map`-Schnittstelle (C++23)

Funktion	Offizielle Semantik	Komplexität	seit
Konstruktion, Zuweisung, Allokator			

Funktion	Offizielle Semantik	Komplexität	seit
(Konstruktor)	leer; aus Compare/Allocator; Bereich; Kopie/Move (auch allokator-erweitert); initializer_list; ab C++23 from_range	$O(1)$ bis $O(N \log N)$	C++98
(Destruktor)	zerstört alle Elemente und Knoten	$O(n)$	C++98
operator=	Kopier-/Move- /initializer_list-Zuweisung mit Allokator-Propagation	$O(n)$	C++98
get_allocator	Kopie des zugeordneten Allokators	$O(1)$	C++98
Elementzugriff			
at †	Referenz auf den Mapped-Wert; wirft std::out_of_range bei Fehlen	$O(\log n)$	C++11
operator[] †	Referenz; fügt bei Fehlen wertinitialisiert ein (wirft nie)	$O(\log n)$	C++98
Iteratoren			
begin/end †	Iterator auf erstes Element bzw. Ende, in Schlüsselordnung (cbegin/cend ab C++11)	$O(1)$	C++98
rbegin/rend	Rückwärts-Iteratoren (crbegin/crend ab C++11)	$O(1)$	C++98
Kapazität			
empty †	leer-Test (begin() == end()); [[nodiscard]] ab C++20	$O(1)$	C++98
size	Anzahl der Elemente	$O(1)$	C++98
max_size	theoretische Maximalgröße	$O(1)$	C++98
Modifikatoren			
clear †	alle Elemente entfernen	$O(n)$	C++98
insert †	Element(e) ohne Überschreiben (Wert, P&&, Hint, Bereich, initializer_list, Node-Handle ab C++17)	$O(\log n)$; Hint amort. $O(1)$	C++98
insert_range	Eingabe-Bereich einfügen, keine Überschreibung	$O(N \log(n+N))$	C++23
insert_or_ assign †	einfügen oder vorhandenen Wert zuweisen; bool meldet Einfügung	$O(\log n)$	C++17
emplace †	in-place konstruieren, nur falls Schlüssel fehlt (sonst verworfen)	$O(\log n)$	C++11

Funktion	Offizielle Semantik	Komplexität	seit
<code>emplace_hint</code>	wie <code>emplace</code> , positionsnah zum Hint	$O(\log n)$; amort. $O(1)$	C++11
<code>try_emplace†</code>	wie <code>emplace</code> , rührt die Argumente bei vorhandenem Schlüssel nicht an	$O(\log n)$	C++17
<code>erase†</code>	nach Iterator/Bereich/Schlüssel (Schlüssel: 0 oder 1 entfernt)	Iter. amort. $O(1)$; Key $O(\log n)$	C++98
<code>swap</code>	Inhalt und Comparator tauschen; Iteratoren bleiben gültig	$O(1)$	C++98
<code>extract</code>	Element als Node-Handle entnehmen (ohne Kopie/Move)	Iter. amort. $O(1)$; Key $O(\log n)$	C++17
<code>merge</code>	Knoten aus Quell-Map splicen; vorhandene Schlüssel bleiben dort	$O(N \log(n+N))$	C++17
Suche			
<code>find†</code>	Iterator auf den Schlüssel oder <code>end()</code>	$O(\log n)$	C++98
<code>count†</code>	Anzahl (0 oder 1)	$O(\log n)$	C++98
<code>contains†</code>	Mitgliedschaft (bool)	$O(\log n)$	C++20
<code>lower_bound†</code>	erster Schlüssel $\geq$ key	$O(\log n)$	C++98
<code>upper_bound†</code>	erster Schlüssel $>$ key	$O(\log n)$	C++98
<code>equal_range†</code>	Paar <code>[lower_bound, upper_bound)</code>	$O(\log n)$	C++98
Beobachter			
<code>key_comp†</code>	Kopie des Schlüssel-Comparators	$O(1)$	C++98
<code>value_comp</code>	<code>value_compare</code> (vergleicht Paare über den Schlüssel)	$O(1)$	C++98
Nicht-Member-Funktionen			
<code>operator==</code>	gleiche Größe und paarweise gleiche Elemente	$O(n)$	C++98
<code>operator<=></code>	lexikografischer Drei-Wege-Vergleich; synthetisiert <code><=>=></code>	$O(n)$	C++20
<code>std::swap</code>	Spezialisierung; ruft <code>lhs.swap(rhs)</code>	$O(1)$	C++98
<code>std::erase_if</code>	entfernt alle Elemente mit <code>pred==true</code> ; gibt die Anzahl zurück	$O(n)$	C++20
Deduktions-Leitfäden	CTAD aus Bereich/ <code>initializer_list</code> / <code>from_range</code>	— (Compile-Zeit)	C++17

Die vor C++20 eigenständigen Vergleichsoperatoren $< <= > >=$ werden seit C++20 aus `operator<=>` synthetisiert, `!=` aus `operator==` umgeschrieben. Die heterogenen Überladungen von `operator[]`, `at`, `try_emplace` und `insert_or_assign` (transparenter Schlüssel) gehören erst zu C++26 und bleiben hier ausgeklammert; die C++23-Neuzugänge sind genau der `from_range`-Konstruktor, `insert_range` und die transparenten Schlüssel-Überladungen von `erase/extract`.

F.2 `std::vector` (Sequenz)

Tabelle F.2 listet die vollständige Schnittstelle von `std::vector<T, Allocator>` nach C++23 ($n = \text{size}()$). Sämtliche Member sind seit C++20 `constexpr`.

Tabelle F.2: Vollständige `std::vector`-Schnittstelle (C++23)

Funktion	Offizielle Semantik	Komplexität	seit
Konstruktion, Zuweisung, Allokator			
(Konstruktor)	leer/Allocator; $n \times$ Default/Wert; Bereich; Kopie/Move (auch allokator-erweitert); <code>initializer_list</code> ; ab C++23 <code>from_range</code>	$O(1)$ oder $O(n)$	C++98
(Destruktor)	zerstört alle Elemente, gibt Speicher frei	$O(n)$	C++98
<code>operator=</code>	Kopie/Move/ <code>initializer_list</code>	$O(n)$	C++98
<code>assign</code>	Inhalt durch $n \times$ Wert/Bereich/ <code>initializer_list</code> ersetzen	$O(n)$	C++98
<code>assign_range</code>	Inhalt durch Eingabe-Bereich ersetzen	$O(n)$	C++23
<code>get_allocator</code>	Kopie des zugeordneten Allokators	$O(1)$	C++98
Elementzugriff			
<code>at</code>	geprüfter Zugriff; wirft <code>std::out_of_range</code>	$O(1)$	C++98
<code>operator[]</code>	ungeprüfter Zugriff (UB außerhalb des Bereichs)	$O(1)$	C++98
<code>front/back</code>	Referenz auf erstes/letztes Element (UB bei leer)	$O(1)$	C++98
<code>data</code>	Zeiger auf den zusammenhängenden Speicher (nicht bei <code>vector<bool></code>)	$O(1)$	C++11
Iteratoren			

Funktion	Offizielle Semantik	Komplexität	seit
begin/end, rbegin/rend	Vorwärts-/Rückwärts-Iteratoren in Indexordnung (c...-Varianten ab C++11)	$O(1)$	C++98
Kapazität			
empty/size/ max_size	leer-Test, Elementanzahl, Maximalgröße	$O(1)$	C++98
reserve	Kapazität $\geq n$; realloziert und invalidiert; wirft <code>length_error</code> bei $n > \text{max_size}()$	$O(n)$	C++98
capacity	aktuell allozierte Kapazität	$O(1)$	C++98
shrink_to_fit	nicht-bindende Reduktion der Kapazität auf <code>size()</code>	$O(n)$	C++11
Modifikatoren			
clear	alle Elemente entfernen (Kapazität bleibt)	$O(n)$	C++98
insert	vor Position: Wert/ $n \times$ /Bereich/ <code>initializer_list</code> ; verschiebt den Schwanz	$O(n)$	C++98
insert_range	Eingabe-Bereich vor Position einfügen	$O(n)$	C++23
emplace	Element in-place vor Position konstruieren	$O(n)$	C++11
erase	Position/Bereich entfernen, Schwanz nachrücken	$O(n)$	C++98
push_back	Element anhängen (Kopie/Move)	amort. $O(1)$	C++98
emplace_back	in-place anhängen; Referenz-Rückgabe ab C++17	amort. $O(1)$	C++11
append_range	Eingabe-Bereich anhängen	amort. $O(n)$	C++23
pop_back	letztes Element entfernen (UB bei leer)	$O(1)$	C++98
resize	auf n kürzen oder mit Default/Wert auffüllen	$O(n)$	C++98
swap	Inhalt und Kapazität mit anderem Vektor tauschen	$O(1)$	C++98
Nicht-Member-Funktionen			
operator==	gleiche Größe und elementweise Gleichheit	$O(n)$	C++98
operator<=>	lexikografischer Drei-Wege-Vergleich	$O(n)$	C++20

Funktion	Offizielle Semantik	Komplexität	seit
<code>std::swap</code>	Spezialisierung; ruft <code>lhs.swap(rhs)</code>	$O(1)$	C++98
<code>std::erase</code>	entfernt alle Elemente gleich dem Wert; gibt die Anzahl zurück	$O(n)$	C++20
<code>std::erase_if</code>	entfernt alle Elemente mit <code>pred==true</code> ; gibt die Anzahl zurück	$O(n)$	C++20
Deduktions- Leitfäden	CTAD aus Bereich/ <code>from_range</code>	— (Compile-Zeit)	C++17

Die Spezialisierung `std::vector<bool>` packt Wahrheitswerte bitweise: Sie besitzt kein `data()` und keine Zusammenhangs-Garantie, liefert über `operator[]/at/front/back` einen Proxy-Typ `reference` statt `bool&` und bietet zusätzlich `flip()` (auf Vektor- und auf `reference`-Ebene), ein statisches `swap(reference, reference)` sowie eine `std::hash`-Spezialisierung. Die Vergleichsoperatoren verhalten sich wie bei `std::map`.

F.3 Zusammengesetzte Funktionen und Konformitäts-Teilmenge

Mehrere Schnittstellen-Funktionen sind „unter der Haube“ Kompositionen primitiverer Operationen (Tabelle F.3). Für diese Arbeit ist das die zentrale Beobachtung: Sämtliche Map-Zugriffe und -Modifikatoren ruhen auf einem einzigen Ordnungs-Such-Primitiv (`lower_bound/find`); eine Achsen-Implementierung muss daher nur eine korrekte Ordnungs-Suche samt Knoten-Verkettung bereitstellen, der Rest ergibt sich als feste Komposition — genau die Austauschbarkeit, deren Vergleich der zentrale Mess-Akt ist (Abschnitt 2.3.1). Das Konformitäts-Gatter prüft die in Tabelle F.1 mit † markierte Kern-Teilmenge.

Tabelle F.3: Zusammengesetzte Funktionen („unter der Haube“)

Funktion	Zusammensetzung	Anmerkung
<code>map::operator[]</code>	<code>lower_bound</code> + Default-Einfügen bei Fehlen → Referenz	mutiert bei Fehlen; wirft nie
<code>map::at</code>	<code>find</code> + Wurf <code>std::out_of_range</code> bei Fehlen	einzigster werfender Zugriff; mutiert nie
<code>map::insert / emplace</code>	Ordnungs-Suche + Knoten verlinken nur bei fehlendem Schlüssel	kein Überschreiben; <code>emplace</code> kann Argumente bei Treffer dennoch konsumieren
<code>map::try_ emplace</code>	wie <code>emplace</code> , aber Argumente bei Treffer unangetastet	bewahrt move-only- Argumente

Funktion	Zusammensetzung	Anmerkung
<code>map::insert_</code> <code>or_assign</code>	Ordnungs-Suche + Einfügen oder Zuweisen	überschreibt (anders als <code>insert</code>)
<code>map::count /</code> <code>contains /</code> <code>equal_range</code>	reduzieren auf <code>find</code> bzw. <code>lower_bound+upper_bound</code>	bei Unikat-Map stets 0 oder 1
<code>vector::</code> <code>push_back /</code> <code>emplace_back</code>	ggf. Wachstum (Realloc + Umzug) + Konstruktion am Ende	versteckte Kosten + Invalidierung beim Realloc
<code>vector::insert</code> <code>/emplace</code>	ggf. Wachstum + Schwanz verschieben + Einsetzen	zusätzlicher $O(n)$ -Shift
<code>vector::resize</code>	anhängen (Default/Wert) oder Schwanz kürzen	bidirektional; verkleinert die Kapazität nicht
<code>vector::assign</code>	<code>clear</code> + Bereich/ $n \times$ einfügen	Kapazität wird ggf. wiederverwendet

Funktionen, die beide Hüllen dem Namen nach *teilen* — `size`, `empty`, `clear`, die Iterator-Familien, `swap`, `get_allocator` sowie `insert/emplace/erase` —, tragen in der Map Schlüssel- und im Vektor Positions-/Index-Semantik; `operator[]` etwa fügt in der Map ein, ist im Vektor hingegen ein ungeprüfter Indexzugriff. Die exakten cache-engine-internen Funktions-Sprünge (*function-handle-hops*) hinter diesen Verträgen dokumentiert ein eigenes Entwickler-Dokument der Cache-Engine.

Danksagung

Platzhalter – Danksagung folgt.

